CONFLUENT

Confluent Overview for SK Telecom

Junhee Shin(jshin@confluent.io) / Sr. Solutions Engineer / Confluent Korea

July 2026

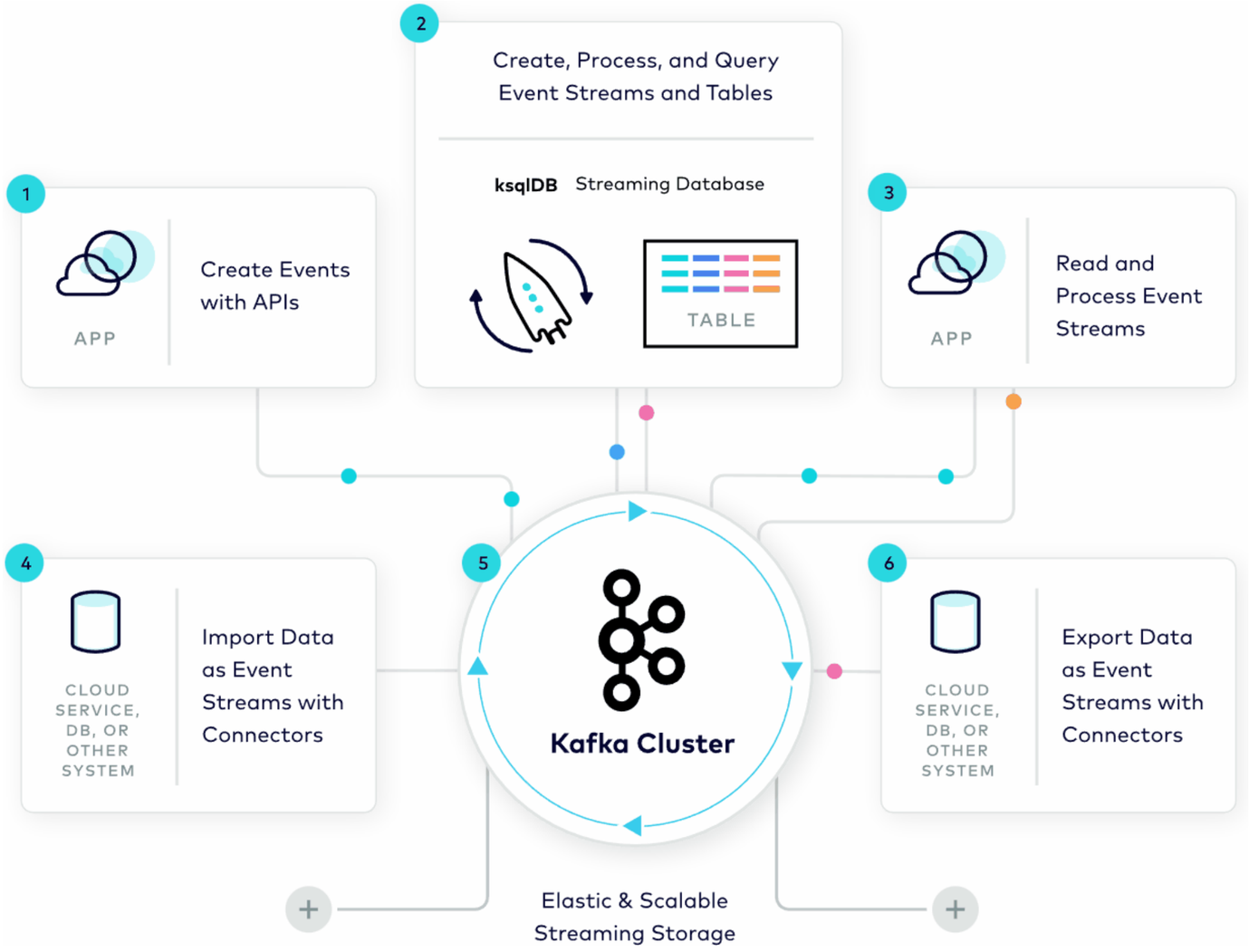


1. 솔루션 소개
2. Confluent Platform Overview
3. 레퍼런스

1. 솔루션 소개

개요: Confluent Solution

What Does Kafka Do?



- **이벤트 스트리밍**
  결제, 주문 및 데이터베이스 변경과 같은 이벤트 스트림을 밀리초 단위로 생성, 가져오기, 공유
- 필요한 만큼 보관
- 타임머신처럼 기록 데이터를 재생하고 재처리

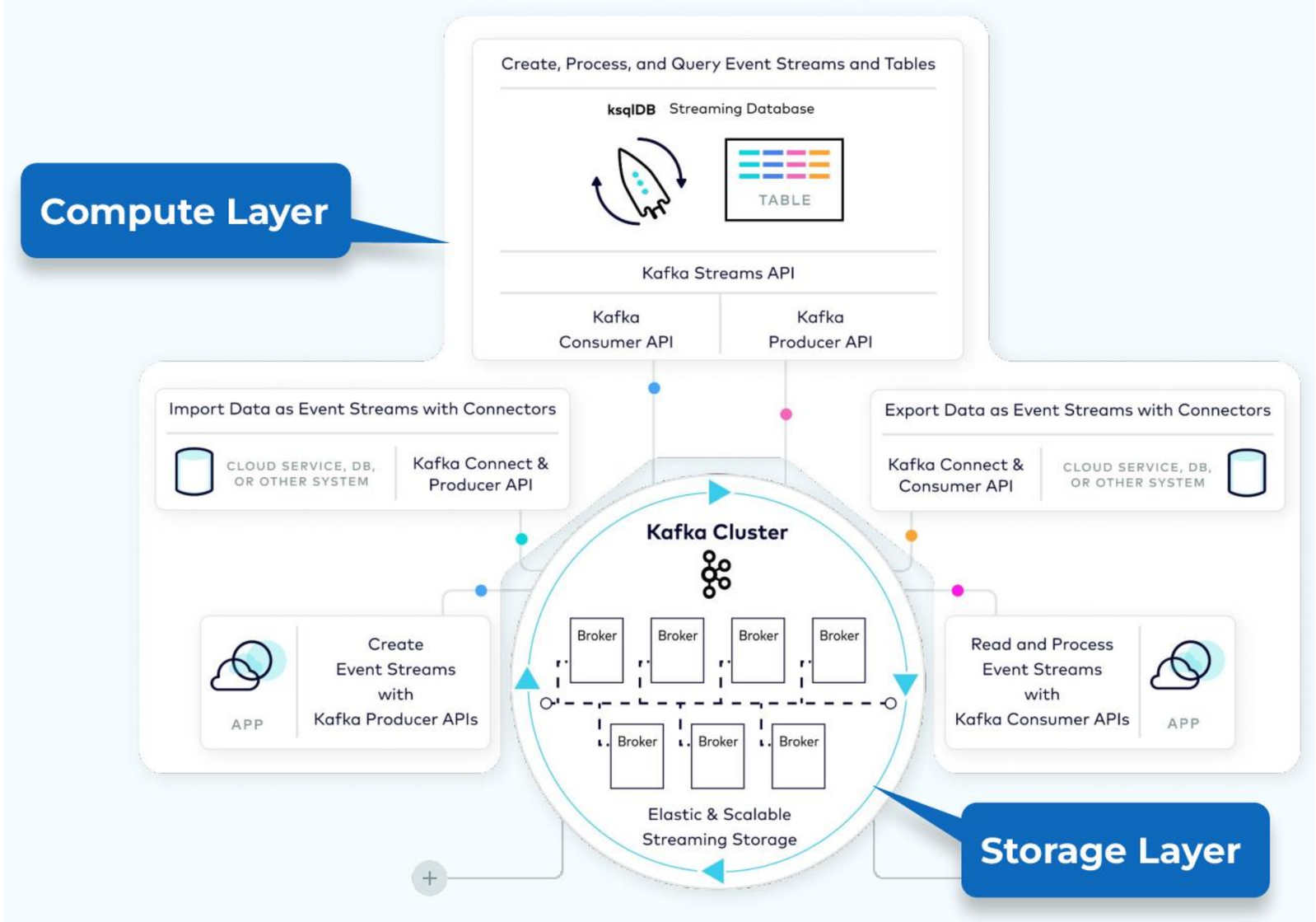
지속적인 실시간 처리

- 이벤트에 즉시 대응하고 실시간 작업 중에 확장할 수 있는 가볍고 탄력적인 애플리케이션 및 마이크로 서비스를 구축
- 실시간으로 데이터 스트림과 테이블을 24x7로 처리, 결합 및 분석 가능

어디서나 데이터 사용 가능

- 단일 플랫폼을 사용하여 전사적으로 데이터를 이동 가능
- 신뢰할 수 있는 동일한 기술로 시스템, 데이터 센터 및 클라우드를 연결
- 최대 규모의 프로덕션 구현은 하루에 수조 개의 이벤트를 처리

Overview of Kafka Architecture



Kafka

- 개발자가 실시간으로 발생하는 새로운 이벤트에 반응할 수 있게 해주는 데이터 스트리밍 시스템

Kafka Architecture

- 스토리지 계층
- 컴퓨팅 계층

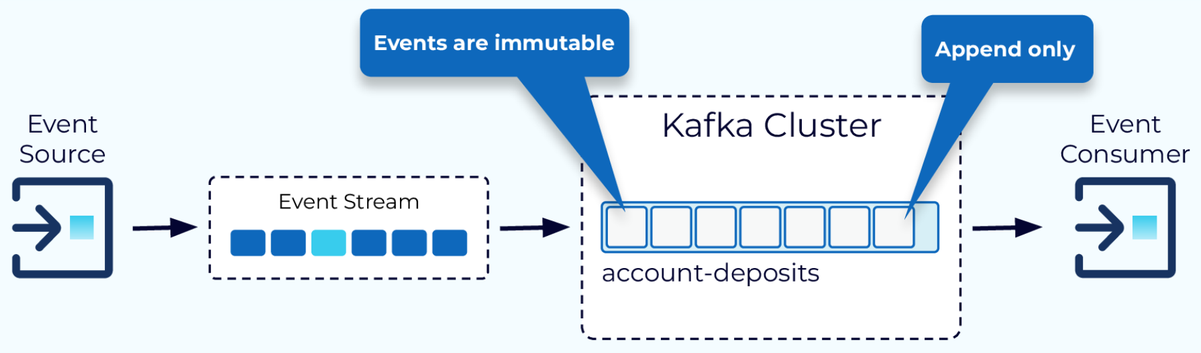
스토리지 계층

- 데이터를 효율적으로 저장하도록 설계되었으며 시간이 지남에 따라 스토리지 요구 사항이 증가하는 경우 확장을 수용하기 위해 시스템을 쉽게 확장할 수 있는 분산 시스템

컴퓨팅 계층

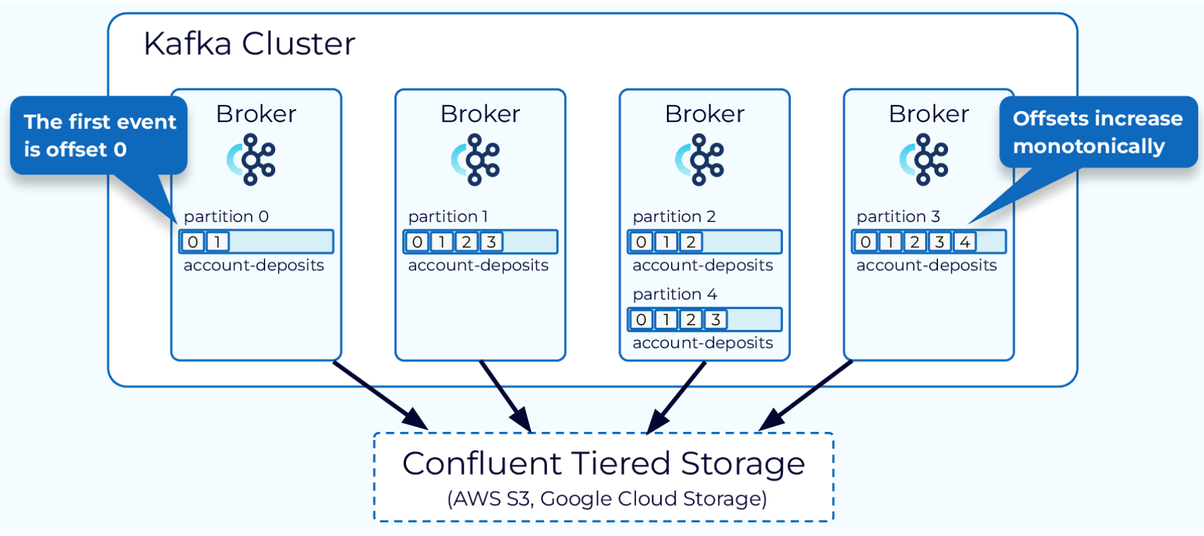
- Kafka가 분산 시스템 전체에서 애플리케이션을 확장할 수 있도록 하는 Producer, Consumer, Stream 및 Connector API의 네 가지 핵심 구성 요소로 구성

Kafka Topic과 Topic Partitions



Topic(Kafka의 핵심 개념)

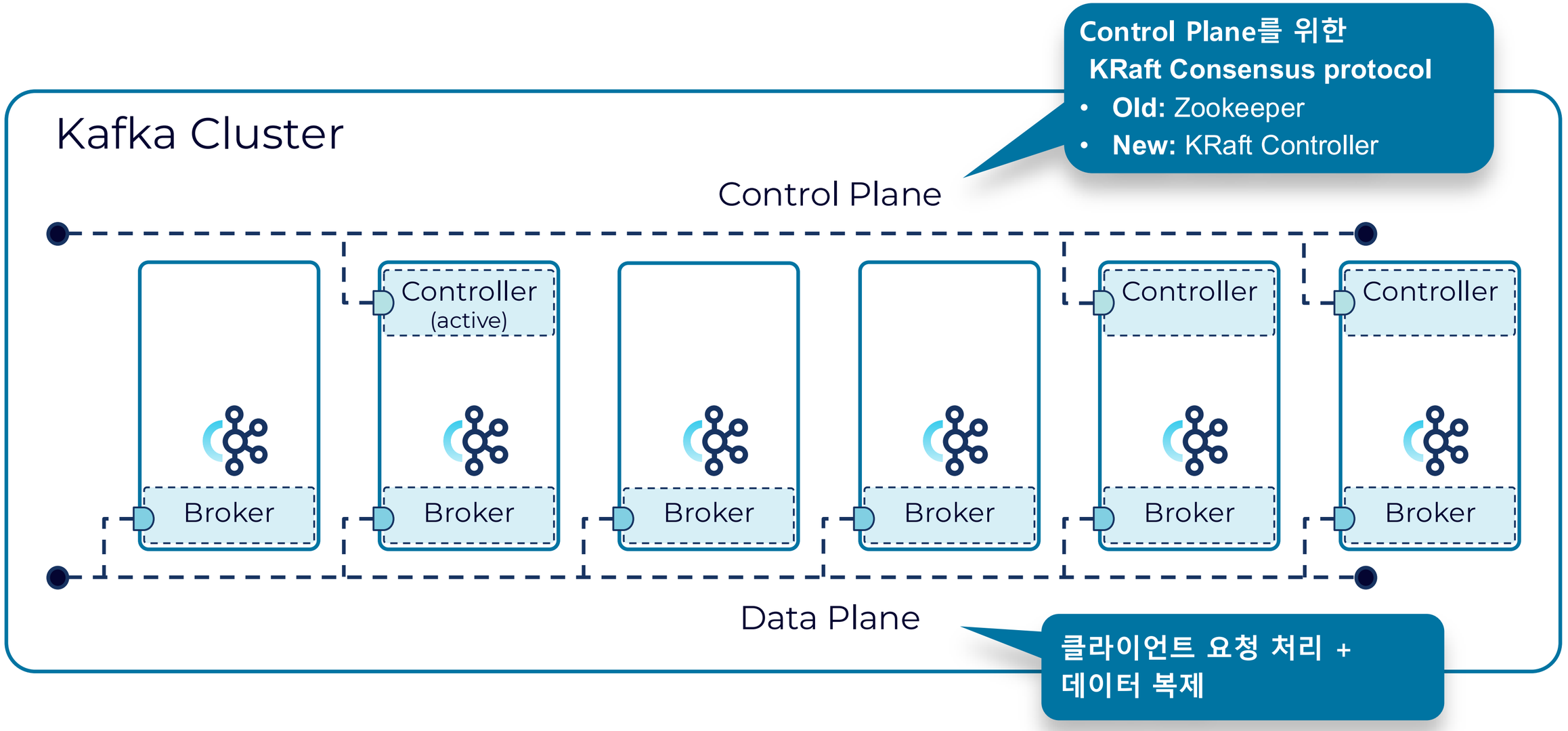
- Kafka 내부에 이벤트(데이터) 저장소
- 추가전용의 불변의 이벤트 로그
- 토픽에서 이벤트의 저장 및 처리를 분산하기 위해 파티션 개념을 사용
- 토픽은 파티션들로 구성되어 Kafka 브로커에 있는 여러 "Buckets"에 분산됨



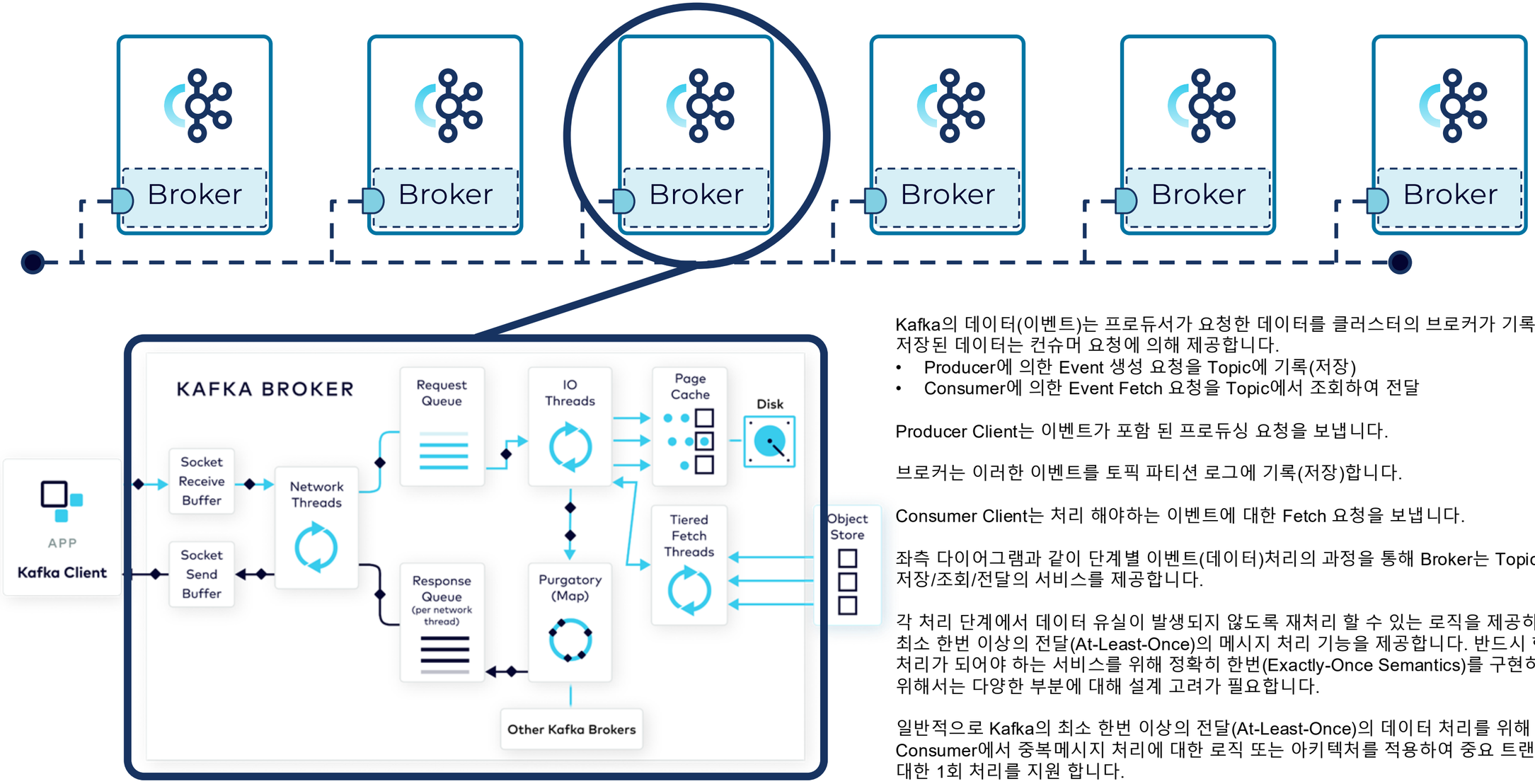
Partition(Commit Log)

- Kafka 이벤트의 기본 스토리지 단위이며 병렬 처리의 기본단위
- 하나의 Topic은 하나 이상의 Partition으로 구성됨
- 이벤트에는 Offset이라는 유니크한 고유 식별자가 지정됨
- 병렬처리(Throughput 향상)를 위해서 다수의 Partition 사용하며 클러스터 노드에 분산됨
- 데이터의 분산 배치로 다수의 Client가 여러 브로커에 동시에 데이터를 Read/Write 가능
- 분산 클러스터 구조로 고가용성과 뛰어난 확장성 제공

Kafka는 데이터 및 메타 데이터를 별도로 관리합니다



Inside The Apache Kafka Broker



Kafka의 데이터(이벤트)는 프로듀서가 요청한 데이터를 클러스터의 브로커가 기록하고, 저장된 데이터는 컨슈머 요청에 의해 제공합니다.

- Producer에 의한 Event 생성 요청을 Topic에 기록(저장)
- Consumer에 의한 Event Fetch 요청을 Topic에서 조회하여 전달

Producer Client는 이벤트가 포함 된 프로듀싱 요청을 보냅니다.

브로커는 이러한 이벤트를 토픽 파티션 로그에 기록(저장)합니다.

Consumer Client는 처리 해야하는 이벤트에 대한 Fetch 요청을 보냅니다.

좌측 다이어그램과 같이 단계별 이벤트(데이터)처리의 과정을 통해 Broker는 Topic을 저장/조회/전달의 서비스를 제공합니다.

각 처리 단계에서 데이터 유실이 발생되지 않도록 재처리 할 수 있는 로직을 제공하여, 최소 한번 이상의 전달(At-Least-Once)의 메시지 처리 기능을 제공합니다. 반드시 한번만 처리가 되어야 하는 서비스를 위해 정확히 한번(Exactly-Once Semantics)를 구현하기 위해서는 다양한 부분에 대해 설계 고려가 필요합니다.

일반적으로 Kafka의 최소 한번 이상의 전달(At-Least-Once)의 데이터 처리를 위해 Consumer에서 중복메시지 처리에 대한 로직 또는 아키텍처를 적용하여 중요 트랜젝션에 대한 1회 처리를 지원 합니다.

Apache Kafka : Event Streaming Platform



01
이벤트 스트림을
Publish & Subscribe



02
이벤트 스트림을
Store



03
이벤트 스트림을
Process & Analyze

Why Kafka vs existing MW

ETL / Data Integration

−

Batch
Time Consuming
Expensive (Ops)

+

High Throughput
Durable

Messaging

−

Difficult to Scale
No Persistence

+

Fast
Low Latency

new Event Streaming Paradigm

+

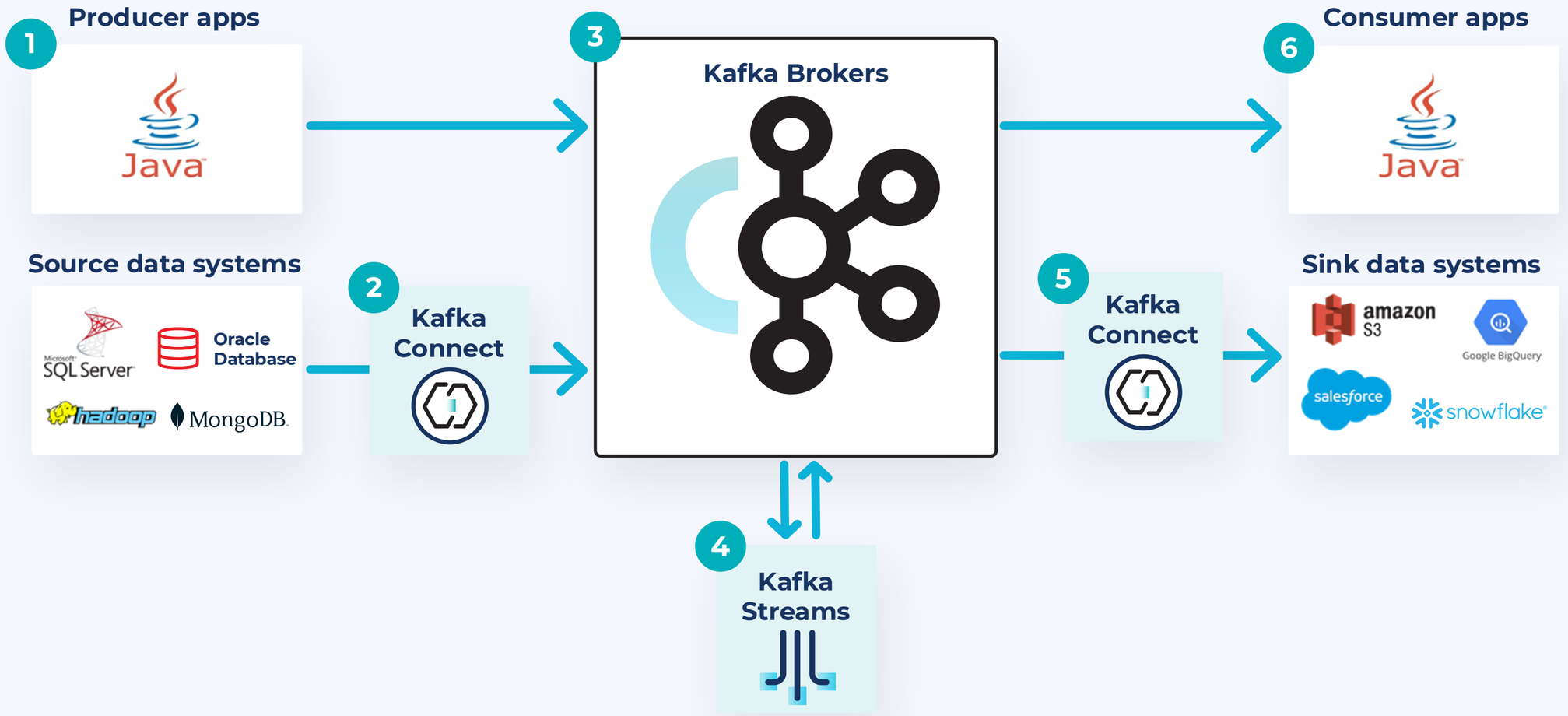
High Throughput
Durable
Persistent
Maintain Order

+

Fast
Low Latency

확장 가능한 메시징 + 지속성 + 실시간 데이터 통합 + 스트림 처리
⇒ Kafka는 이벤트 기반 패턴(약 15만+ 조직에서 사용)에 대한 사실상의 표준이 되고 있습니다.

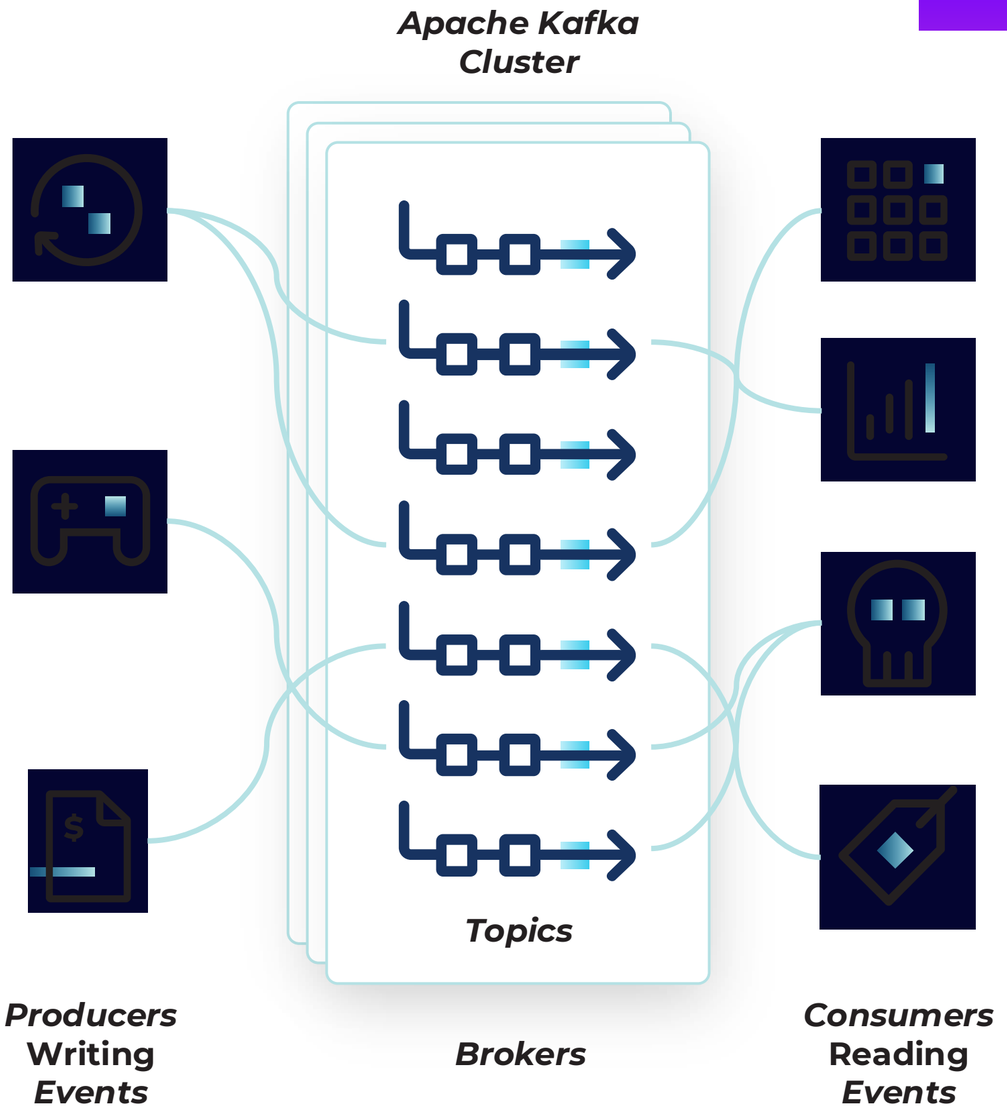
Open-source Apache Kafka at-a-glance



1. 커스텀 앱에서 Kafka에 데이터 쓰기
2. 다른 시스템에서 카프카에 데이터 쓰기
3. 토픽에 데이터 스트림 저장
4. 실시간으로 데이터 스트림 처리
5. Kafka에서 다른 시스템으로 데이터 읽기
6. 카프카에서 커스텀 앱으로 데이터 읽기

Kafka 아키텍처의 장점

- 소비자와 생산자는 분리되어 있으며 서로 통신하지 않습니다(즉, 1:1 의존성/중단점 없음)
- 높은 처리량 실시간 게시/구독 데이터 스트리밍
- 엄격한 순서 지정, 지속성, 되감기 및 재생
- 파일 시스템의 확장성 + 데이터베이스의 보장 + 메시지 큐의 즉시성



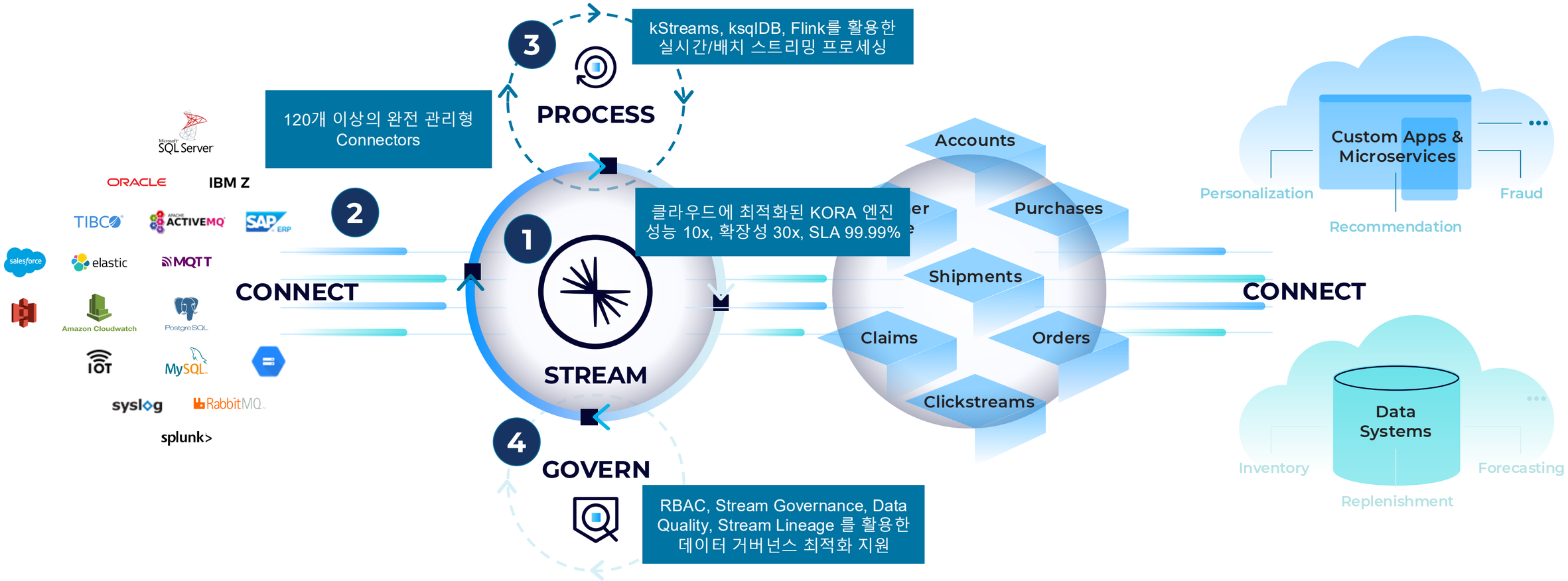
어디서나 앱과 데이터 시스템 전반에 걸쳐 데이터를 스트리밍하여 실시간 데이터를 활용할 수 있습니다



Database Pipelines	Data Warehouse Pipelines	Mainframe Integration	AI/ML Pipelines	SIEM & Observability Pipelines	...

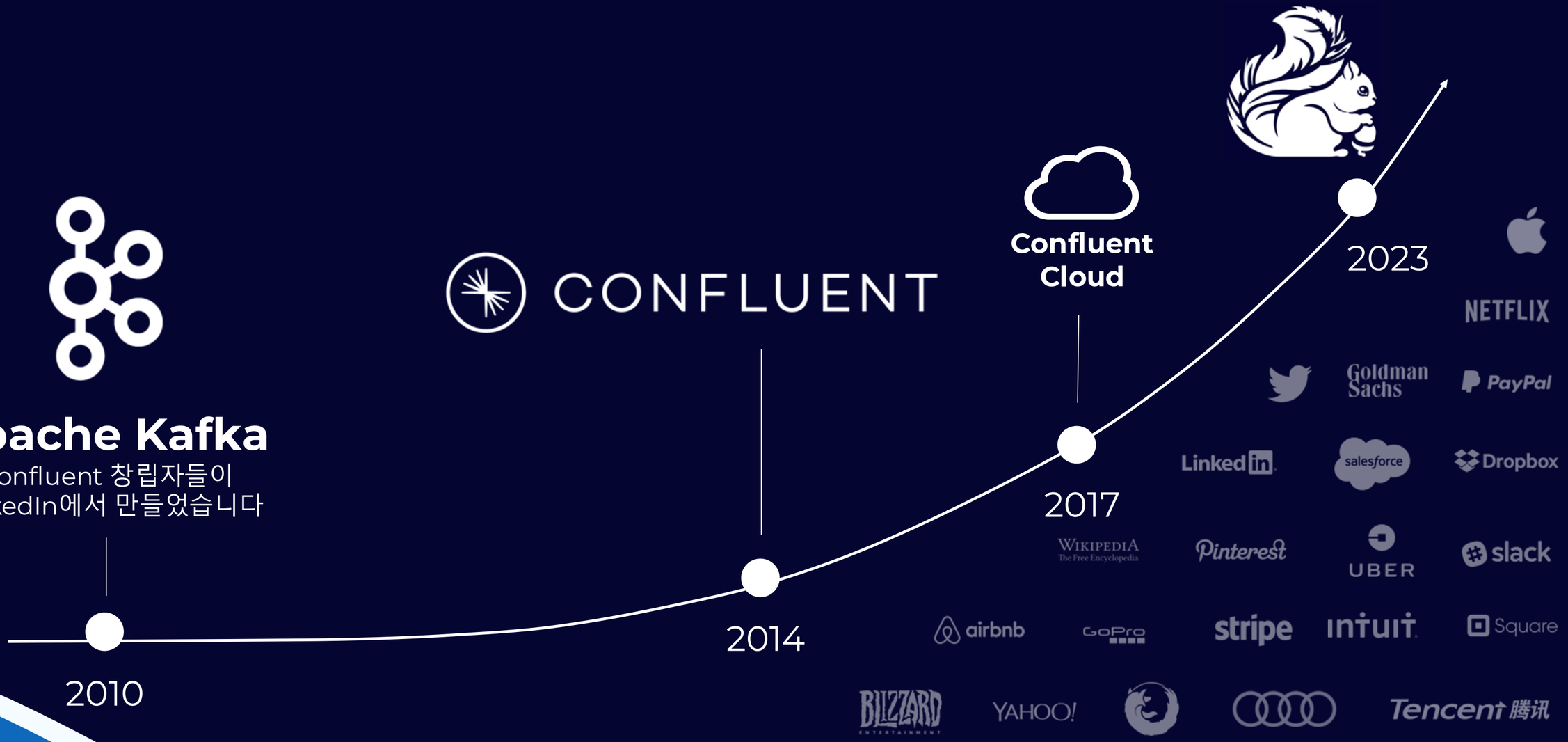
데이터를 움직이기 위한 새로운 패러다임: Data Streaming Platform

모든 데이터는 지속적으로 스트리밍 되고, 처리되고, 관리되고, 제품으로 공유되어
어디서나 즉시 가치 있고, 사용 가능하며, 신뢰할 수 있게 됩니다.



Data Mess로 부터 ⟶ Data Products 으로 ⟶ 어디에서나 즉각적인 가치 제공

이벤트 스트리밍의 부상


Apache Kafka
Confluent 창립자들이
LinkedIn에서 만들었습니다
2010
CONFLUENT
2014
Confluent
Cloud
2017
2023
NETFLIX
Goldman
Sachs
PayPal
LinkedIn
salesforce
Dropbox
WIKIPEDIA
The Free Encyclopedia
Pinterest
UBER
slack
airbnb
GoPro
stripe
INTUIT
Square
BLIZZARD
ENTERTAINMENT
YAHOO!
Tencent 腾讯

아파치 카프카 - 데이터 스트리밍의 사실상 표준



10 OUT OF 10

Travel & Retail



10 OUT OF 10

Banking



10 OUT OF 10

Insurance



10 OUT OF 10

Entertainment



10 OUT OF 10

Technology



10 OUT OF 10

Healthcare



9 OUT OF 10

Auto & Trans.



7 OUT OF 8

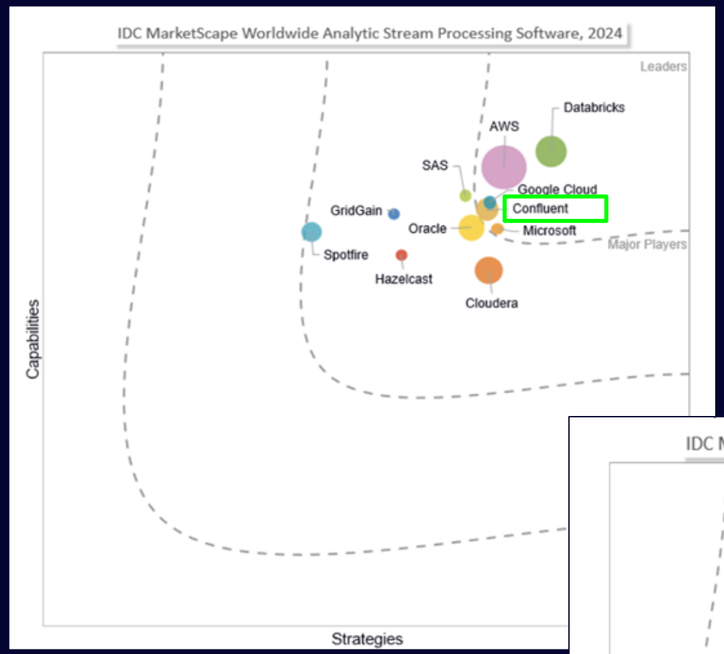
Telco

>80% 포춘 500대 기업 중 카프카를 사용하는 것으로 추정되는 기업 수

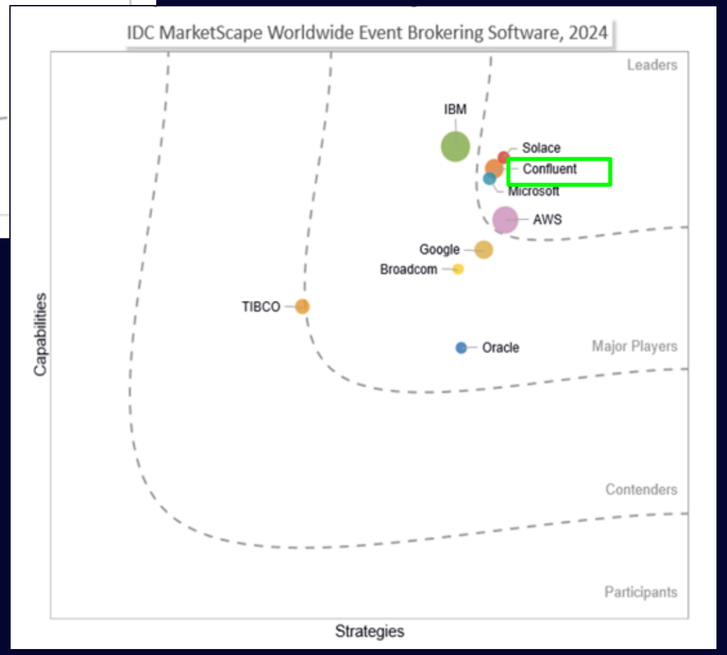
Note: Top company count by industry and >75% are calculated using the 2022 Fortune 500 list, released in August 2022, and are as of March 31, 2023

주요 기술 분석 기관에서 지속적으로 리더로 인정받는 기업

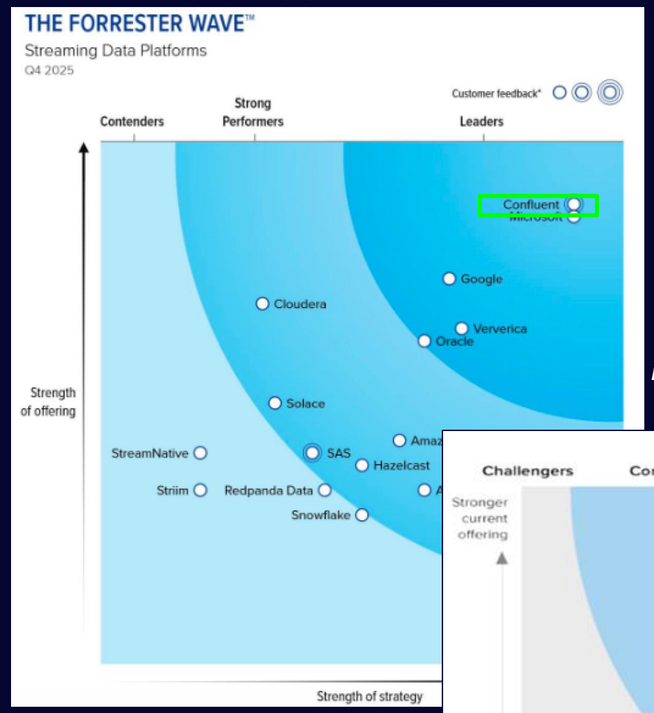
IDC MarketScape:
2024년 전 세계 분석 스트림 처리 소프트웨어



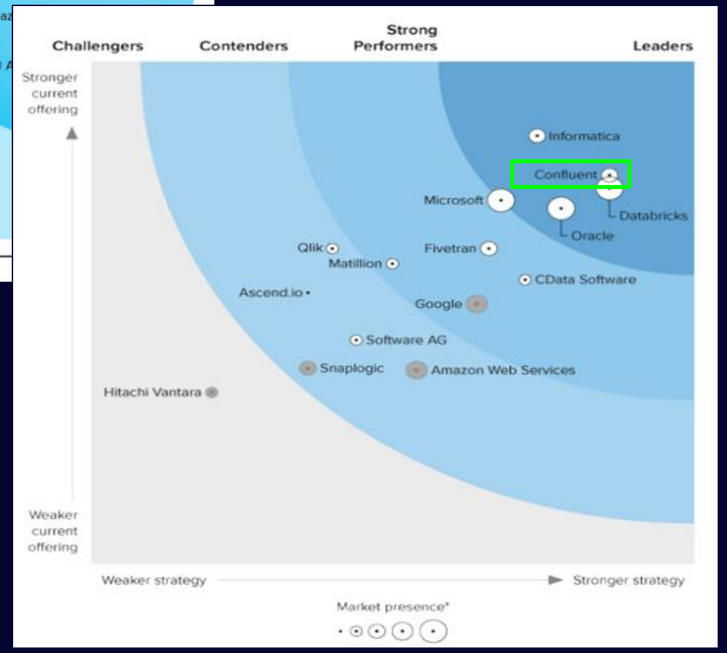
IDC MarketScape:
2024년 전 세계 이벤트 브로커 소프트웨어



Forrester Wave(™):
2025년 4분기 스트리밍 데이터 플랫폼 부문



Forrester Wave(™): 2023년 4분기 클라우드 데이터 파이프라인 부문



Gartner Peer Insights – Event Stream Processing 리뷰 및 평점에서 Banking 산업 최고 평점: https://www.gartner.com/reviews/market/event-stream-processing

Confluent 솔루션의 차별점

Kafka를 직접 운영하는 것은 어렵습니다

오픈소스 Apache Kafka를 운영하는 것은 매우 많은 노력과 시간, 비용이 소요됩니다.
특히 업그레이드 및 장애 처리는 악몽과 같을 수 있습니다.

주요 과제:

운영 부담 및 리소스

지속적으로 증가하는 수요를 지원하기 위해 플랫폼을 관리 및 확장

보안 및 거버넌스

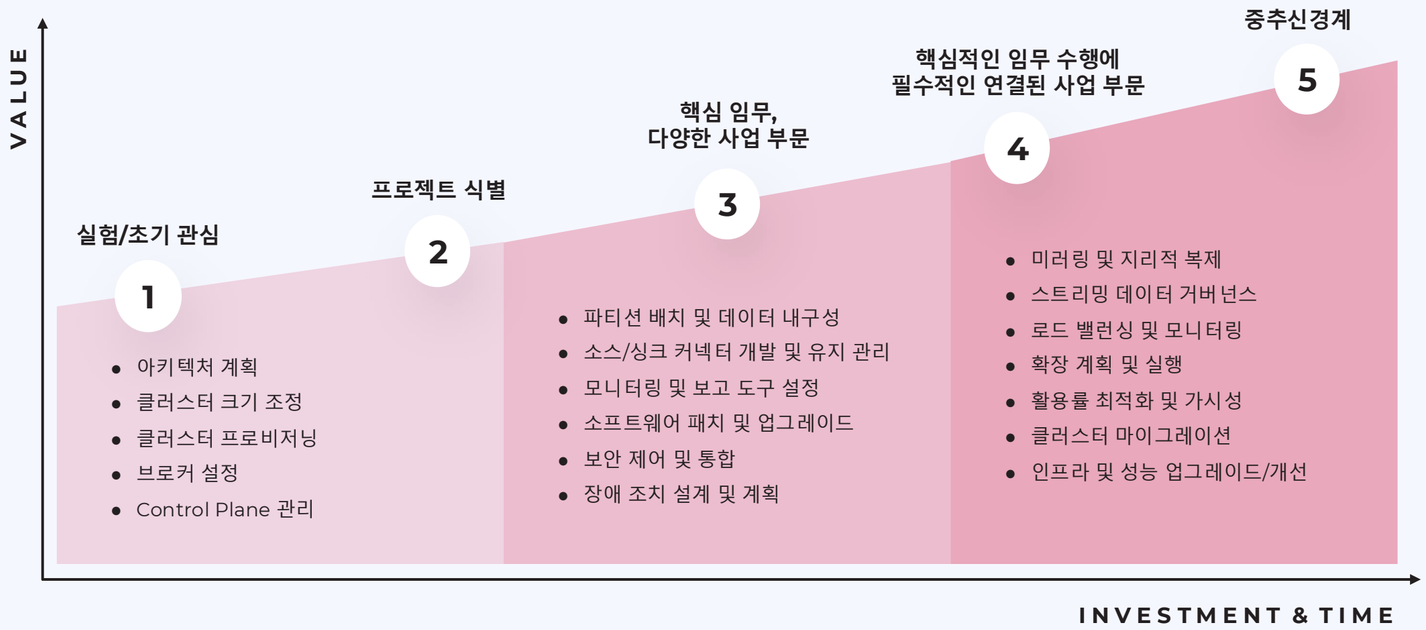
Kafka 사용량이 증가함에 따라 스트리밍 데이터가 저장 데이터만큼 안전하고 보안이 유지되도록 보장

실시간 연결 및 프로세싱

가치 있는 기존 데이터를 활용하여 최신 클라우드 기반 앱 및 사용자 경험을 강화

글로벌 가용성

Maintain high availability across environments
최소한의 다운타임으로 다양한 환경에서 높은 가용성을 유지

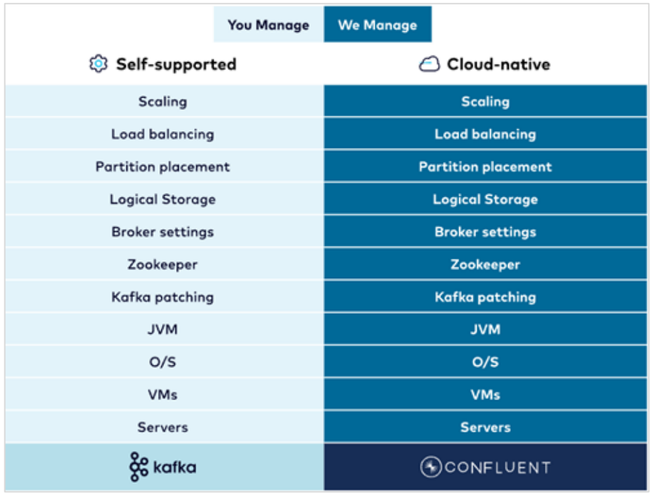


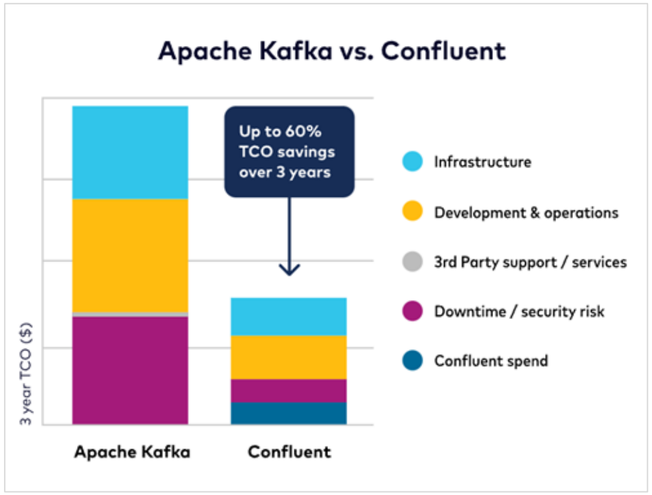
nuuly
URBAN OUTFITTERS

"저희는 옷을 판매하고 대여하는 사업을 하는 거지, 이벤트 스트리밍 플랫폼을 관리하는 사업을 하는 게 아닙니다... 만약 모든 걸 직접 관리해야 했다면 시스템을 유지하기 위해 최소 10명은 더 고용해야 했을 겁니다."

OSS Apache Kafka vs Confluent | 요약

Confluent는 게임 산업에서 요구하는 높은 보안성과 안정성, 가용성 및 사용 편의성을 충족하는 Kafka 제품과 서비스를 모두 제공할 수 있는 유일한 기업입니다. OSS Apache Kafka를 기반으로 하는 타사 제품은 게임 산업에서 요구하는 보안기능 (RBAC, Audit Log 등) 및 재해복구 요구사항(Cluster Linking, MRC, Replicator)와 같은 필수 기능을 대부분 충족하지 못하고 있습니다. 따라서 글로벌 주요 게이밍 기업에서 Confluent를 선택하여 사용중입니다.







" 오픈소스 운영에 대한 위험/부담 제거 "

오픈소스 Kafka를 직접 운영하기 위해서는 스케일링, 주키퍼 관리, 파티션 조정 등의 수많은 부가적인 업무가 동반됩니다.

반면 Confluent는 Kafka를 처음부터 완전히 재설계하여 몇 분 만에 배포, 운영 및 확장에 대한 수고를 덜어 줍니다.

" 오픈소스 운영에 대한 비용 절감 "

오픈소스 Kafka에는 기본적인 수준의 커넥터, 데이터 거버넌스 및 보안 기능이 제공되어 배포 시점이 되면 그 외 부분에 대한 운영 부담을 발생합니다.

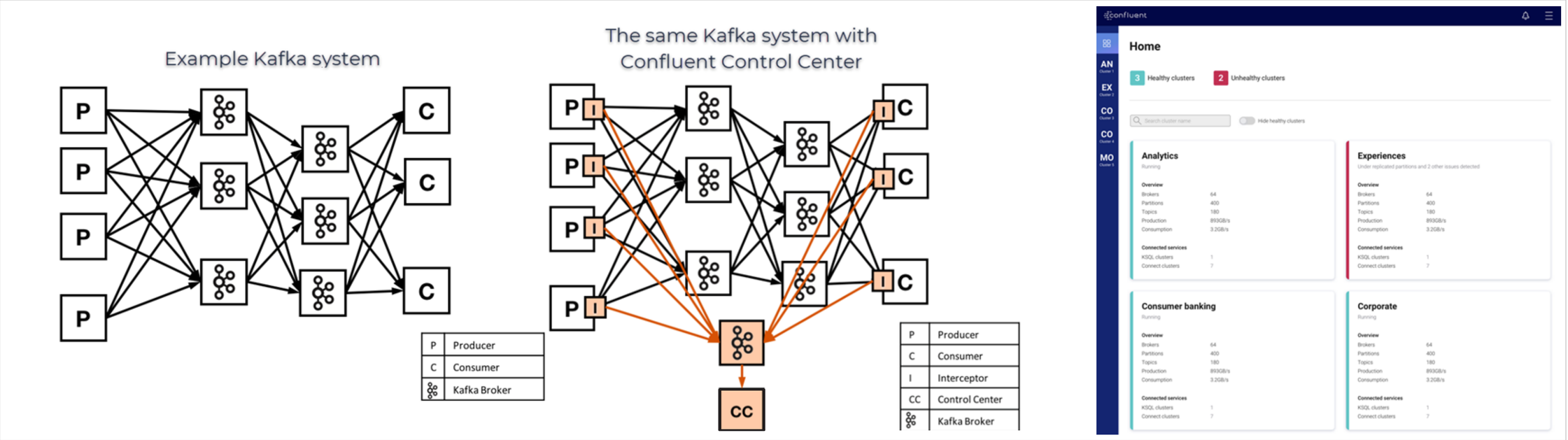
반면 Confluent는 비용이 많이 드는 개발 주기를 도구를 구축하고 유지 관리하는 데 소비할 필요가 없도록 설계된 종합적인 엔터프라이즈급 기능 세트를 제공하여 고객이 TCO를 최대 60% 절감하고 실시간 사용 사례에서 가치를 빠르게 실현할 수 있도록 지원합니다.

" IT 환경에 영향을 미치지 않는 완전 관리형 서비스 "

디지털 아키텍처를 실시간으로 간소화하고, 데이터 사일로를 깨고자하는 목표를 달성하기 위해서는 온프레미스(현재) 뿐만아니라 클라우드, 하이브리드(미래) 전반에 걸쳐 모든 환경을 포괄하는 솔루션이 필요합니다.

Confluent는 애플리케이션과 데이터가 있는 모든 곳에 적용가능하여 온프레미스 워크로드에 배포할 수 있는 모든 주요 퍼블릭 클라우드 및 자체 관리형 소프트웨어에서 완전 관리형 서비스를 자유롭게 활용할 수 있습니다.

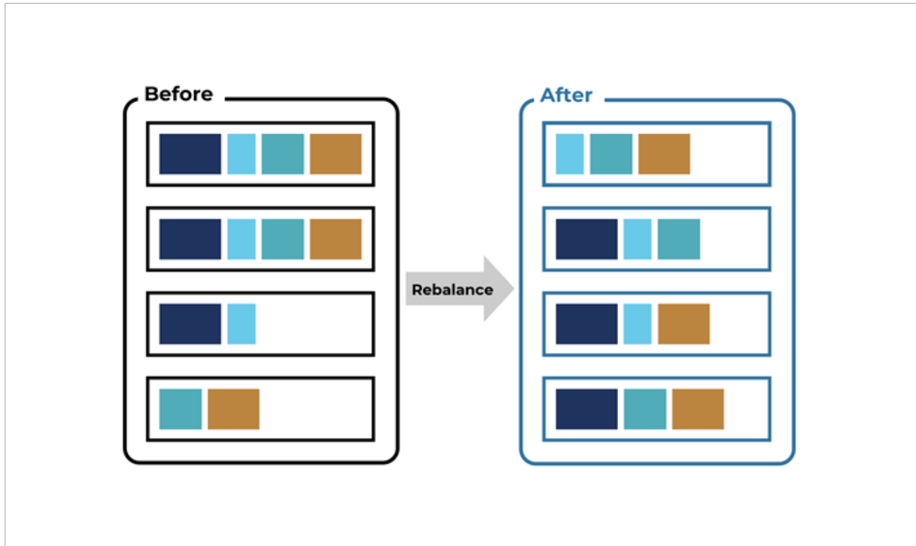
OSS Apache Kafka vs Confluent | GUI 기반 관리 및 모니터링

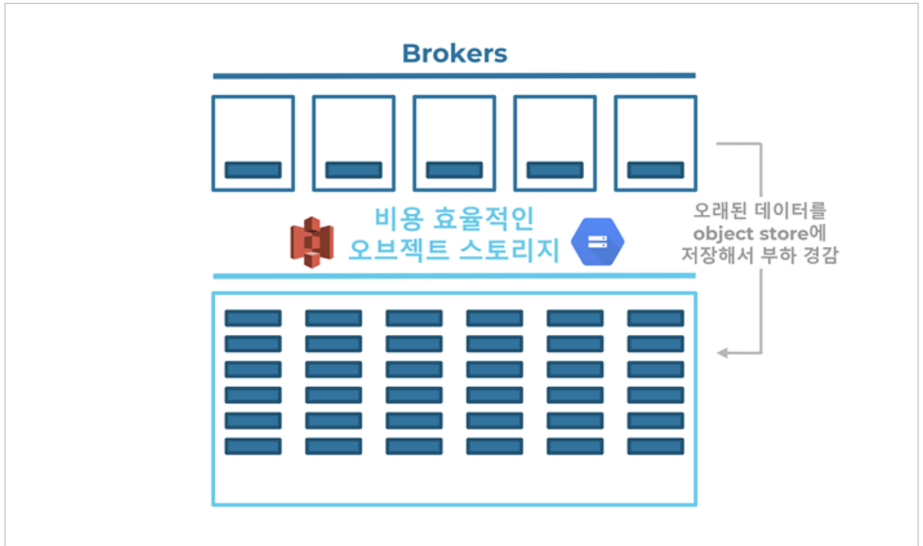


Control Center

- 다중 클러스터 환경 및 보안을 중앙에서 관리 및 모니터링 가능
- 모든 Connector 및 Topic을 한 곳에서 관리(생성, 수정, 삭제, 모니터링)
- Partition별로 Offset 또는 Timestamp 검색
- Schema 생성, 수정, 보기 및 버전 간 비교

OSS Apache Kafka vs Confluent | 동적인 성능 및 유연성





Self-Balancing Cluster

- 수작업으로 인한 에러의 위험 이 없는 자동 리밸런싱
- 최단 시간으로 전체 클러스터 리밸런싱
- 네트워크 대역폭을 보장하기 위한 Traffic throttling

Tiered Storage

- 저렴한 오브젝트 스토리지로 상당한 비용 절감
- Kafka Topic에서 직접 기록 데이터 재생
- 규정 준수 요구 사항 충족

OSS Apache Kafka vs Confluent | 엔터프라이즈급 보안

Password exposed in PLAINTEXT

```
security.protocol=SASL_PLAINTEXT
sasl.jaas.config=org.apache.kafka.common.security.oauthbearer.OAuthBearerLoginModule required \
username="robertsmith" \
password="ABC12345" \
```

VS.

Encrypted using Secret Protection

```
security.protocol=SASL_PLAINTEXT
sasl.jaas.config=${securepass:/home/robertsmith/secrets.text:robertsmith.properties/sasl.jaas.config}
config.providers=securepass
```

Event	Description	Category	Capture Default
Authorize	An RBAC authorization is being requested.	MANAGEMENT	Yes
CreateTopics	A topic is being created.	MANAGEMENT	Yes
Produce	A Kafka producer is writing a batch of records to a topic.	PRODUCE	No
FetchConsumer	A Kafka consumer is reading a batch of records from a topic.	CONSUME	No
LeaderAndIsr	Controller is sending leader and ISR state to a broker.	INTERBROKER	No



Secret Protection

- Master encryption key를 사용하여 설정 파일에 대한 안전한 암호화
- Platform 전반 (Kafka, Connect, KSQL, Schema Registry, REST Proxy 등)에 대한 표준화

Structured Audit Logs

- 비정상적인 동작 감지 및 규정 준수 요구 사항 충족

Role-Based Access Control

- 접근 관리 위임
- 중앙에서 시각적으로 관리
- 운영 오버헤드 감소

Confluent Kafka 기능은 DIY Apache Kafka 대비 TCO 절감

고객 혜택	Confluent의 차별화된 기능	상세 설명
글로벌 탄력성	**Multi-region Clusters**	완전 자동화된 복구 기능을 갖춘 단순화된 DR 아키텍처로 운영 복잡성을 줄이고 가동 중지 시간 및 데이터 손실 위험을 최소화
탁월한 성능과 탄력성	**Tiered Storage**	인프라 비용을 줄이기 위해 오래된 데이터를 저렴한 오브젝트 스토리지로 오프로드 자동화 지원
	Self-balancing Clusters	성능을 최적화하고 복잡한 수동 작업을 제거하기 위한 자동화된 데이터 밸런싱
엔터프라이즈 등급 보안	**RBAC**	보안 위반을 방지하기 위해 사용자/그룹별로 세분화된 접근 권한 부여
	Audit Logs	분석 및 포렌식을 단순화하기 위한 보안 관련 사용자 작업의 관찰 가능성 제공
	Secret Protection	민감한 정보(예: 개인 식별 정보 및 비밀번호 등 민감정보) 암호화 기능 내장
풍부하고 준비된 생태계	**Connectors**	S3, Elastic, HDFS 및 JDBC를 포함하여 100개 이상의 사전 구축되고 완벽하게 지원되는 커넥터
GUI 기반의 쉬운 관리	**Control Center**	규모에 따라 운영을 단순화하고 사전 설정된 SLA 준수 여부를 모니터링을 지원하는 GUI
유연한 DevOps 자동화	**Operator**	가치 실현 시간을 단축하고 DevOps 민첩성을 향상시키는 Kubernetes 자동화
커미터 기반의 전문성	**Support**	애플리케이션 라이프사이클 전반에 걸쳐 지원을 제공하는 세계 최고의 Kafka 전문가 그룹
	PS and Training	다양한 경험과 노하우를 기반으로 고객의 요구에 최적화된 성공사례의 적용 지원 및 지식 이전

Confluent Kafka 기능은 DIY Apache Kafka 대비 TCO 절감

고객 혜택	Confluent의 차별화된 기능	인프라 절감	비용 효율적인 플랫폼 개발	단순화된 플랫폼 운영	효율적인 지원 경험	다운타임 최소화	보안 위험 감소
글로벌 탄력성	Multi-region Clusters	●	●	●		●	
탁월한 성능과 탄력성	Tiered Storage	●					
	Self-balancing Clusters	●	●	●			
엔터프라이즈 등급 보안	RBAC	●					●
	Audit Logs		●				●
	Secret Protection						●
풍부하고 준비된 생태계	Connectors		●			●	
GUI 기반의 쉬운 관리	Control Center		●	●		●	
유연한 DevOps 자동화	Operator			●		●	
커미터 기반의 전문성	Support		●	●	●	●	●
	PS and Training		●	●	●	●	●

기업 고객의 요구사항을 반영한 Enterprise Kafka 기능 제공

	Features		Apache Kafka	Confluent
Apache Kafka®	**Open Source features**			
	Kafka Connect		✓	✓
	Kafka Streams		✓	✓
	Apache ZooKeeper™		✓	✓
	Access Control Lists		✓	✓
	Mirror Maker 2		✓	✓
	Java Client		✓	✓
Unrestricted Developer Productivity	Non-Java Clients		--	✓
	Ansible Playbooks		--	✓
	Confluent Community features			
	Schema Registry		--	✓
	ksqlDB		--	✓
	REST Proxy		--	✓
	Community Connectors		--	✓
	Confluent Commercial features			
	Stream processing & integration	Commercial, Premium, & Custom Connectors	--	✓
		Flink	--	✓
		Stream Designer	--	✓
Efficient Operations at Scale	**Cloud-native performance & elasticity**	Elastic Scaling	--	✓
		Infinite / Tiered Storage	--	✓
		Self-Balancing Clusters	--	✓
		Confluent for Kubernetes	--	✓
	Management & monitoring	Cloud UI / Control Center	--	✓
		Health+	--	✓
		Metrics API	--	✓
Production-stage Prerequisites	**Enterprise security & governance**	Stream Governance	--	✓
		Stream Sharing	--	✓
		Data Portal	--	✓
		Role-Based Access Control	--	✓
		Structured Audit Logs	--	✓
		Encryption / Secret Protection	--	✓
		Private Networking	--	✓
		HIPAA/GDPR/CCPA readiness	--	✓
	Global resiliency	Cluster Linking	--	✓
		Multi-AZ / Multi-Region Clusters	--	✓
		Replicator	--	✓
		99.99% Uptime SLA	--	✓

이러한 역량은 우리를 오픈 소스 카프카의 주요 과제를 해결하기 위한 완벽한 파트너로 만듭니다

	Apache Kafka	Confluent
운영 부담 및 리소스	클러스터를 배포, 관리 및 확장하기 위한 수동 및 리소스 집약적인 일상 작업	즉각적인 클러스터 프로비저닝 및 확장. 자동화된 용량 관리, 로드 밸런싱 및 업그레이드
보안 및 거버넌스	기술 및 운영 부채를 만드는 커스텀 보안 도구 및 통합, 스트리밍 데이터에 대한 거버넌스 부족	철저한 테스트를 거쳐 즉시 사용 가능한 엔터프라이즈급 보안 및 거버넌스 기능 제공 (예: RBAC, Audit Logs, Encryption, Stream Governance)
실시간 연결 및 처리	사일로화된 솔루션과 맞춤형 통합으로 인해 플랫폼이 확장됨에 따라 플랫폼을 관리하고 유지하는 데 드는 복잡성과 비용이 증가	풍부하고 사전 구축된 커넥터 생태계와 스트림 처리를 통해 앱 및 데이터 시스템 전반에 걸친 기업 전체 데이터 패브릭을 구축
글로벌 가용성	리소스 집약적인 장애 조치 설계 및 운영으로, 환경의 규모와 다양성이 증가함에 따라 복잡성과 위험도 기하급수적으로 증가	모든 클라우드와 온프레미스를 아우르는 모든 환경에서 데이터에 대한 높은 가용성과 일관된 액세스를 제공

Morgan Stanley

"Confluent는 Apache Kafka를 활용하여 운영 투명성을 높이고 데이터 기반의 적시 결정을 내리는 데 중요한 역할을 하고 있습니다. 비즈니스 흐름 모니터링, 대출 승인 속도 향상, 더 빠르고 시의적절한 사기 분석 제공 등에서 개선된 점에 대해 기쁘게 생각하며, 플랫폼을 활용하여 비즈니스 결정을 간소화, 자동화 및 추진하는 서비스를 확장할 수 있기를 기대합니다."

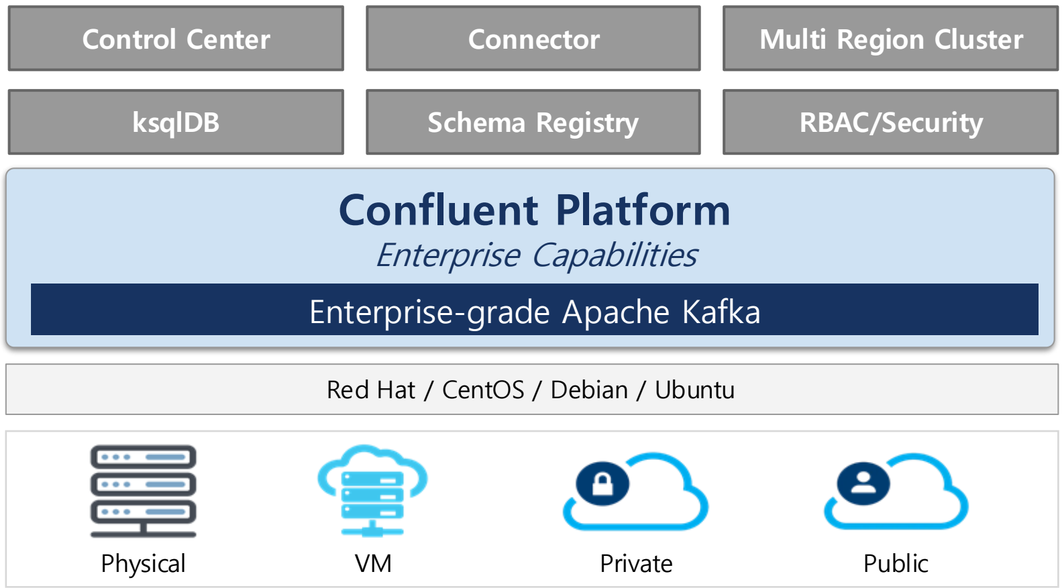
Confluent Platform(설치형)/Confluent Cloud(SaaS형)

Confluent는 온프레미스 환경, VM 환경, Private Cloud 환경 및 Public Cloud 환경에 직접 설치 구성하여 사용할 수 있는 Confluent Platform을 제공합니다. 또한, AWS, Azure 및 GCP 상에서 Confluent가 직접 운영 관리하는 SaaS Cloud Service 형태인 Confluent Cloud도 제공합니다. 고객의 요구사항 및 사용 목적에 따라서 원하는 제품을 사용할 수 있습니다.

설치형 소프트웨어/완전 관리형 클라우드 SaaS 서비스

Confluent Platform(설치형 소프트웨어)

- 물리서버, VM, Private Cloud 및 Private Cloud 등 모든 환경을 지원
- 오픈소스 Apache Kafka가 제공하지 않는 엔터프라이즈급 기능들을 제공



Confluent Cloud(클라우드 SaaS 서비스)

- Zero Downtime: 멀티 AZ(Available Zone) 기반으로, 하나의 Zone 장애시 완벽하게 보호
- Cluster Linking: Multi Region 또는 Hybrid Cloud 환경에서의 HA/DR 완벽히 지원

완전 관리형 서비스와 가용성

인프라 및 완전 관리형 구성 요소에 대해 99.99%의 가동 시간을 보장합니다.

수준 높은 DR/HA 지원

멀티 AZ 복제 및 데이터 동기화를 위한 Cluster Linking

Mission critical 기술 지원 제공

커미터 중심의 Kafka 지원 및 응답 시간 SLA 제공

2. Confluent Platform Overview

Confluent Platform Overview

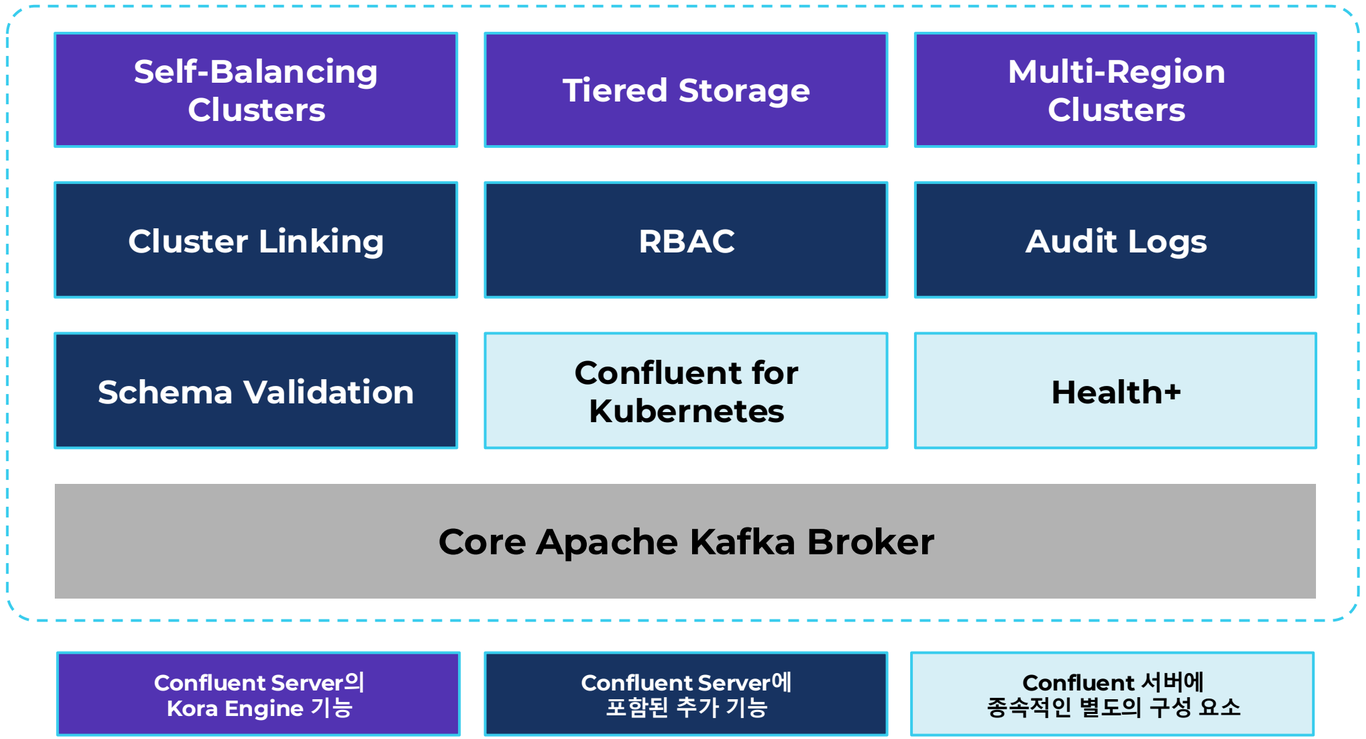
Confluent Platform은 Kora Engine의 구성 요소를 사용하여 클라우드 네이티브의 이점을 제공합니다.

Confluent Server는 온프레미스 환경에 적용 가능한 Kora의 기능을 실행합니다.

저희는 Kora를 사용하여 Kafka 아키텍처를 재설계했고, 이를 통해 Confluent Cloud는 완전 관리형 클라우드 서비스가 되었습니다.



Confluent Platform의 기반 브로커인 Confluent Server는 Kora의 요소들을 활용하여 탄력성, 무제한 스토리지, 고가용성과 같은 클라우드 네이티브의 이점을 제공합니다



효율적인 운영을 위한 사용자 편의성 제공

자동화 툴을 통해 자동화된 배포 및 무중단 업데이트를 지원합니다

Confluent Platform은 Confluent Ansible Playbook과 Confluent Operator를 제공하여 서비스 무중단 유지 관리 자동화를 지원합니다.

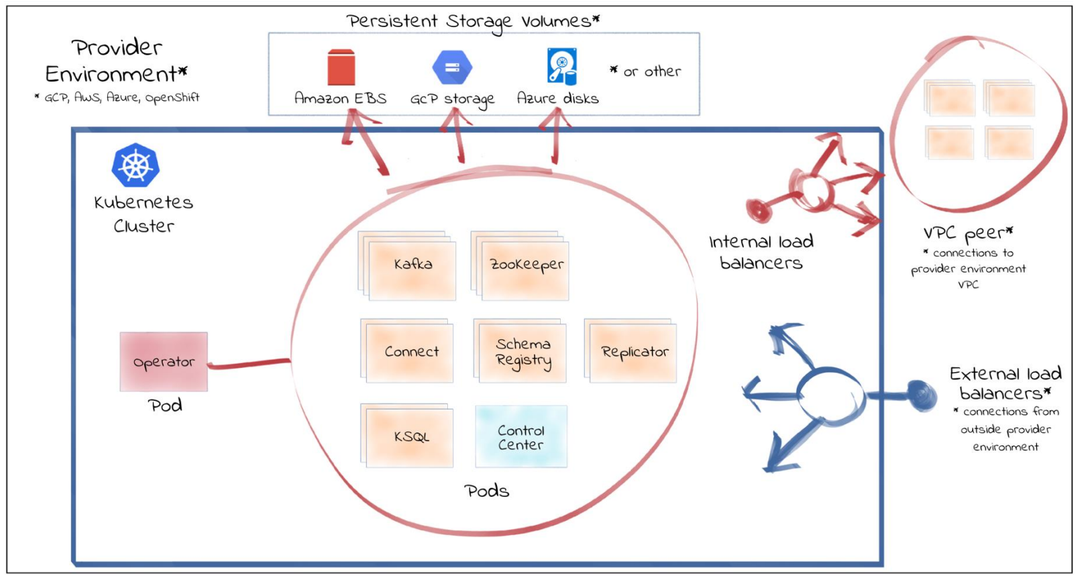
- **Confluent Operator**는 Kubernetes에서 Confluent Platform 리소스 배포를 자동화하고, **Ansible Playbooks**는 가상머신 또는 물리서버 같은 컨테이너화 되지 않은 환경에서 Confluent Platform 리소스 **배포 자동화**를 지원합니다.
- **Confluent 전문가가 설계한 표준화되고 검증된 아키텍처를 기반**으로 Confluent Platform의 여러 구성 요소를 한번에 **쉽게 배포할** 수 있습니다.
- 또한 **Upgrade/Patch/증설** 등의 작업을 수행 할 때 Rolling 배포 전략을 활용하여 **서비스 다운타임 없이 또는 최소화 하여 자동화 할 수 있습니다.**

Confluent Operator & Ansible Playbooks

Confluent Operator for Kubernetes

Confluent Cloud의 실제 환경에서 검증된 기술: Confluent Operator는 수년간의 Confluent Cloud 경험을 바탕으로, 온프레미스 또는 퍼블릭 클라우드상의 Kubernetes 환경위에서 검증된 엔터프라이즈급 Confluent Platform을 배포 및 구성합니다.

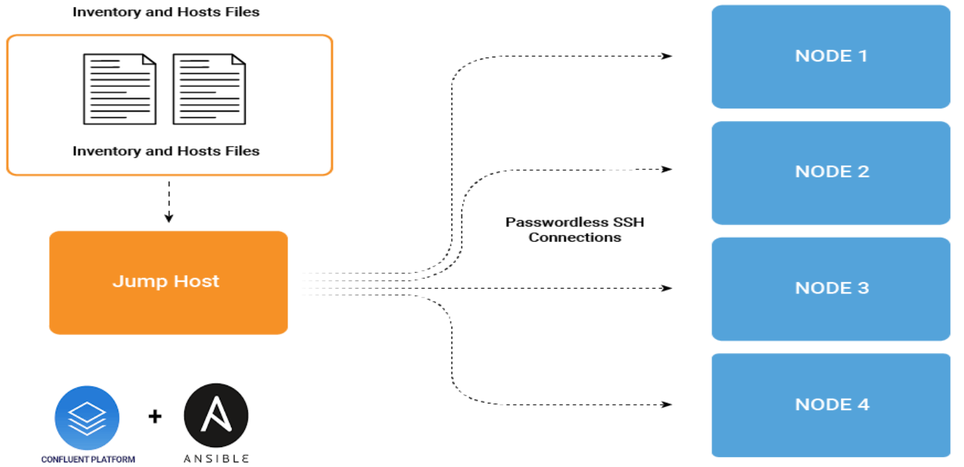
Confluent Operator를 이용하여 Kubernets 환경에 유지 관리 자동화를 지원합니다.



Confluent Ansible Playbooks

Kafka 전문 지식으로 구축된 아키텍처: Ansible Playbooks은 총 100만 시간 이상의 Kafka 배포 및 구성을 지원한 세계 최고의 Kafka 전문가가 설계한 아키텍처를 기반으로, 비컨테이너 환경위에 엔터프라이즈급 Confluent Platform을 배포 및 구성합니다.

CP-Ansible을 이용하여 베어메탈/가상화/클라우드 노드의 유지 관리 자동화를 지원합니다.



웹 관리 도구를 통한 사용자 편의성 제공- Control Center

Confluent Platform은 오픈소스 Apache Kafka와 다르게 GUI 화면의 Control Center를 제공합니다. Control Center를 사용하여 Multi Confluent Platform Cluster의 Broker, Topic, Consumer에 대해 통합 모니터링이 가능합니다.

Confluent Control Center

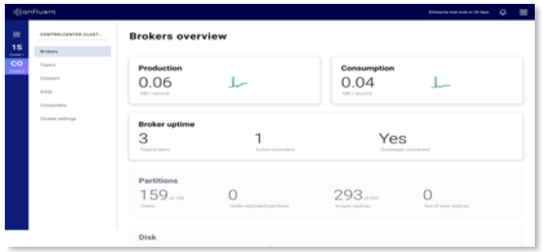
단일 GUI 화면으로 모든 구성 요소의 통합 모니터링 기능 제공

Multi Cluster 통합 모니터링



- Cluster들의 상태 요약
- Cluster에 대한 검색 기능
- Cluster의 현재 상태를 시각화
- Cluster의 이름과 현재 상태
- Cluster 내부의 상세 데이터

Broker 통합 모니터링



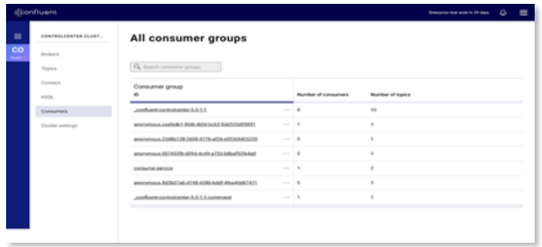
- Broker 통합 메시지 발행 수치 및 추이
- Broker 통합 메시지 소비 수치 및 추이
- Broker, Controller, ZooKeeper의 현재 상태
- 모든 Partition의 상태 정보, ISR, OSR 상태
- Node들의 디스크 사용량, 분산 상태
- Node의 네트워크 사용량

Topic 통합 모니터링



- 전체 Topic 목록
- Topic 검색 기능
- Topic 추가/삭제 기능
- Topic별 총 Partition 수, 복제 상태, OSR 상태
- Topic별 Pub/Sub 처리량
- Topic 상세 도구 이동 메뉴

Consumer Group 통합 모니터링



- 전체 Consumer Group 목록
- Consumer Group별 Consumer 수, Topic 수
- Consumer Group별 구독 Topic 수
- Consumer Group 상세 도구 이동 메뉴

웹 관리 도구를 통한 사용자 편의성 제공- Control Center

모니터링 대시보드를 통해 상태 정보의 시각화를 지원하며, 알람 메트릭을 이용하여 알람 내역의 조회 및 알람 설정을 지원합니다. 알람 설정을 통해 email, slack, pager duty와 같은 툴을 통해 알람을 실시간으로 받아 볼 수 있습니다.

Confluent Control Center

상태 정보 시각화 기능

수치 정보에 대한 그래프 시각화



- 수치 정보에 대한 그래프 형태의 시각화 제공

상태 이상 감지에 대한 표시



- 클러스터에 대한 상태 이상 감지시 시각화된 정보 표시 기능
- Broker, Controller, ZooKeeper에 대한 상태 이상 감지시 시각화된 정보 표시 기능

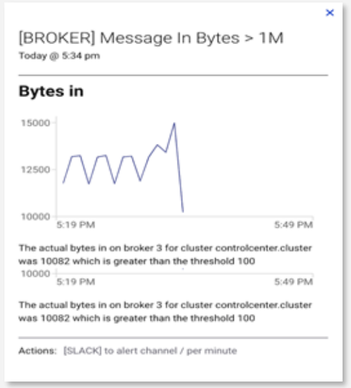
이슈 탐지 기능

알람 Metric 및 방식 설정



- Cluster, Broker, Topic, Consumer Group 알람 설정 기능
- Email, Slack, PagerDuty와 연동된 알람 수단 설정 기능
- 다대다 방식의 알람 설정 기능
- 알람 설정 목록 조회 및 on/off 기능

알람 내역 목록 및 상세 조회



- 알람 내역 목록 조회
- 알람 내역 상세 조회

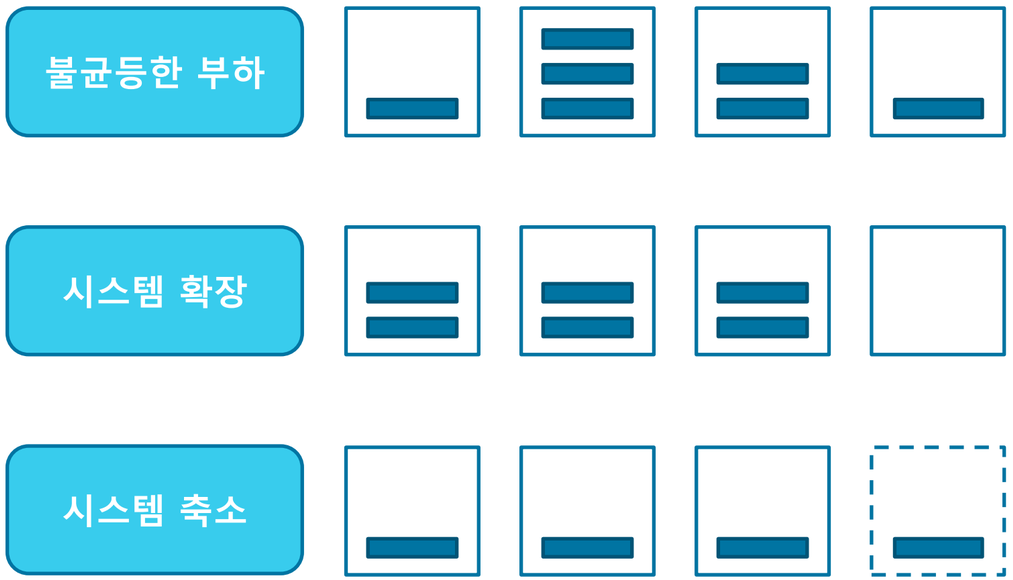
Self Balancing Clusters를 통해 탄력적인 부하분산 지원

Confluent Platform은 Self Balancing Clusters를 제공하여 파티션 재조정(Partition Rebalancing)을 자동화하여 모든 Broker의 Partition의 균형을 맞출 수 있습니다. 이를 통해 Confluent Platform내 Kafka의 성능 향상 및 탄력적인 부하분산 효과를 얻을 수 있습니다.

Self Balancing Clusters

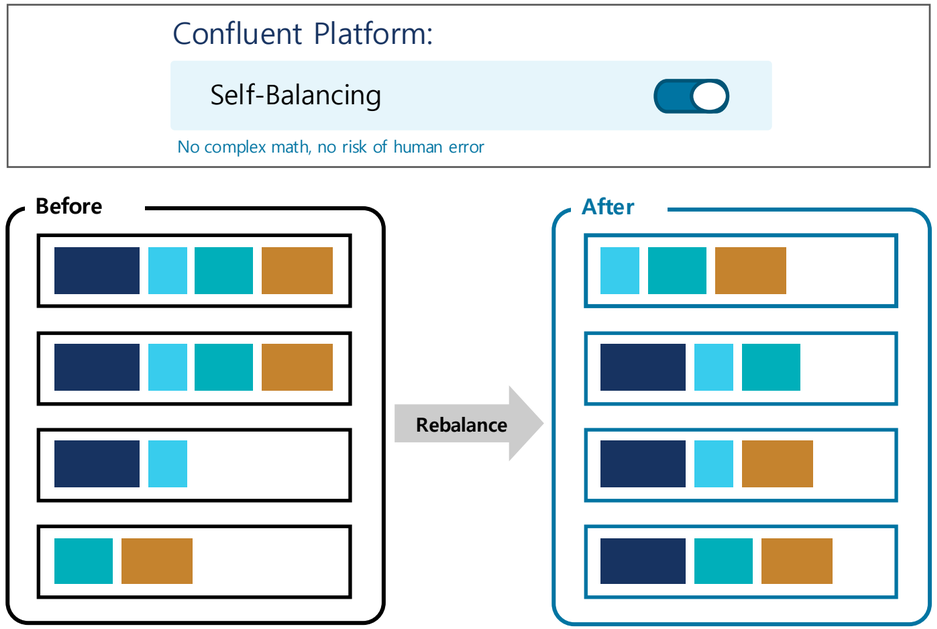
Broker내의 Partition의 불균형 발생 케이스

- 시간이 지남에 따라 Broker내의 Partition의 불균형이 발생할 수 있음
- 이로 인해 Cluster에 대한 Read 및 Write 처리량이 저하될 수 있음
- 파티션 재조정을 위한 오픈소스가 있지만 매우 수동적인 절차로 구성되어 있으며, 사람의 실수로 인한 클러스터 문제 유발 가능성이 높음



Self Balancing Clusters를 통한 자동화

- 수작업으로 인한 에러 발생의 위험이 없는 자동 리밸런싱
- 최단 시간으로 전체 Cluster 리밸런싱
- 네트워크 대역폭을 보장하기 위한 Traffic Throttling 기능 제공
- 명령어 수행이 아닌 단순 버튼 조작으로 on/off



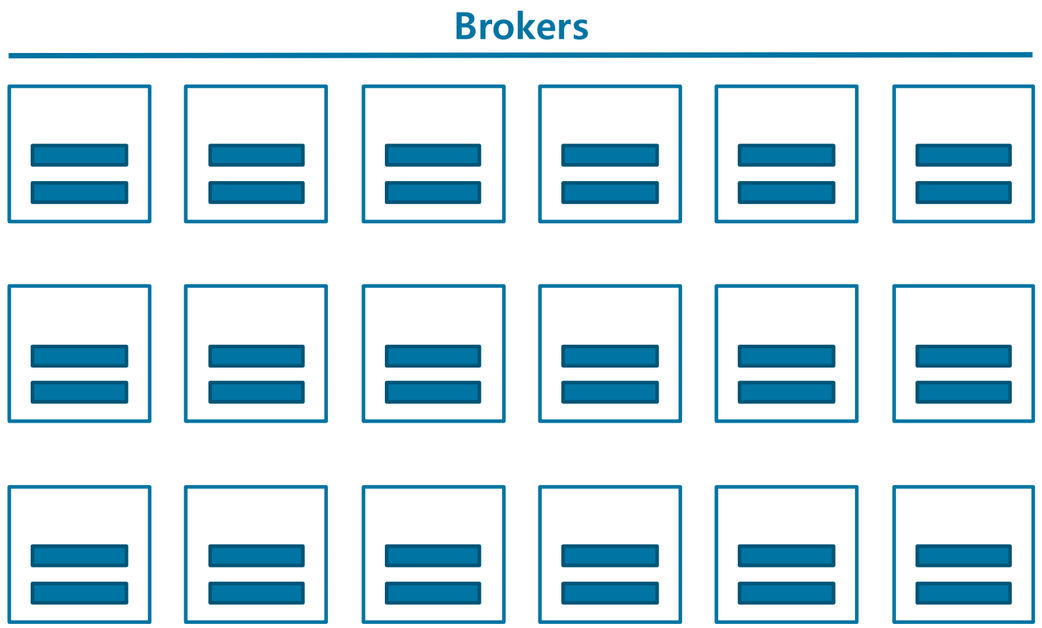
자동화된 스토리지 티어링 기능을 통해 비용 효율 극대화

Confluent Platform은 Tiered Storage를 통해서 상대적으로 저렴한 Object Storage를 사용하여 보다 저렴한 비용으로 Confluent Platform에 데이터를 저장할 수 있으므로 사실상 무한에 가까운 보존이 가능합니다. 모든 Topic 데이터를 Broker에 직접 저장하는 대신 Broker는 데이터에 대한 Consumer의 성능을 유지하면서 오래된 메시지를 Object Storage에 저장함으로써 Broker 스토리지의 부담을 줄일 수 있습니다.

Tiered Storage based on Object Storage

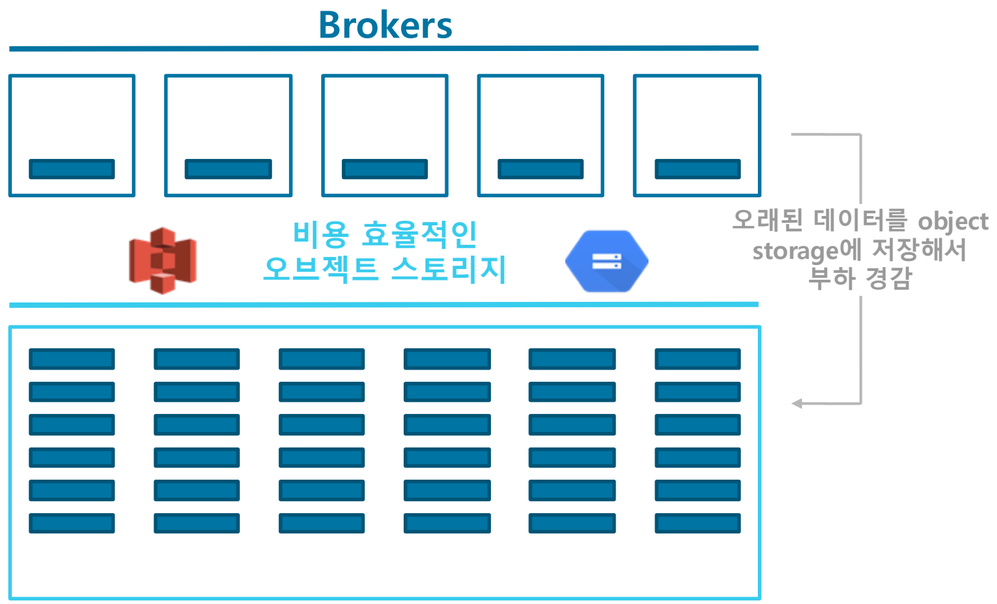
Standard Storage on Confluent Platform

- Apache Kafka만 사용시, 모든 데이터는 Broker에 저장
- 만약 보관 주기(Retention Period)를 길게 가져가야 한다면, Storage를 충분히 구성해야 하기 때문에 추가적인 스토리지 하드웨어로 인한 비용이 많이 발생할 수 있음



Tiered Storage on Confluent Platform

- 상대적으로 저렴한 오브젝트 스토리지로 비용 절감 가능
- Kafka Topic에서 직접 기록 데이터 재생
- 오래된 데이터를 object storage에 저장하여 Broker 스토리지 부담 경감
- 데이터 유지 등의 규정 준수 요구 사항 충족

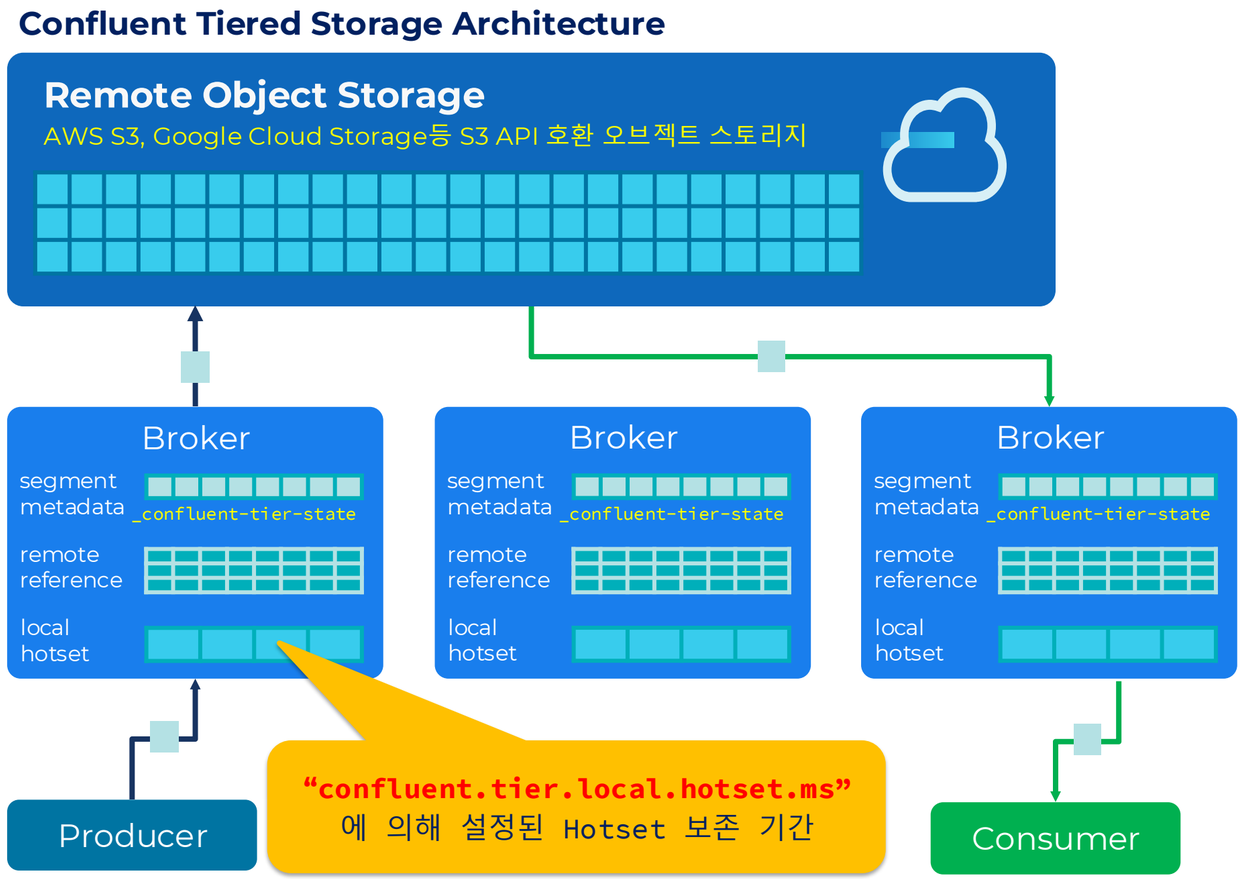


Kafka Tiered Storage (계층형 메시지 저장)

Kafka 브로커의 로컬 디스크에 저장된 로그 데이터를 일정 기간 이후 저비용 외부 스토리지로 자동 이동하여 저장하는 구조
➔ 장기 데이터를 보관하여 분석 데이터 시뮬레이션에 활용 가능(비용 절감 탁월)
➔ Consumer의 수정 없이 Broker Level에서 Storage Tiering 기능 제공

계층	역할
Hot Tier	최근 데이터 (빠른 처리, Broker 내장 스토리지 활용)
Cold Tier	과거 데이터 (장기 보관, S3 API 호환 Object Storage 활용)

항목	Tiered Storage 사용 전 (로컬 디스크만)	Tiered Storage 사용 후 (Object Storage 병행)
스토리지 비용	대용량 SSD/로컬 디스크 필요 → 비용 상승	오래된 데이터는 S3 호환 Object Storage에 보관 → GB당 비용 크게 절감
브로커 디스크 부담	장기 보관으로 디스크 사용률 높음, I/O 병목·장애 리스크 증가	브로커는 최신 데이터 위주 저장 → 디스크 여유·I/O 안정성 향상
확장성 (Compute vs Storage)	브로커 수·디스크 증설이 곧 저장기간 확장 → 함께 커져야 함	브로커(Compute)와 Object Storage(Storage)를 **독립적으로** 확장 가능
장기 데이터 활용	보관 기간을 짧게 가져가거나 별도 아카이브 필요	Tiered Storage에 내려간 세그먼트까지 포함해 전체 범위 재처리·재소비 가능
운영 복잡도	주기적 삭제·압축·디스크 증설 등 수작업 많음	정책 기반 자동 오프로드 → 운영 단순, 용량 관리 수월
대용량 센서/ FDC 데이터 적합성	고주파 센서/FDC 데이터 장기 보관 시 비용·운영 부담 큼	브로커 1일 + Object Storage N일 구조로 대용량 시계열도 경제적 장기 보관 가능



생산성 향상을 위한 기능 제공

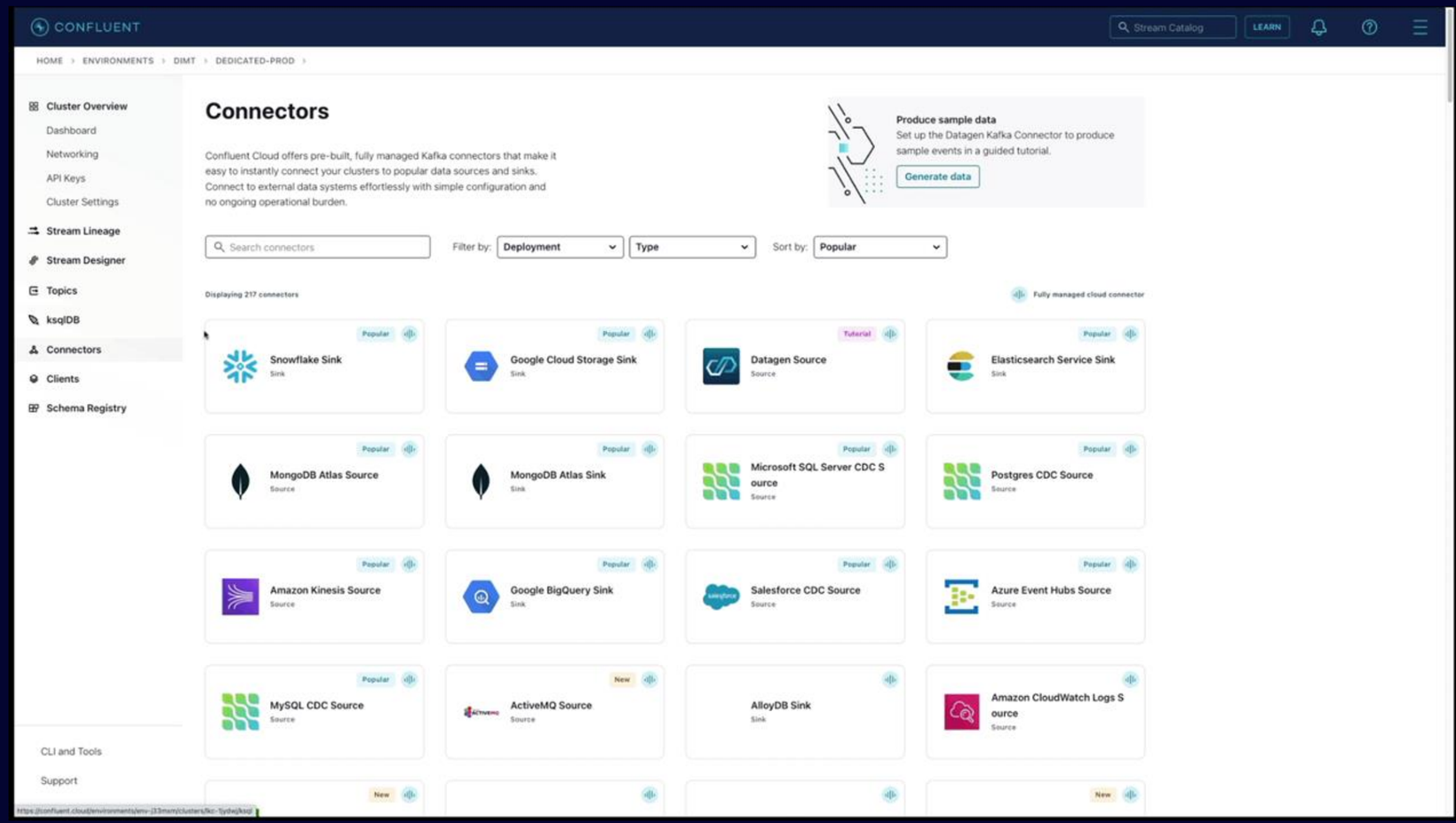
CONNECT

소스 및 타겟 시스템 연결을 위한 커넥터

120+ 프리빌트 커넥터
70+ 풀리 매니지드 커넥터

커스텀 커넥터 기능 제공

다양한 제품과 손쉬운 연계



완전 관리형 커넥터를 통해 손쉽게 소스 및 타겟 시스템 연결

다양한 Client 지원을 위한 다양한 기능 및 템플릿 제공

Confluent는 오픈소스 Apache Kafka가 지원하지 않는 다양한 형태의 Client를 지원합니다. C/C++, Java, Go, .NET, Python 등의 개발언어를 지원하며, 이와 다른 개발언어로 개발된 Client의 경우는 REST Proxy 및 MQTT Proxy를 통해서 Confluent로 메시지(데이터)를 전송합니다. Client 개발을 지원하기 위한 가이드 및 템플릿을 제공합니다.

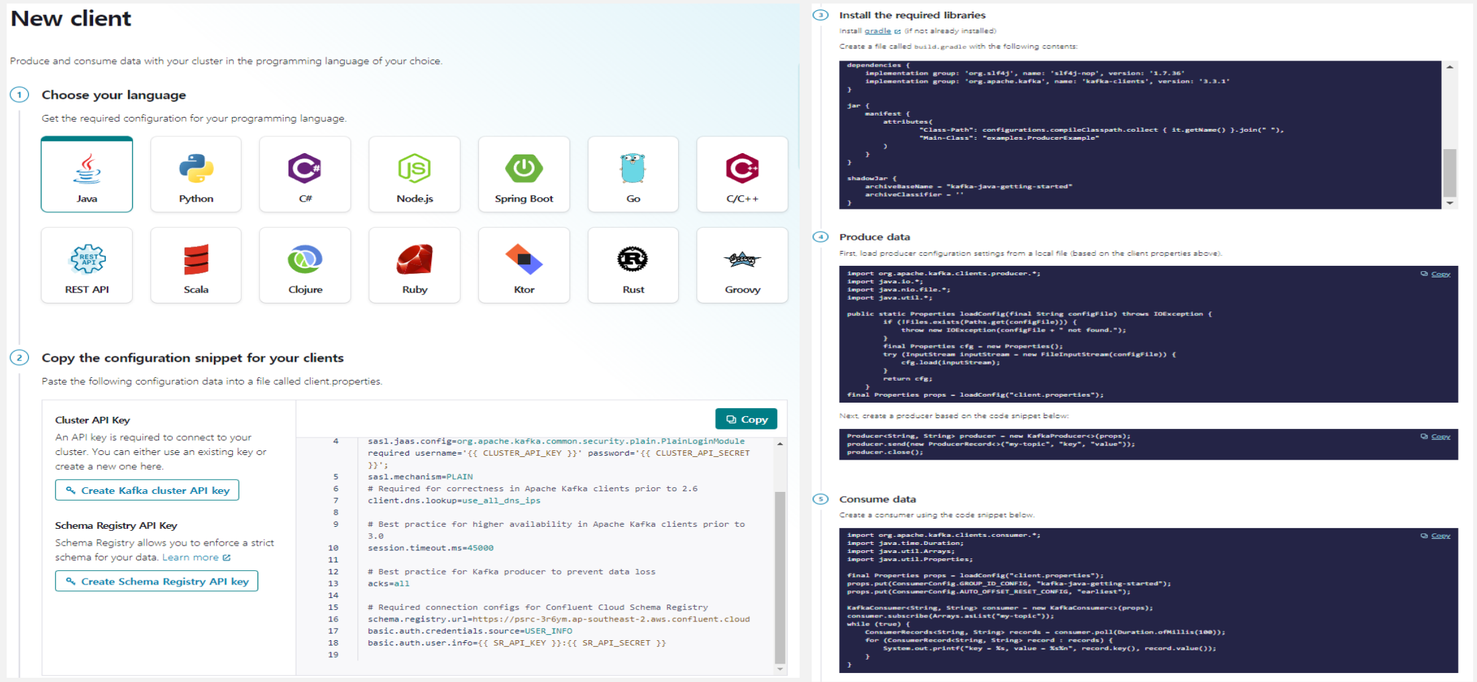
다양한 Client 지원

Client 개발용 다양한 개발 언어 지원 01

- 모든 개발자가 다양한 Confluent 클라이언트를 통해 조직 전체에서 Kafka를 활용할 수 있도록 지원
- https://docs.confluent.io/platform/current/installation/versions-interoperability.html#clients

Client template 및 guide 제공 02

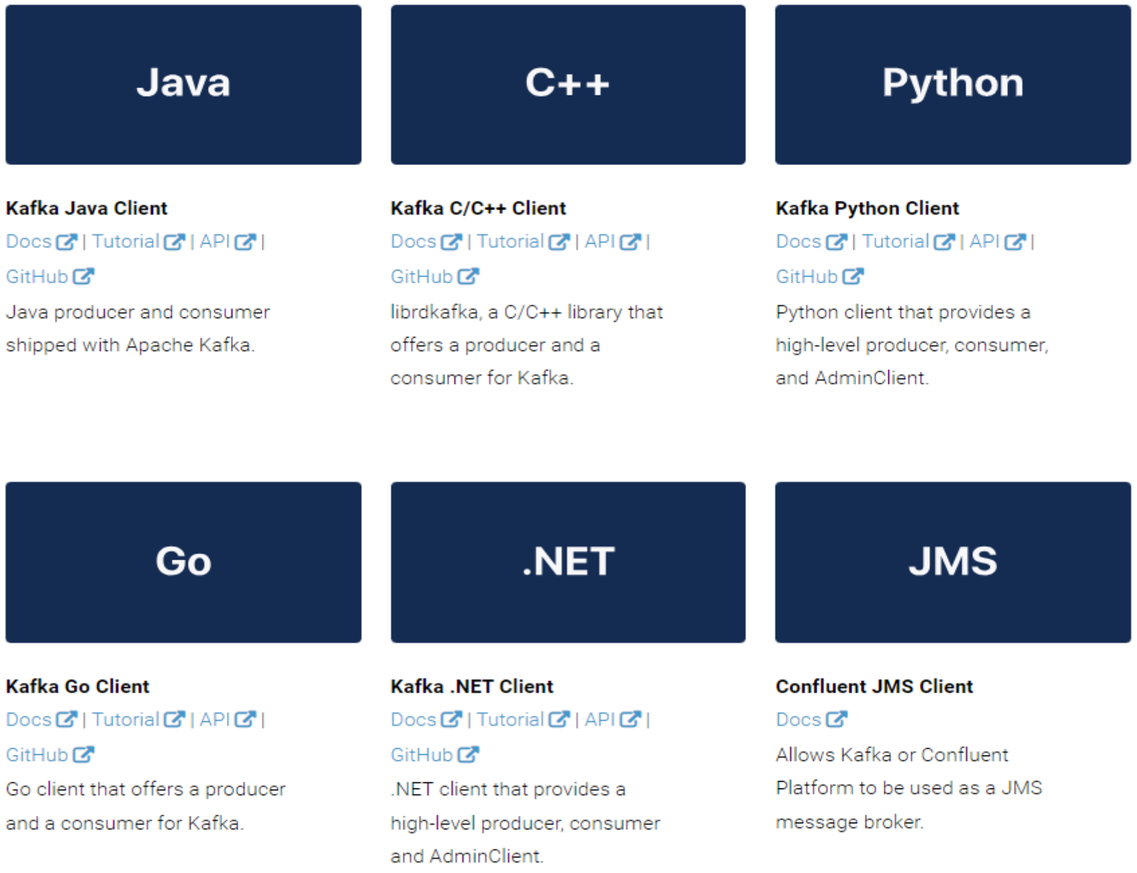
- 신규 클라이언트를 위한 다양한 Language에 대한 API Key Snippet과 필요 라이브러리 설치 가이드 제공
- Data Produce와 Consume에 대한 예제 Code 제공



다양한 Client 지원을 위한 개발 가이드 제공

Confluent는 다양한 Client 지원을 위한 개발 가이드를 제공합니다.

“Build Kafka Client Applications on Confluent Cloud”에 Client 응용프로그램 개발을 위한 가이드를 제공하고 있습니다.



Kafka Java Client
Docs | Tutorial | API | GitHub
Java producer and consumer shipped with Apache Kafka.

Kafka C/C++ Client
Docs | Tutorial | API | GitHub
librdkafka, a C/C++ library that offers a producer and a consumer for Kafka.

Kafka Python Client
Docs | Tutorial | API | GitHub
Python client that provides a high-level producer, consumer, and AdminClient.

Kafka Go Client
Docs | Tutorial | API | GitHub
Go client that offers a producer and a consumer for Kafka.

Kafka .NET Client
Docs | Tutorial | API | GitHub
.NET client that provides a high-level producer, consumer and AdminClient.

Confluent JMS Client
Docs
Allows Kafka or Confluent Platform to be used as a JMS message broker.

Pre-built Connectors를 통한 쉽고 빠른 데이터 통합

Confluent Platform는 데이터를 가져오는 대상인 Data Source 및 데이터를 내보내는 대상인 Data Sink가 될 수 있는 다양한 대상 시스템들과 손쉽게 연결하기 위한 다양한 Connector들을 제공합니다. 대상 source 및 sink와 Confluent Platform간 연결을 위해 client를 별도로 개발하지 않고 **Connector와 설정만으로 쉽게 연결**할 수 있으므로, 개발 **소스 관리 및 유지 보수를 피할 수 있습니다.**

다양한 Pre-built Connector 제공 및 지원

Pre-built Connectors

80+ Confluent 지원

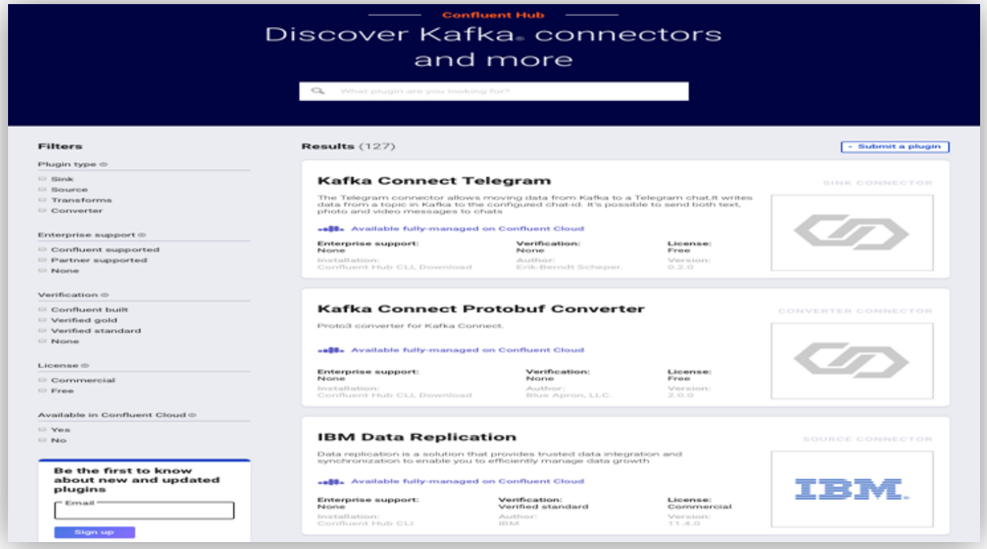




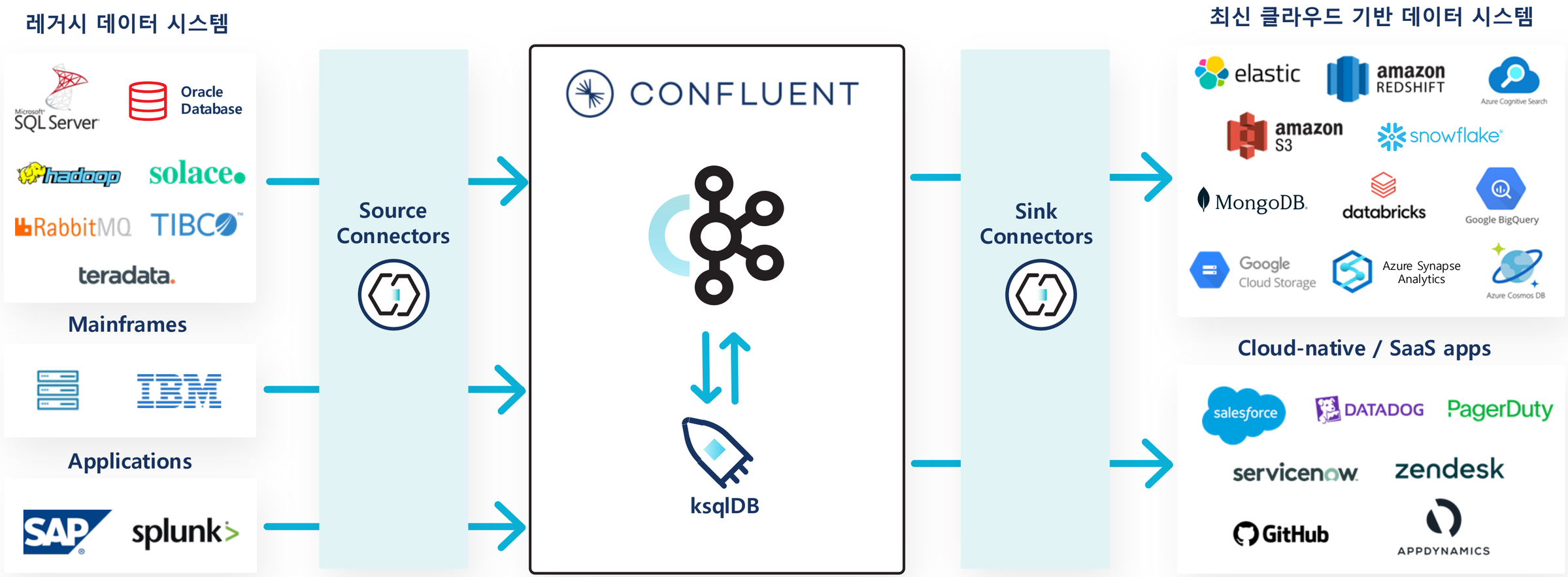
20+ Partner 지원, Confluent 검증

Connector Hub

- Connector를 쉽게 검색:
  - Source 및 Sinks
  - Confluent 및 Partner supported
  - Commercial 및 Free
    - Confluent Cloud에서도 동일하게 사용 가능
- https://www.confluent.io/hub

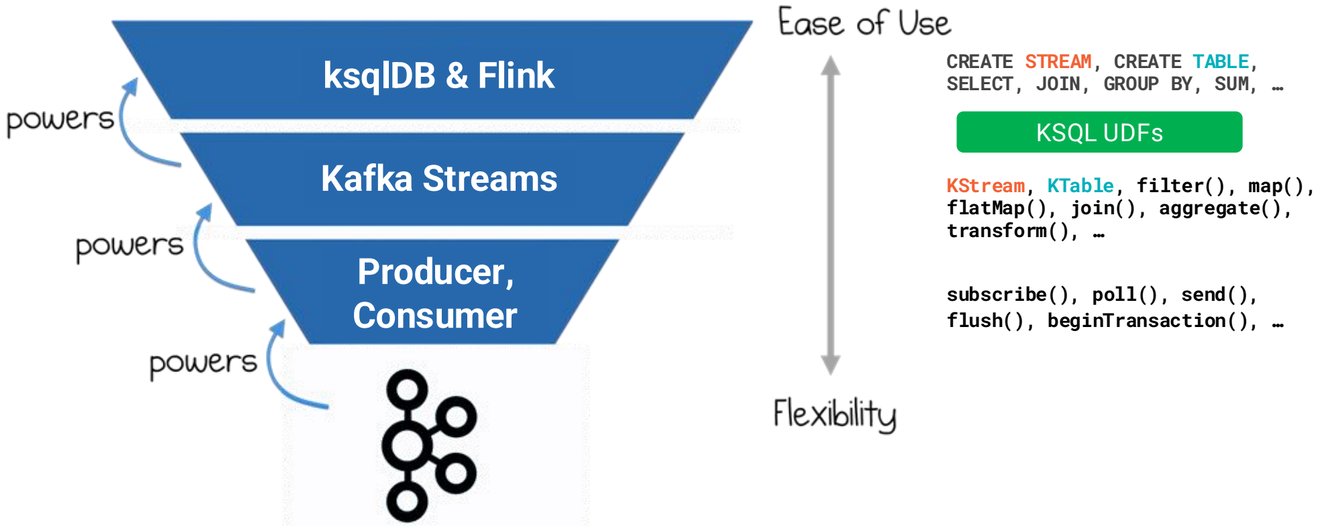


레거시 온프레미스 시스템에서 최신 클라우드 기반 기술로의 마이그레이션 가속화

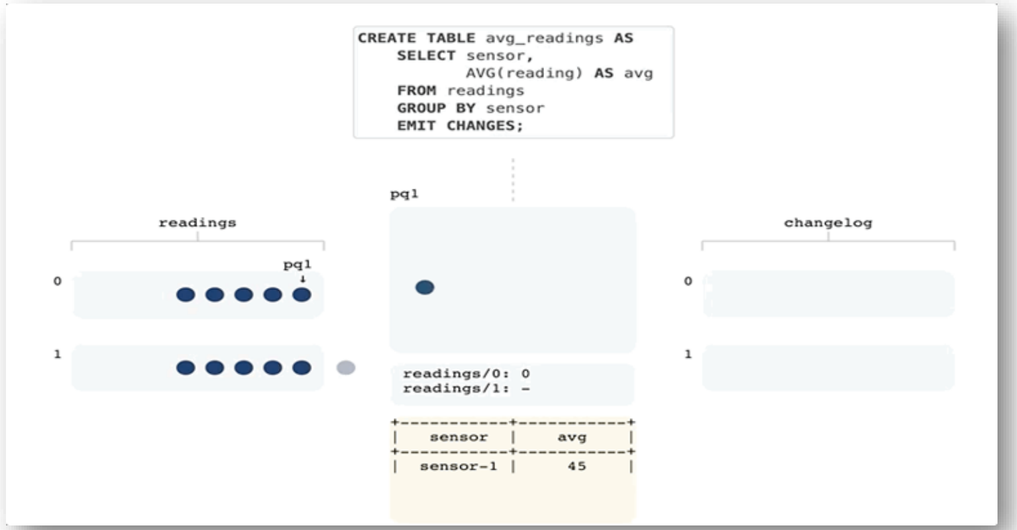


Kafka 스트림 프로세싱

Kafka Streams, ksqlDB, Flink 를 통한 Kafka 스트림 분석 프로세싱 기능 지원



Kafka 스트림 분석 프로세싱 통해 실시간 프로세싱 처리 가능



Kafka Clients (Pub/Sub API)

```
ConsumerRecords<String, String> records = consumer.poll(100);
Map<String, Integer> counts = new DefaultMap<String, Integer>();
for (ConsumerRecord<String, Integer> record : records) {
  String key = record.key();
  int c = counts.get(key)
  c += record.value()
  counts.put(key, c)
}
for (Map.Entry<String, Integer> entry : counts.entrySet()) {
  int stateCount;
  int attempts;
  while (attempts++ < MAX_RETRIES) {
   try {
     stateCount = stateStore.getValue(entry.getKey())
     stateStore.setValue(entry.getKey(), entry.getValue() + stateCount)
     break;
   } catch (StateStoreException e) {
     RetryUtils.backoff(attempts);
   }
  }
}
```

Kafka Streams (Java 라이브러리)

```
builder
    .stream("input-stream",

Consumed.with(Serdes.String(),
Serdes.String()))
    .groupBy((key, value) -> value)
    .count()
    .toStream()
    .to("counts",
Produced.with(Serdes.String(),
Serdes.Long()))
```

ksqlDB (SQL 인터페이스 랩핑)

```
SELECT x, count(*) FROM stream
GROUP BY x EMIT CHANGES;
```

Flink SQL (범용 SQL 엔진)

```
SELECT x, count(*) FROM stream
GROUP BY x;
```

Stream Processing with Confluent

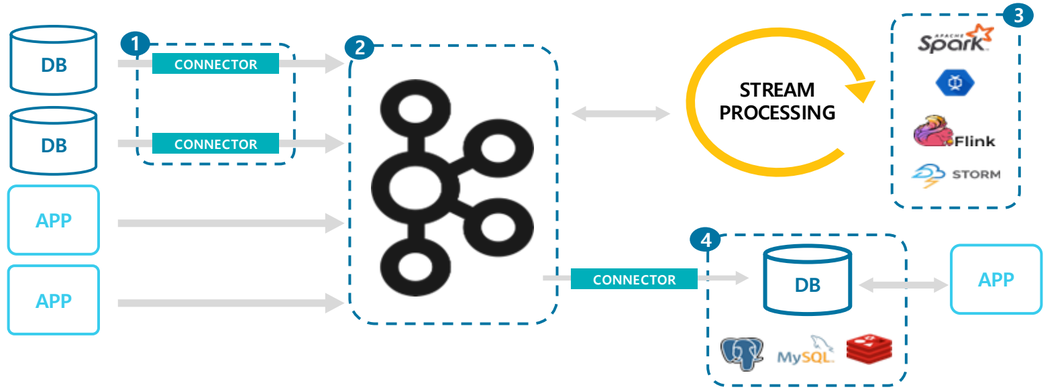
<table>
<tr><th>  CP-Flink</th><th>  CC-Flink</th><th>  Kafka Streams</th><th>  ksqlDB</th></tr>
<tr><td colspan="2">Apache Kafka와 독립적으로 개발된 스트림 처리 프레임워크</td><td>Apache Kafka 프로젝트의 일부인 Java 애플리케이션용 임베디드 클라이언트 라이브러리</td><td>SQL을 통해 Kafka Streams 기능을 노출하는 스트림 처리 프레임워크</td></tr>
<tr><td colspan="2" rowspan="2">• 통합된 스트림 및 일괄 처리
• 여러 Kafka 클러스터 및 기타 시스템에서 읽기 지원
• 가장 최근의 증분 스냅샷에서 장애 발생 후 상태 복원</td><td>• 스트림 처리만 지원
• 단일 Kafka 클러스터에서 읽기 지원</td><td>• 스트림 처리만 지원
• 단일 Kafka 클러스터에서 읽기 지원</td></tr>
<tr><td>• 모든 메시지를 재생하여 실패 후 상태 복원</td><td>• 모든 메시지를 재생하여 실패 후 상태 복원</td></tr>
<tr><td>• Confluent Platform을 통해 자체 관리형 제공</td><td>• 완전 관리형: 클러스터 배포 없음, Scales to zero</td><td>• 자체 관리형
• 클러스터가 없는 임베디드 클라이언트 라이브러리</td><td>• 완전 관리형 및 자체 관리형
• 별도의 클러스터 배포</td></tr>
<tr><td>• 계층화된 API 지원을 갖춘 완벽한 Flink 제공</td><td>• ANSI-compliant SQL(GA)
• Java와 Python 지원(GA)</td><td>• Java</td><td>• SQL-like</td></tr>
</table>

Flink/ksqlDB를 활용한 실시간 데이터 스트림 처리(데이터 가공)

Confluent Platform은 여러 개의 서로 다른 분산 시스템을 연결해야 했던 기존의 스트림 처리 아키텍처를 대폭 단순화한 ksqlDB를 제공합니다.
익숙한 SQL syntax를 사용하여 수행되므로 이벤트 스트리밍 처리 애플리케이션 구축 과정을 크게 단순화할 수 있습니다.

스트리밍 처리용 데이터베이스

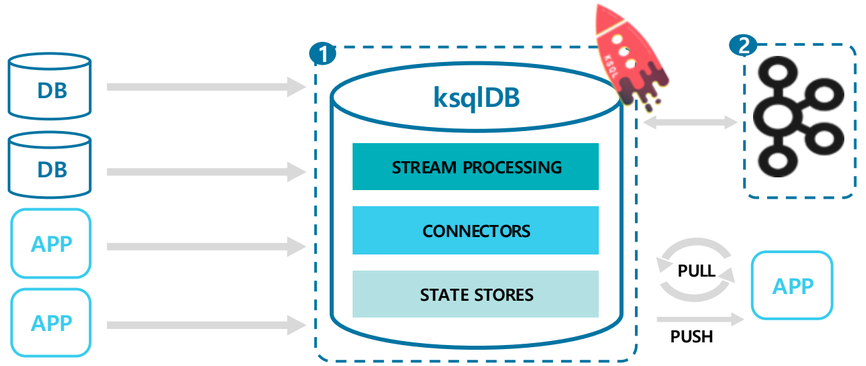
기존 스트림 처리 아키텍처



서로 다른 분산 시스템을 연결하고 관리해야만 했던 문제점

- 여러 개의 서로 다른 분산 시스템을 복잡하게 연결해야만 함
  ① 기존 데이터 저장소에서 데이터를 수집하기 위한 Connector cluster
  ② 이벤트 데이터 저장용 Kafka Cluster
  ③ 이벤트를 가져와서 스트림 처리 수행후 Kafka에 다시 저장
  ④ 애플리케이션에 Query를 제공하기 위한 시스템에 처리된 이벤트 저장
- 성능이 뛰어나고 확장 가능하며 안전한 아키텍처를 만들고 통합하기 어려움

ksqlDB 기반의 스트림 처리 아키텍처



ksqlDB를 통한 통합 구성

- 두 개의 시스템으로 간단하게 구성
  ① 스트림 처리를 위한 ksqlDB
  ② 이벤트 데이터를 저장하기 위한 Kafka Cluster는 동일
- ksqlDB가 다른 시스템들의 기능들을 포함
  - ksqlDB내의 Connector를 사용하여 기존 시스템 연결
  - 가져온 이벤트에 대한 스트림 처리 수행
  - 애플리케이션에 Query를 제공하기 위한 상태 저장소
- 익숙한 SQL syntax를 사용하여 이벤트 스트리밍 애플리케이션을 개발
- Pull/Push 방식 제공

Flink/ksqlDB를 활용한 실시간 데이터 스트림 처리(데이터 가공)

Confluent Platform의 이벤트 스트리밍 처리용 데이터베이스인 ksqlDB를 활용하여, Kafka Streams 만 사용하는 경우 보다 조직 내에서 더 많은 개발자에게 스트림 처리 및 이벤트 스트리밍 애플리케이션을 개발할 수 있는 손쉬운 방안을 제공합니다.

스트리밍 처리용 데이터베이스

이벤트 스트리밍 처리 애플리케이션을 쉽게 개발

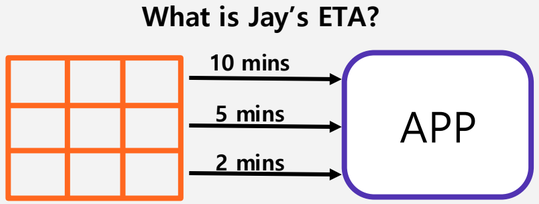
익숙하고 가벼운 SQL syntax를 사용하여 완전한 애플리케이션 구축

```
CREATE STREAM payments(user VARCHAR,
payment_amount INT)
WITH (kafka_topic = 'all_payments',
      key = 'user',
      value_format = 'avro');
```

Kafka의 입력 이벤트 스트림을 기반으로 새로운 스트림 및 테이블 생성

USER	Payment
Jay	$10
Sue	$15
Fred	$5
...	...

새로운 이벤트가 발생하면 즉시 반응하는 애플리케이션 구축

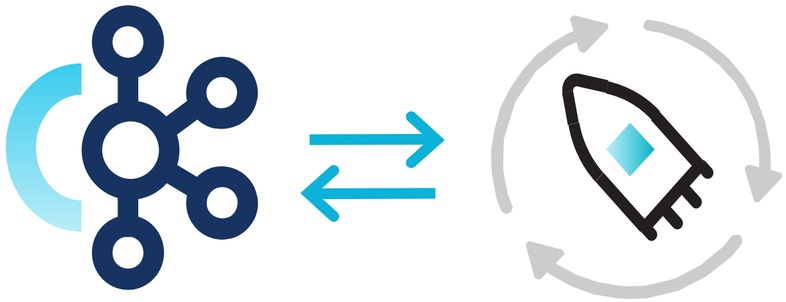


Control Center로 모든 클러스터의 요약 보기, 쿼리 개발 및 실행



Flink/ksqlDB - SQL를 이용한 실시간 스트리밍 ETL 구성

Flink/ksqlDB를 활용한 실시간 스트리밍 ETL 지원(Scheduling 배치가 필요 없는 실시간 처리 기능 지원)을 지원하고, Data Pipeline구성이 가능합니다.



Kafka
Storage

Flink
& ksqlDB
Compute

Transform	Filter
Join	Aggregate
Window	Query

End-to-End 실시간 데이터 흐름을 SQL로 구현

이벤트 캡처, 커넥터 실행

```
CREATE SOURCE CONNECTOR riders WITH (
  'connector.class' = 'JdbcSourceConnector',
  'connection.url'  = 'jdbc:postgresql://...',
  'topic.prefix'    = 'rider',
  'table.whitelist' = 'geoEvents, profiles',
  'key'             = 'profile_id',
  ...);
```

지속적인 변환 수행
(aggregate, filter, join)

```
CREATE STREAM locations AS
  SELECT rideId, latitude, longitude,
         GEO_DISTANCE(latitude, longitude,
                      dstLatitude, dstLongitude
                      ) AS kmToDst
  FROM geoEvents
  EMIT CHANGES;
```

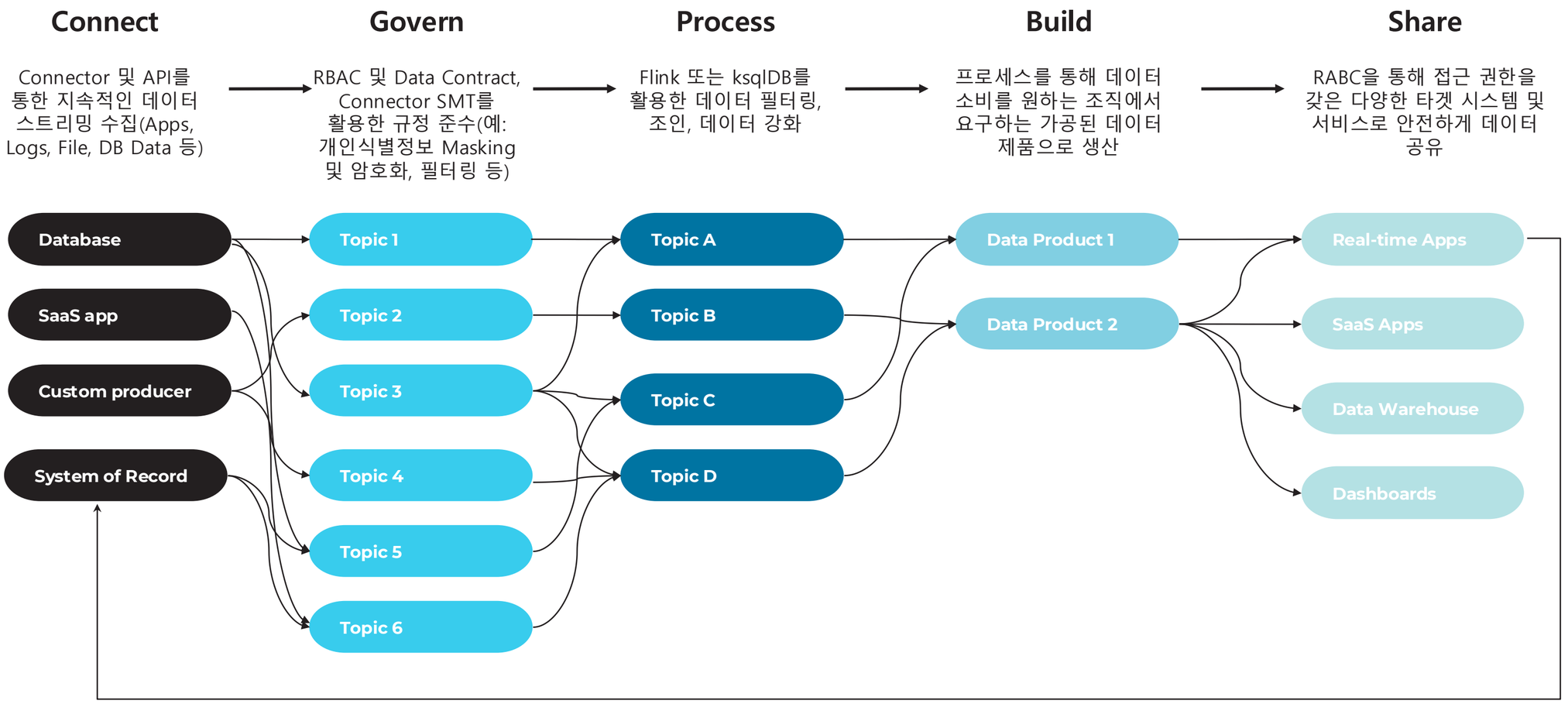
Materialized views 생성

```
CREATE TABLE activePromotions AS
  SELECT rideId,
         qualifyPromotion(kmToDst) AS promotion
  FROM locations
  GROUP BY rideId
  EMIT CHANGES;
```

Materialized views에 대한 조회 제공

```
SELECT rideId, promotion
FROM activePromotions
WHERE ROWKEY = '6fd0fcdb';
```

조직에서 요구하는 거버넌스와 규정 준수하는 데이터 파이프라인 구축(데이터 수집에서 가공, 활용, 공유)



Flink/ksqlDB - JSON Struct (SQL을 이용한 데이터 파이프 라인)



```
{
    "id": "STBCKS289803838HHDHD",
    "transaction": {
        "num_shares": 50000,
        "amount": 50044568.89,
        "txn_ts": "2020-11-18 02:31:43",
        "customer": {
            "first_name": "Junhee",
            "last_name": “Shin",
            "id": 1234567,
            "email": "jshin@acme.com"
        },
        "company": {
            "name": "ACME Corp",
            "ticker": "ACMC",
            "id": "ACME837275222752952",
            "address": "Anytown USA,
333333"
        }
    }
}
```

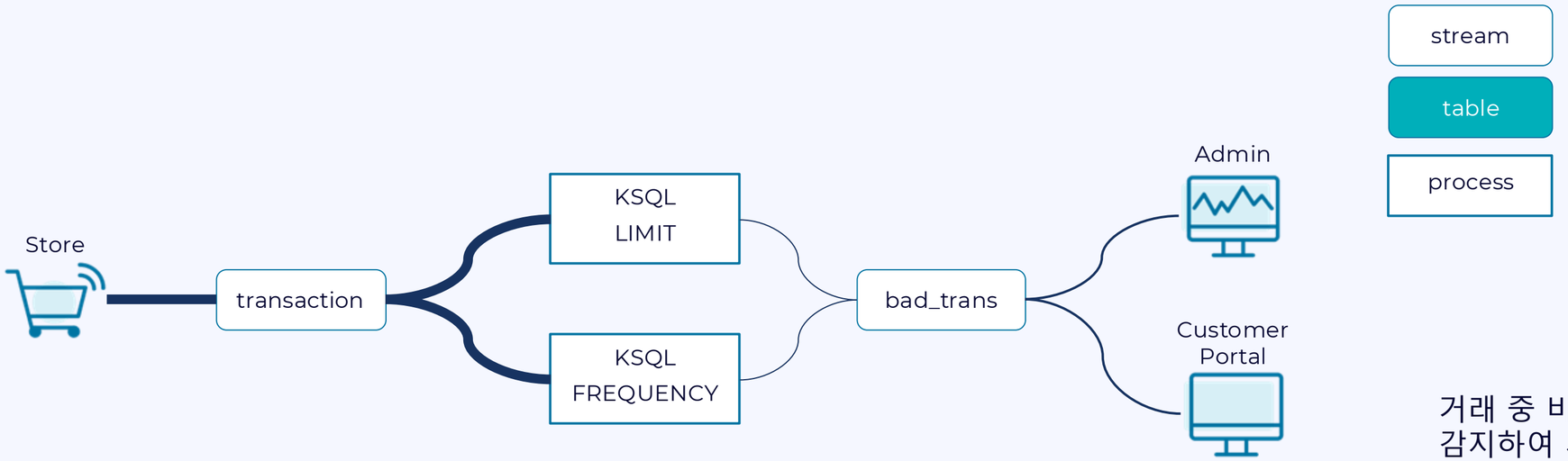
```
CREATE STREAM TRANSACTION_STREAM (
    id VARCHAR,
    trans STRUCT<
        num_shares INT,
        amount DOUBLE,
        txn_ts VARCHAR,
        customer STRUCT<
            first_name VARCHAR,
            last_name VARCHAR,
            id INT,
            email VARCHAR>,
        company STRUCT<
            name VARCHAR,
            ticker VARCHAR,
            id VARCHAR,
            address VARCHAR>
    >);
```

```
SELECT
    TRANS->num_shares AS SHARES,
    TRANS->CUSTOMER->ID AS CUST_ID,
    TRANS->COMPANY->TICKER AS SYMBOL
FROM
    TRANSACTION_STREAM
EMIT CHANGES;
```

SHARES	CUST_ID	SYMBOL
5000	1234567	ACMC
3000	2345678	IMPC
20000	3456789	HECH



Payment - Fraud Detection (비정상 거래 감지)




```
INSERT INTO bad_trans
  SELECT
    account_id,
    'over_limit',
    SUM(amount)
  FROM transactions
  GROUP BY account_id
  HAVING SUM(amount) > 10000000
  WINDOW TUMBLING (SIZE 1 HOUR)
EMIT CHANGES;
```

1시간 동안 1천만원 이상의 거래 발생

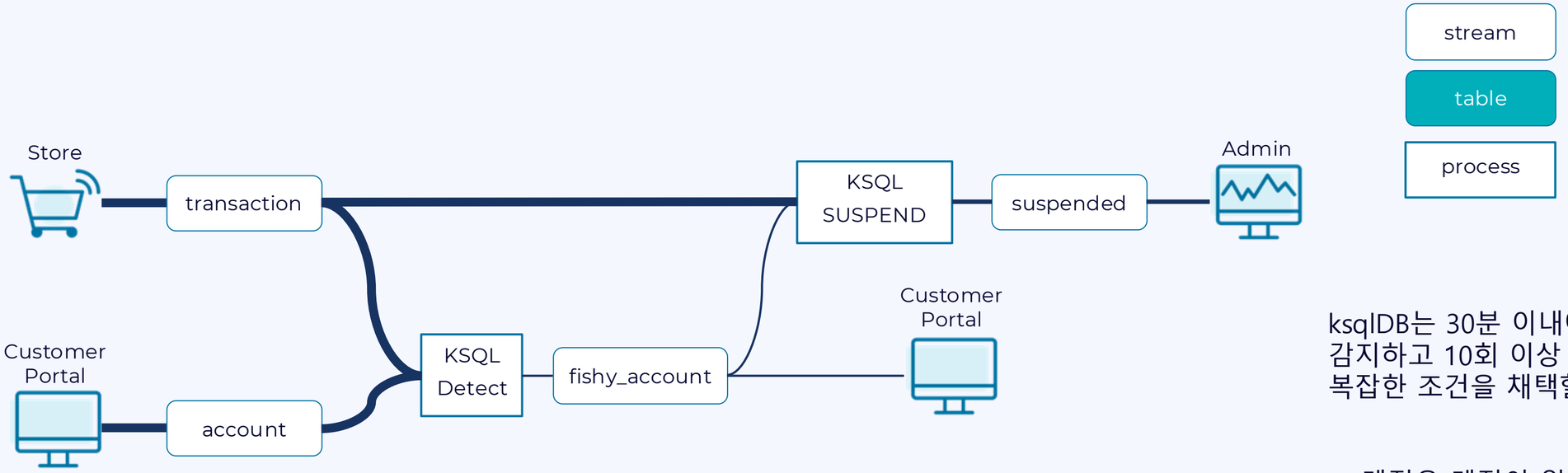
```
INSERT INTO bad_trans
  SELECT
    account_id,
    'too_frequent',
    COUNT(*)
  FROM transactions
  GROUP BY account_id
  HAVING COUNT(*) > 10
  WINDOW TUMBLING (SIZE 1 DAY)
EMIT CHANGES;
```

하루동안 10건 이상 거래 발생

거래 중 비정상적인 계정 행위를 실시간으로 감지하여 사기 위험을 최소화합니다.

- 하나의 트랜잭션 스트림은 다양한 규칙에 따라 별도로 대기열에 추가되며 의심스러운 트랜잭션의 단일 소스로 요약됩니다.
- 각 규칙은 기존 규칙(시간별 거래 총액, 일일 거래 수 등)에 영향을 주지 않고 독립적으로 정의할 수 있습니다.
- 동일한 의심스러운 활동 이벤트는 판매 운영 대시보드에 요약하고 최종 사용자 고객의 모바일 앱에 직접 푸시하는 등 다양한 용도로 사용될 수 있습니다.

Payment - Fraud Detection ver.2 (비정상 거래)



ksqlDB는 30분 이내에 업데이트되는 계정을 감지하고 10회 이상 트랜잭션을 수행하는 보다 복잡한 조건을 채택할 수 있습니다.

- 계정은 계정이 업데이트되어야만 생성됩니다. - 계정 정보를 변경하지 않고 자주 사용하는 사용자는 캡처되지 않습니다.
- 모바일 앱을 통해 사용자에게 계정에 대해 즉시 경고합니다.
- 의심스러운 계정은 정지되었습니다. 최종 사용자 확인이나 백엔드 관리 작업이 필요합니다.

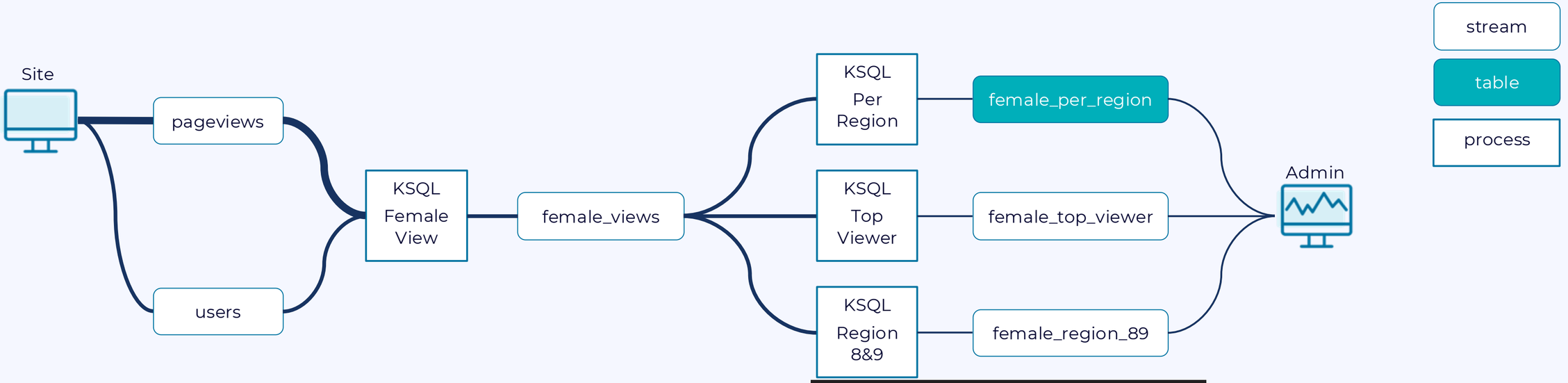
```
CREATE STREAM fishy_account AS
  SELECT
    account.id,
    COUNT(*)
  FROM transaction trans
  INNER JOIN account
    ON trans.account_id = account.id
  GROUP BY account.id
  HAVING COUNT(*) >= 10
  WINDOW TUMBLING (SIZE 30 MINUTES)
EMIT CHANGES;
```

30분 동안 10건 이상의 거래

```
CREATE STREAM suspended AS
  SELECT
    transaction.id
    account.id,
    transaction.amount
  FROM fishy_account
  INNER JOIN transaction
    ON bad_trans.account_id
       = transaction.account.id
EMIT CHANGES;
```

이상 거래 계정에 대한 계정 정지

Site Access Analysis (고객 개인화 서비스 적용)




```
CREATE STREAM female_views AS
  SELECT
    USERS.ID USERID,
    PAGEVIEWS.PAGEID PAGEID,
    USERS.REGIONID REGIONID,
    USERS.GENDER GENDER
  FROM pageviews
  LEFT OUTER JOIN USERS users ON
    ((pageviews.USERID = users.ID))
  WHERE (users.GENDER = 'FEMALE')
EMIT CHANGES;
```

여성 유저의 페이지뷰 분석

```
CREATE TABLE female_per_region AS
  SELECT
    views.GENDER GENDER,
    views.REGIONID REGIONID,
    COUNT(*) NUMUSERS
  FROM FEMALE_VIEWS views
  WINDOW TUMBLING ( SIZE 30 SECONDS )
  GROUP BY views.GENDER, views.REGIONID
  HAVING (COUNT(*) > 1)
;
```

30초 텀블링 동안 여성 유저의 페이지뷰가 발생된 지역 분석

여성 유저의 탑 페이지뷰 분석

```
CREATE STREAM female_top_viewer AS
  SELECT *
  FROM female_views views
  WHERE (CAST(SPLIT(views.PAGEID,
    '_')[2] AS INTEGER) >= 50)
EMIT CHANGES;
```

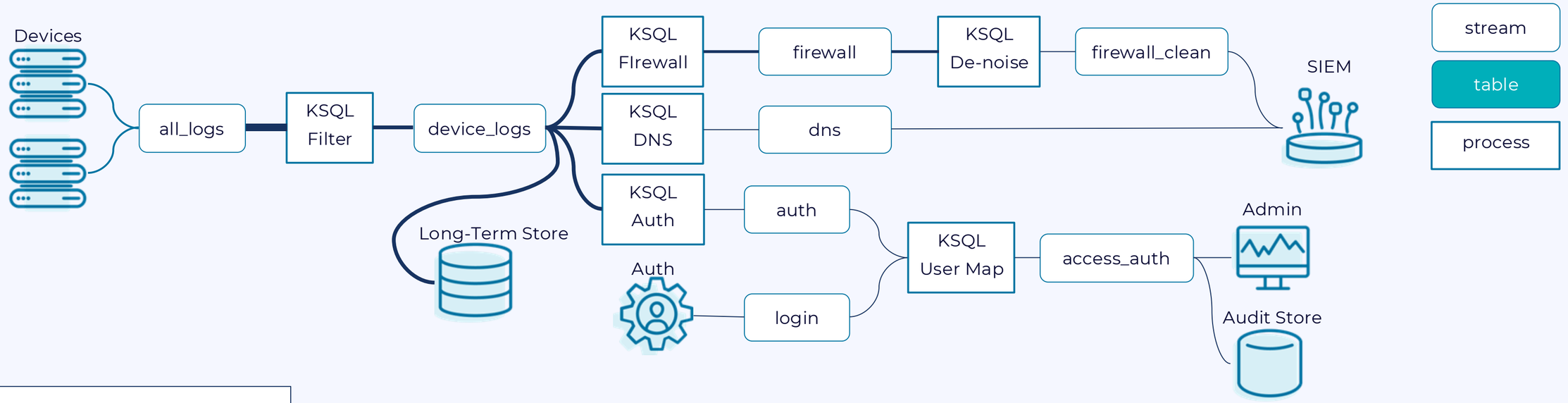
```
CREATE STREAM female_region_89 AS
  SELECT *
  FROM female_views views
  WHERE ((views.REGIONID LIKE '%_8') OR
    (views.REGIONID LIKE '%_9'))
EMIT CHANGES;
```

여성 유저 페이지뷰 8%~9% 지역 분석

실시간 사이트 접속 분석

- 하나의 시스템에서 2개의 서로 다른 스트림을 추출합니다. 하나는 페이지 뷰용이고 다른 하나는 사용자 로그인용입니다.
- 데이터를 결합하고 필터링한 후 결과를 다양한 분석 사용 사례에 사용할 수 있습니다.
- 필요에 따라 결과가 구체화되거나 최종 시스템으로 스트리밍될 수 있습니다.

SIEM - Cost optimization(로그 유량 제어)




```
CREATE STREAM device_logs AS
  SELECT
    event_id,
    event_type,
    EXPLODE (event)
    timestamp,
    ....
  FROM all_logs
  WHERE event_type = 'firewall'
    OR event_type = 'dns'
    OR event_type = 'auth'
EMIT CHANGES;
```

Event 타입별 구조화

```
CREATE STREAM auth AS
CREATE STREAM dns AS
CREATE STREAM firewall AS
  SELECT
    event_type,
    port,
    message,
    timestamp,
    ....
  FROM device_logs
  WHERE event_type = 'firewall'
EMIT CHANGES;
```

Event 타입별 스트림 라우팅

```
CREATE STREAM access_auth AS
  SELECT
    auth.auth_id,
    auth.auth_type,
    auth.timestamp,
    login.authorized_by
    login.timestamp,
    ....
  FROM auth
  OUTER_JOIN login
    ON auth.login_id = login.login_id
EMIT CHANGES;
```

로그인 아이디와 인증 아이디 매핑

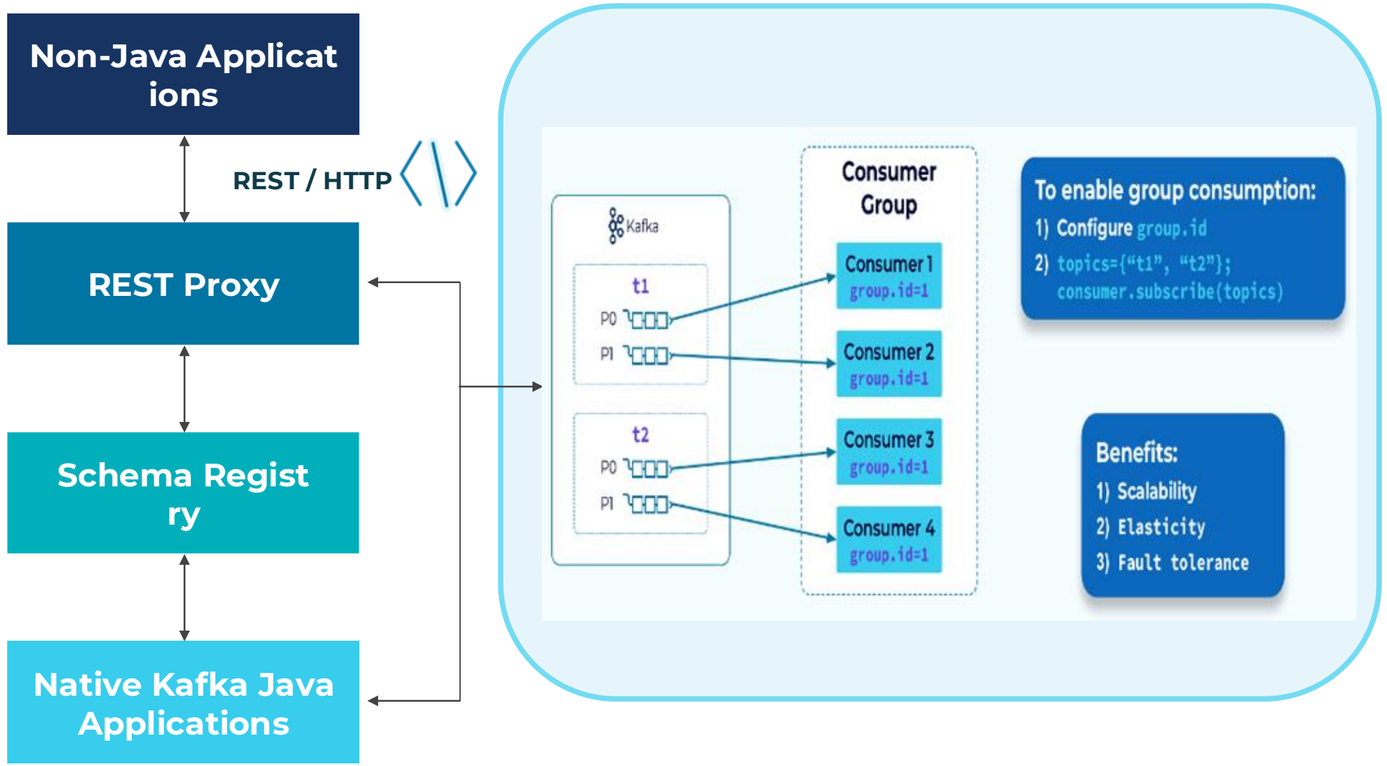
SIEM 시스템으로 들어가는 로그의 양을 제한합니다.

- N-M 연결(device-system)에서 N-1-M (device-confluent-system)로 전환하여 연결 수를 최소화합니다.
- 로그는 SIEM 시스템에 도달하기 전에 정리, 필터링, 집계됩니다.
- 로그는 장기간 저렴한 오브젝트 스토리지에 저장하고 필요할 때 요청 시 사용할 수 있습니다.

API 개발 환경과 유연한 통합을 위한 RESTfull API 인터페이스 제공

REST Proxy는 Confluent에서 제공하는 Kafka Cluster를 위한 RESTful 인터페이스입니다.
Consumer, Producer, Admin 등 Kafka-clien에서 사용하던 다양한 기능들을 REST API(HTTP Protocol) 를 통해 쉽게 연동할 수 있습니다.
Kafka는 Consumer Group 기능을 통해 Consumer Group에 속한 다수의 Consumer가 중복 Polling이 되지 않도록 지원합니다.

REST Proxy로 HTTP Protocol 지원



- REST Proxy Consumer는 Stateful
- Consumer는 Topic의 메시지를 순차적 읽음
- REST Proxy Instance에서 Consumer 재사용
- 동일한 REST Proxy Instance로 라우팅 될 수 있도록 Sticky Load Balancer Session을 권장
- REST Proxy에서 지원하는 Data format
  - JSON, base64 encoded raw bytes
  - JSON-encoded Avro, Protobuf, 또는 JSON Schema
  - Admin 오퍼레이션
  - Schema Registry integration for Avro, Protobuf, 또는 JSON Schema

안정적인 서비스를 위한 보안 및 거버넌스 지원

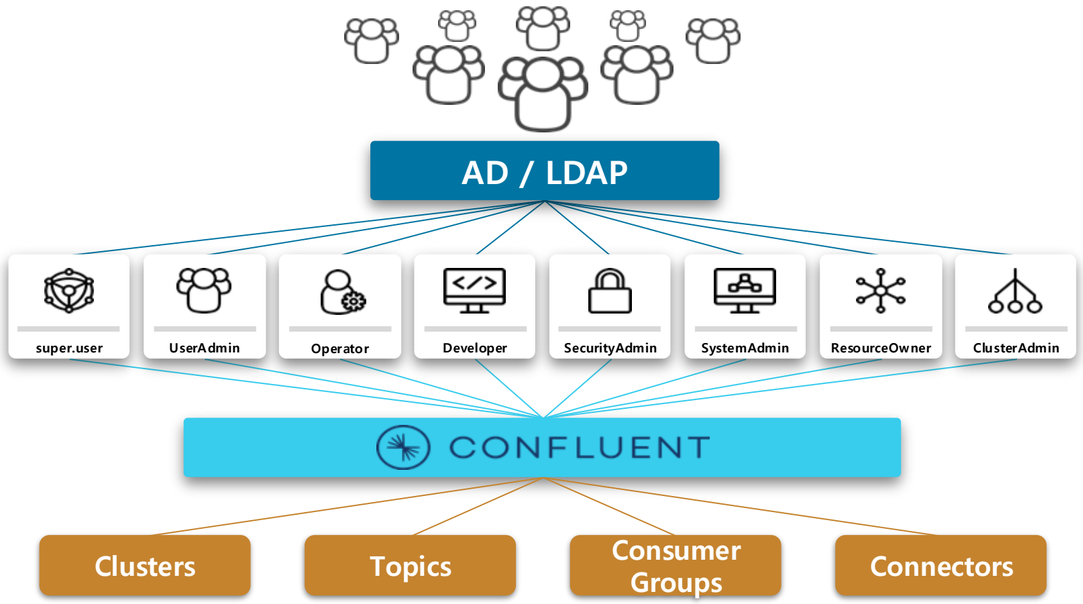
RBAC(Role-based Access Control) 기능 제공

Confluent Platform은 오픈소스 Apache Kafka와 달리, RBAC(Role-Based Access Control) 기능을 도입하여 플랫폼 전체 리소스들에 대해서 세밀하게 조정된 접근 권한을 중앙에서 제어할 수 있습니다. Cluster 레벨에서부터 개별 Topic, Consumer Group 및 개별 Connector 등의 리소스들에 대해서 사용자 및 그룹의 권한을 제어할 수 있습니다.

Role-Based Access Control

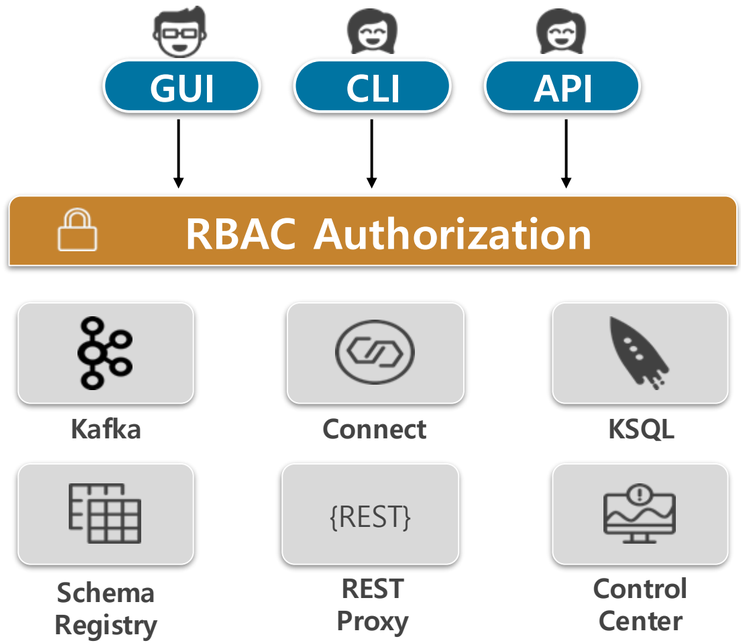
중요 리소스에 대한 세분화된 권한 관리 지원

- AD(Active Directory)및 LDAP과 같은 기존 인증 시스템을 연동
- 멀티 테넌트 Connect 클러스터를 실행할 수 있으며, 이를 통해 여러 팀이 동일한 클러스터 내에서 공존하며 리소스를 효율적으로 사용 가능



Confluent Platform 전반의 보안 표준화

- 모든 사용자 인터페이스(GUI, CLI 및 API)에 RBAC 적용
- 모든 Confluent Platform 구성 요소(Control Center, Kafka Connect, KSQL, Schema Registry 및 REST/MQTT Proxy)에 대한 포괄적인 접근 제어를 제공



RBAC(Role-based Access Control) 기능 제공

RBAC(Role-Based Access Control) 기능을 제공하여 플랫폼 전체 리소스들에 대해서 세밀하게 조정된 접근 권한을 중앙에서 제어할 수 있습니다.
Cluster 레벨에서부터 개별 Topic, Consumer Group 및 개별 Connector 등의 리소스들에 대해서 사용자 및 그룹의 권한을 제어할 수 있습니다.
동일한 API도 API-Key와 바인딩된 사용자의 Role 에 따라 다르게 동작합니다.

Confluent Platform에서 제공하는 사전 정의된 Role

Role Name	Role Scope	View Role Bindings of Others	Manage Role Bindings	Monitor	Resource Read	Resource Write	Resource Manage
super.user	Cluster -level	Yes	Yes	Yes	Yes	Yes	Yes
SystemAdmin	Cluster -level	Yes	Yes	Yes	Yes	Yes	Yes
ClusterAdmin	Cluster -level	No	No	Yes	Yes	Yes	Yes
UserAdmin	Cluster -level	Yes	Yes	No	No	No	No
SecurityAdmin	Cluster -level	Yes	No	No	No	No	No
AuditAdmin	Cluster -level	No	No	No	No	No	Yes (1)
Operator	Cluster -level	No	No	Yes	No	No	Yes (2)
ResourceOwner	Cluster -level	Yes	Yes	No	Yes	Yes	Yes
DeveloperRead	Cluster -level	No	No	No	Yes	No	No
DeveloperWrite	Cluster -level	No	No	No	No	Yes	No
DeveloperManage	Cluster -level	No	No	No	No	No	Yes

감사를 위한 Secret Protection과 Audit Logs 기능 제공

Confluent Platform은 오픈소스 Apache Kafka와 다르게, Kafka 파일뿐만 아니라 만들어진 모든 구성 파일(예: DB 접속 정보)을 암호화하고 이는 모든 Confluent Platform 구성 요소에 지원되므로, 플랫폼 전체 표준화를 통해 중요하고 민감한 정보를 보호할 수 있습니다. 또한, 비정상적인 동작을 감지하고 잠재적인 보안 위협을 식별하며 정보 보안과 관련된 규정 준수 요구 사항을 해결하기 위해, 사용자가 수행한 작업을 추적할 수 있도록 구조화된 감사 로그 기능이 추가되었습니다.

Secret Protection/Structured Audit Logs

Secret Protection

- 암호 같은 민감한 정보를 보호
  - master encryption key를 사용하여 설정 파일에 대한 안전한 암호화
- Platform 전반에 대한 표준화
  - Kafka, Connect, KSQL, Schema Registry, REST Proxy 및 Control Center의 Property 암호화 지원

PLAINTEXT 사용시 정보 노출 위험 존재

```
security.protocol=SASL_PLAINTEXT
sasl.jaas.config=org.apache.kafka.common.security.oauthbearer.OAuthBearerLogin
Module required ₩
username="robertsmith" ₩
password="ABC12345" ₩
```

VS.

Secret Protection 사용시 암호화로 정보 보호

```
security.protocol=SASL_PLAINTEXT
sasl.jaas.config=${securepass:/home/robertsmith/secrets.text:robertsmith.properti
es/sasl.jaas.config}
config.providers=securepass
```

Structured Audit Logs

- 비정상적인 동작 감지 및 규정 준수 요구 사항 충족
  - Capture logs in dedicated topics에서 로그 캡처
  - KSQL로 처리하거나 외부 시스템으로 전송 (예 : Splunk, S3)
- 산업 표준 지원
  - CloudEvents specification을 사용하여 로그 구문을 정의함

Cloud Events Integration https://cloudevents.io/

Event	Description	Category	Capture Default
Authorize	An RBAC authorization is being requested.	MANAGEMENT	Yes
CreateTopics	A topic is being created.	MANAGEMENT	Yes
Produce	A Kafka producer is writing a batch of records to a topic.	PRODUCE	No
FetchConsumer	A Kafka consumer is reading a batch of records from a topic.	CONSUME	No
LeaderAndIsr	Controller is sending leader and ISR state to a broker.	INTERBROKER	No

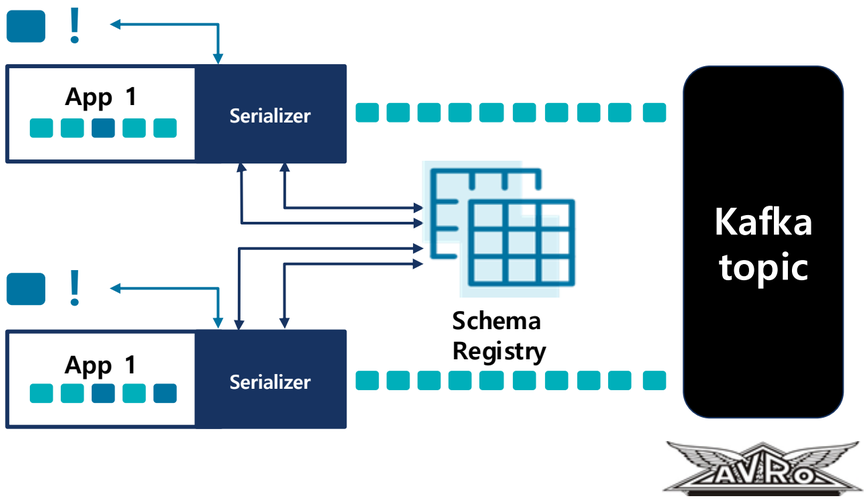
Schema Registry/Schema Validation

Confluent Platform은 개발자가 표준 스키마를 정의하고 모든 애플리케이션을 등록하여 스키마를 준수하는지 확인하기 위한 RESTful 인터페이스가 있는 중앙 저장소인 Schema Registry를 제공합니다. 또한, 오픈소스 Apache Kafka와는 다르게, Confluent Platform은 Schema Validation을 제공하여 Kafka Broker에서 직접 스키마를 검증하고 적용하는 프로그래밍 방식을 제공합니다.

Schema Registry/Schema Validation

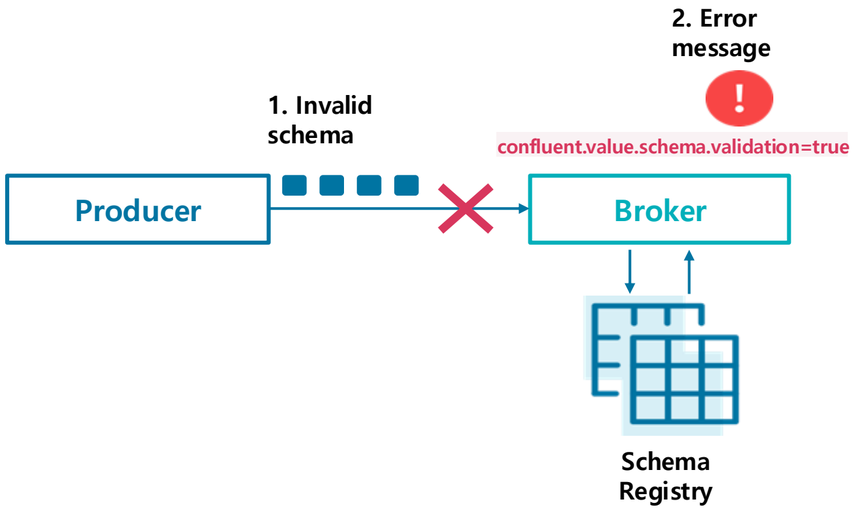
애플리케이션 개발 호환성을 가능하게

- 개발자는 모든 표준 스키마의 버전별 기록을 저장 및 공유하고 클라이언트(애플리케이션) 수준에서 데이터 호환성을 검증
- 표준 schema를 사용하여 개발
  - 모든 표준 schema의 버전 히스토리 저장 및 공유
  - 클라이언트 레벨에서 데이터 호환성 검사
- 운영 복잡성 제거
  - Schema 표준화를 위한 개발자들간의 시간 소모적인 업무를 제거

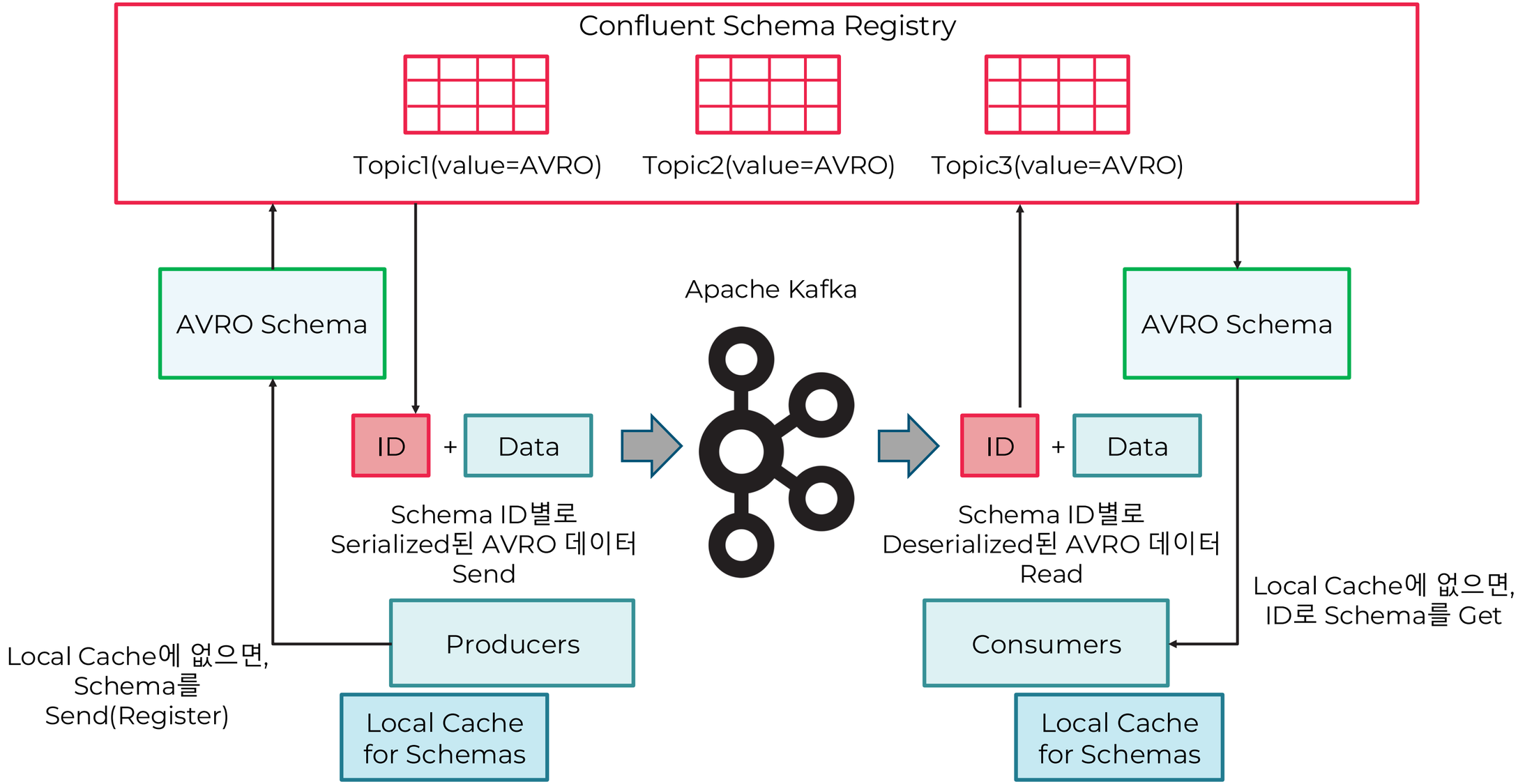


운영 환경에 안전하게 배포

- Kafka 스키마를 인식하기 위해 Schema Validation은 클라이언트가 항상 수행했던 것과 동일한 방식으로 Broker에서 Confluent Schema Registry로 직접 인터페이스를 추가
- 안정적으로 schema 확장
  - 자동화된 Broker측 스키마 검증 및 시행
  - Broker에서 Confluent Schema Registry 로의 직접 인터페이스
- 세분화된 제어
  - Topic level에서 유효성 검사 활성화



메시지 형식 호환성을 제공하는 Confluent Schema Registry



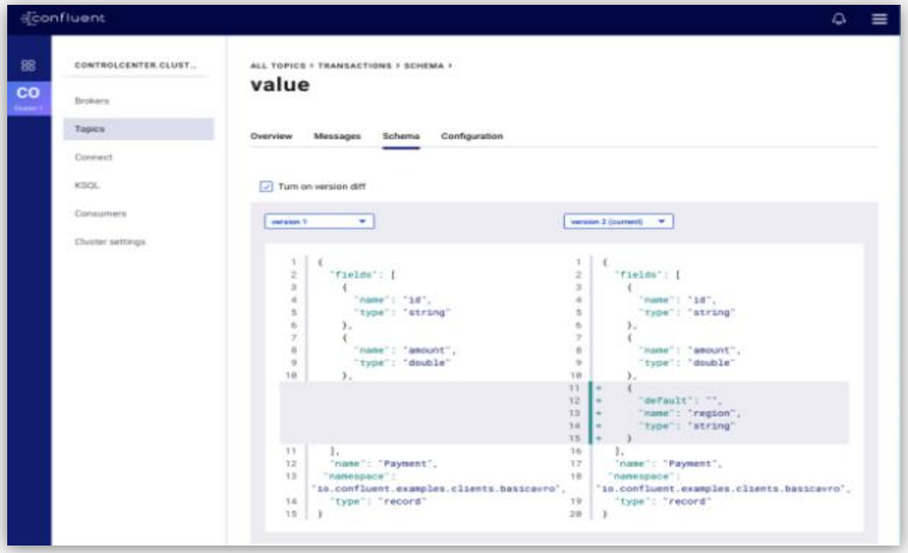
Schema Registry/Schema Validation(계속)

Confluent Platform은 오픈소스 Apache Kafka와 달리, Confluent Platform 운영자가 스키마를 중앙에서 관리할 수 있도록 Confluent Control Center와 통합되어 있습니다. Confluent Control Center GUI 화면을 통해 Topic 스키마를 생성, 편집 및 관리하고 스키마 버전을 비교할 수 있고, Topic을 만들거나 편집할 때 Topic 레벨에서 스키마 유효성 검사에 대해서 구성할 수 있습니다.

Confluent Control Center와의 통합

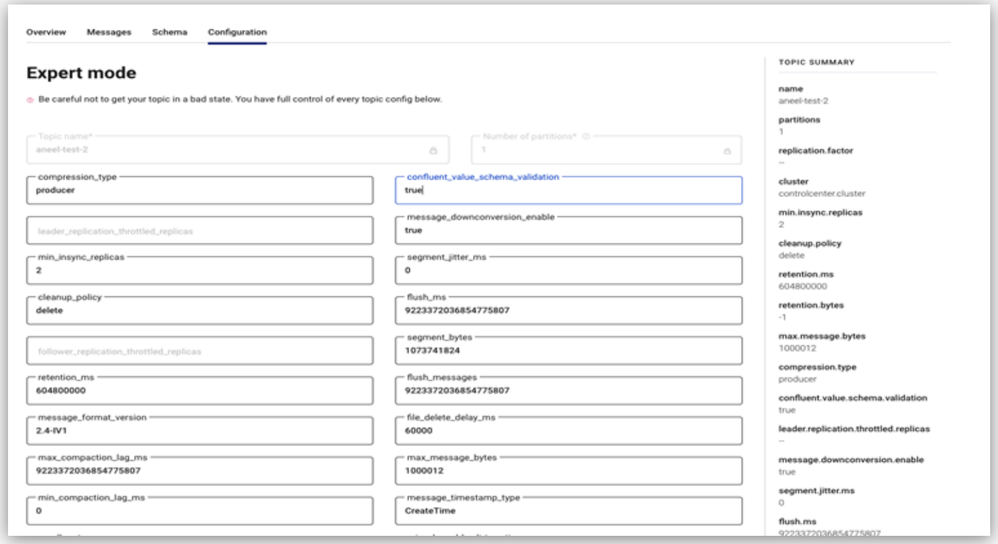
Schema Registry 관리

중앙에서 Topic Schema를 생성, 편집 및 확인하고 Schema version을 비교



Schema Validation 구성

Topic를 만들거나 편집할 때 Topic level에서 기능 사용



Confluent 인증 정보

Confluent는 다양한 산업군과 기관에서 요구하는 보안, 규정 준수 및 개인 정보 보호에 대한 기업 고객의 요구 사항을 지원하도록 설계되었습니다.



엔터프라이즈 등급 보안

- Confluent는 고객 데이터를 안전하게 보호하도록 설계된 계층형 보안 제어를 구현합니다.
- Confluent의 보안 포털을 통해 보안 백서와 규정 준수 인증(ISO, SOC 2)을 확인 할 수 있습니다.

Ensure Data Confidentiality

Confluent Cloud is uncompromising when it comes to data security. It secures your data through encryption at rest and in transit, and offers additional options,, including BYOK encryption and private networking connectivity.

- Encrypt data at rest with Bring Your Own Key (BYOK) options
- Data-in-motion encryption
- Secure private network connectivity

Control Identity and Access Management

CConfluent offers rich Identity and Access Management controls that helps you manage and monitor where and who accesses the data.

- Cluster Authorization with Role-Based Access Control (RBAC)
- Audit Logs

Bug Bounty

Confluent is committed to working with industry experts and security researchers to ensure our products are the most secure they can be for our customers. Please note that our bug bounty program is currently invitation-only and we do not add researchers based on undisclosed issues. If you have found a security impacting issue, we encourage you to share your findings with us at bugbounty@confluent.io. Based on the nature of the issue and the quality of the report, you could potentially be eligible to join the program.

Penetration Testing

Confluent employs a third party security firm to perform Security, Vulnerability, and Penetration testing for all our products. These are run at least annually and findings are remediated according to their criticality and prioritization. Confluent's penetration and security assessment test summaries can be requested.

출처: https://www.confluent.io/ko-kr/trust-and-security/

보안 이슈에 신속하게 대응할 수 있는 기술지원 체계 제공

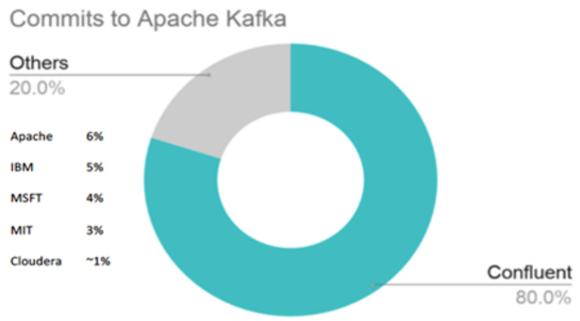
- Confluent는 Apache Kafka 커미터 레벨의 전문가의 지원과 대응을 통해 신속한 이슈(버그, 보안 등)에 대한 해결을 지원 합니다.
- 특히 보안 및 중요 버그에 대해 오픈소스 커뮤니티 Apache Kafka 보다 빠른 패치를 제공하여 엔터프라이즈 환경에서 보다 안전하게 운영 할 수 있도록 지원합니다.
- 또한 고객의 기능 개선 요구를 적극 수용하여 향후 Apache Kafka와 Confluent Platform에 반영 할 수 있도록 적극적인 지원을 합니다.
- Apache Kafka에 가장 많은 코드 기여를 하고 있을 뿐 아니라 Schema Registry, REST Proxy와 같은 엔터프라이즈 환경에 꼭 필요한 카프카 요소를 직접 개발하여 커뮤니티 라이선스로 오픈소스로 제공 하고 kafka 생태계에서 잘 사용할 수 있도록 코드 유지 보수 및 패치를 제공하고 있습니다.

Confluent Platform Release 정책

릴리즈 타입	버저닝	컨텐츠 / 기능	릴리즈 주기
Hotfix	X.Y.Z-cpN (7.1.1-cp1)	단일 버그 픽스	고객 요청 및 필요 에 따라(비정기)
Quarterly Patch & Security	X.Y.N (6.1.1)	버그 픽스 묶음과 CVEs 픽스 묶음	매 분기 세번째 달
Minor	X.N.N (7.1.0)	새로운 기능, 버그 픽스 묶음과 CVEs 픽스 묶음	Apache Kafka 릴리스(AK)에 맞춰 연간 2회 조정
Major	N.N.N (8.0.0)	대규모/중요한 기능, API 변경, 버그 수정, CVE용 수정(예: KRaft / KIP-500)	연간 1회

Confluent Platform Release Note: https://docs.confluent.io/platform/current/release-notes/index.html

가장 많은 소스 코드 기여



- 전체 소스코드의 80% 기여
- Kafka Core, KSQL, Connector 등 다양한 Kafka 에코 시스템에 기여

Apache Kafka Committers: 커미터 기반의 전문적 기술 지원/대응

		PMC
Ismael Juma	CONFLUENT	✓
Jason Gustafson	CONFLUENT	✓
Guozhang Wang	CONFLUENT	✓
Jun Rao (PMC Chair)	CONFLUENT	✓
Matthias Sax	CONFLUENT	✓
Rajini Sivaram	CONFLUENT	
Neha Narkhede	CONFLUENT	✓
Damian Guy	CONFLUENT	
Manikumar Reddy	CONFLUENT	✓
David Arthur	CONFLUENT	
Onur Karaman	LinkedIn	
Chris Burroughs	Joyent	

		PMC
Colin McCabe	CONFLUENT	✓
John Roesler	CONFLUENT	✓
Ewen Cheslack	CONFLUENT	
Jay Kreps	CONFLUENT	✓
Sophie Blee-Goldman	CONFLUENT	
Bill Bejeck	CONFLUENT	
Boyang Chen	CONFLUENT	
Vahid Hashemian	Pinterest	✓
Grant Henke	phData	
Prashanth Menon	Student	✓
Henry Saputra	salesforce	
Jakob Homan	Lyft	

		PMC
Eno Thereska	aws	
Dong Lin	CONFLUENT	✓
Jiangjie Qin	Alibaba	✓
Joel Koshy	LinkedIn	
Mickael Maison	IBM	✓
Konstantine Karantasis	CONFLUENT	
Randall Hauch	CONFLUENT	
Joe Stein	MUFG	✓
Sriharsha Chintalapani	CONFLUENT	✓
Owen O'Malley	CONFLUENT	
Philip Rhodes	CONFLUENT	

DR/Hybrid 환경을 위한 특화된 고가용성 아키텍처 및 기술 제공

재해복구를 위한 특화된 기술 제공

Confluent는 엔터프라이즈 환경에서 요구하는 다양한 데이터 센터간 데이터 보호기능을 제공합니다.

Replicator를 이용하여 데이터 센터간 topic 복제와 Cluster Linking을 이용한 Kafka Cluster Level의 복제(topic data, topic configs, consumer offsets, ACLs)를 지원합니다.

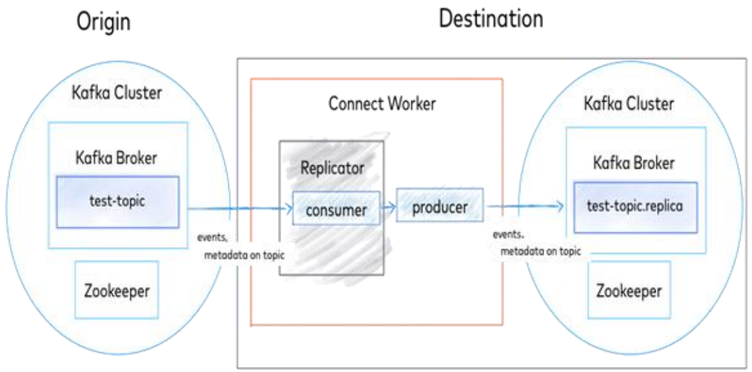
또한 Multi-Region Cluster 기능을 통해 3개의 데이터 센터에 걸쳐 확장된 Kafka Cluster를 구성하여 RTO와 RTO를 0에 가깝게 Active-Active 서비스를 지원합니다.

Replicator

한 클러스터에서 다른 클러스터로 데이터를 복사하는 비동기식 애플리케이션입니다.

Replicator를 구성하기 위해서는 Kafka Cluster에 별도의 Replicator Node 구성이 필요합니다.

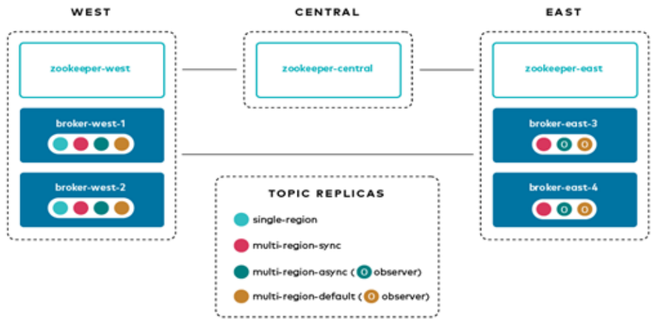
멀티 데이터 센터간 데이터 복제가 필요한 Topic을 Replicator를 이용하여 쉽게 복제하여 데이터를 안전하게 보호할 수 있습니다.



MRC(Multi-Region Cluster)

동기식 및 비동기식으로 복제할 수 있는 여러 지역에 걸쳐 확장된 클러스터입니다. 최소 3개의 데이터 센터/지역이 필요합니다(Zookeeper의 경우 최소 3개 지역에 배포 필요).

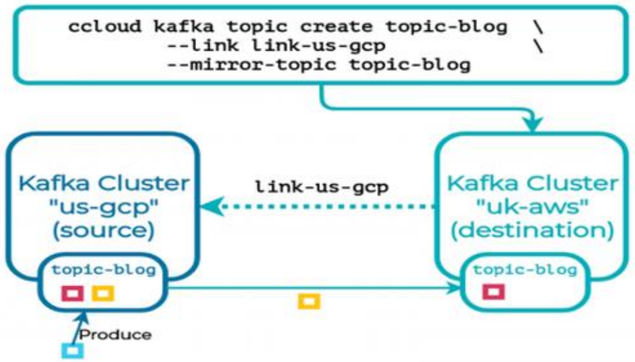
오프셋이 보존되며 커스텀 코드 없이 자동으로 클라이언트 페일오버가 가능합니다.



Cluster Linking

Cluster Linking을 사용하면 추가적인 connector 연결 없이 클러스터를 함께 직접 연결하고 한 클러스터에서 다른 클러스터로 Topic을 미러링할 수 있습니다.

Cluster Linking을 사용하면 다중 데이터 센터, 다중 클러스터 및 하이브리드 클라우드 배포를 훨씬 쉽게 구축할 수 있습니다.

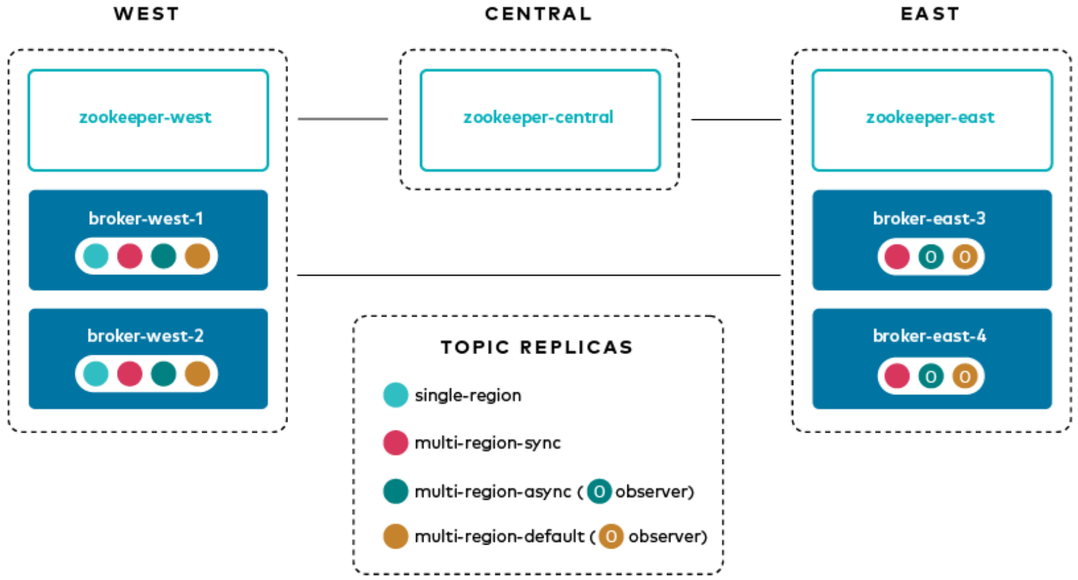


Multi-Region Cluster (MRC)

동기식 및 비동기식으로 복제할 수 있는 여러 지역에 걸쳐 확장된 클러스터입니다. 최소 3개의 데이터 센터/지역이 필요합니다(Zookeeper의 경우 최소 3개 지역에 배포 필요).

오프셋이 보존되며 커스텀 코드 없이 자동으로 클라이언트 페일오버가 가능합니다.

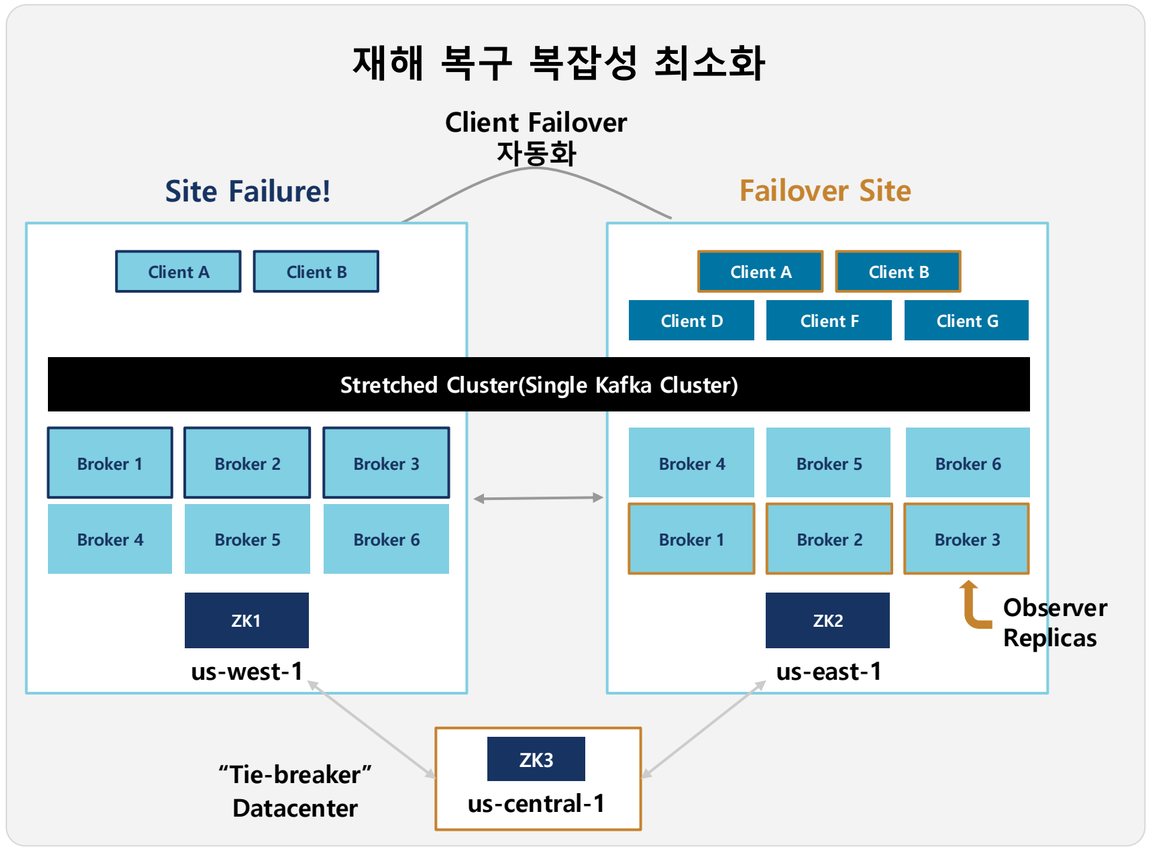
참고: Rack에는 토픽에 대한 동기 또는 비동기 복제본만 있을 수 있으며 둘 다 가질 수는 없습니다. 그러나 DC/Zone에 여러 rack을 가질 수 있습니다.

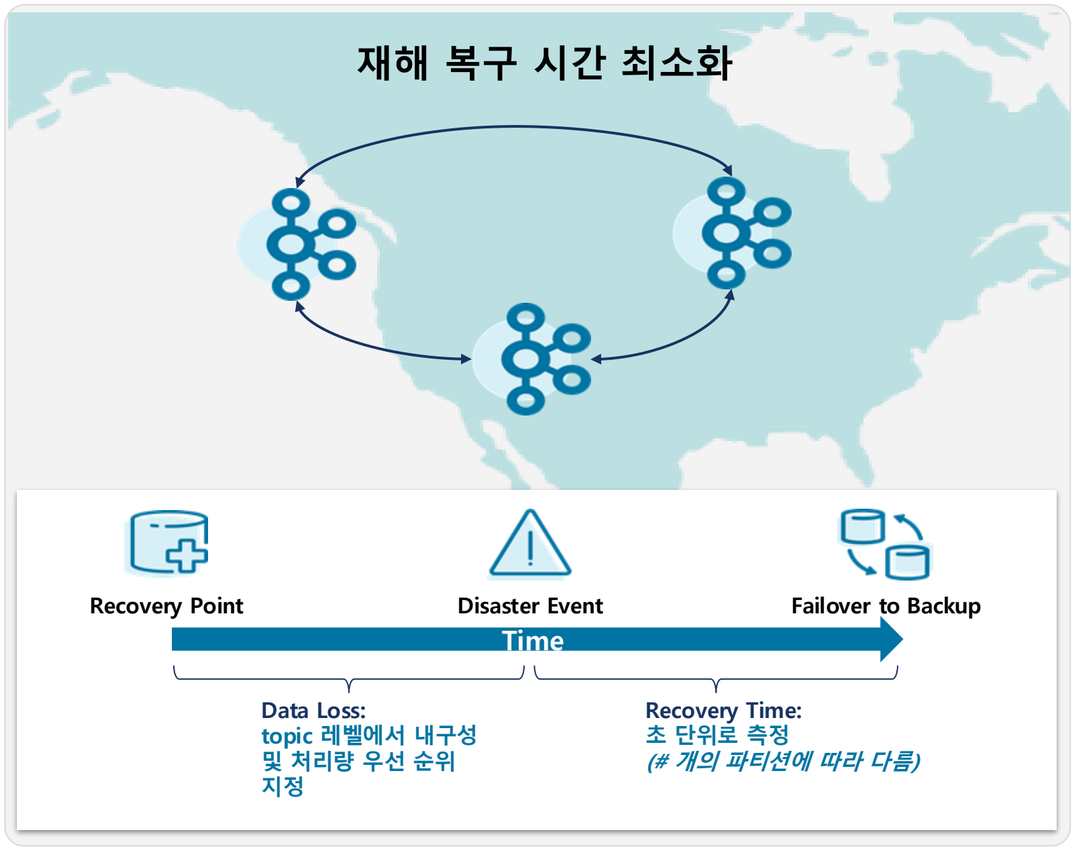


Multi-Region Clusters: 글로벌 가용성 제공

Confluent Platform의 Multi-Region Clusters는 재해 복구와 관련된 모든 작업을 간소화합니다. Connect 클러스터를 가동할 필요가 없으며, 클러스터들이 여러 데이터 센터에 배포되고 Confluent Platform의 내부 복제 엔진을 활용하여 Topic 데이터가 여러 위치에 자동으로 복제되도록 합니다. 또한 자동화된 클라이언트 장애 조치를 지원하므로 운영자는 여러 개발 팀 간에 조정하거나 DNS 재구성 및 오프셋 변환 없이 백업 데이터 센터로 즉시 장애 조치하도록 할 수 있습니다.

자동화된 장애 복구 방안





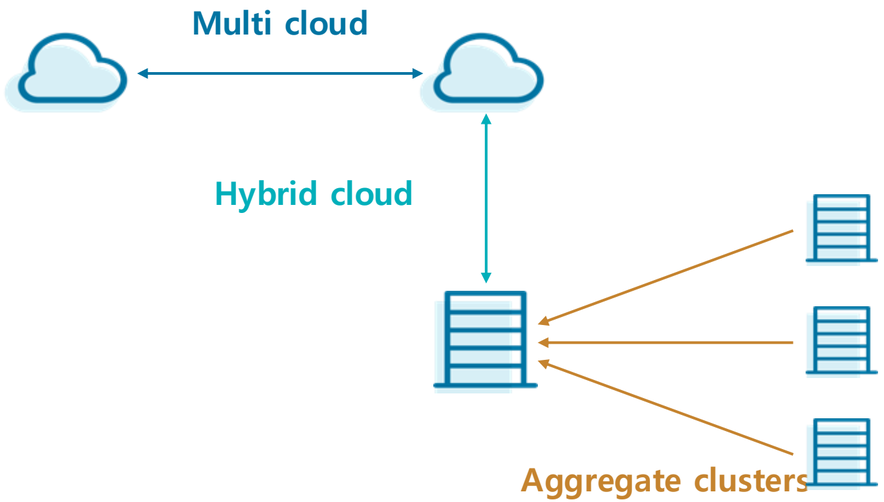
Replicator: DR 구성 및 데이터 마이그레이션을 위한 복제 도구

Confluent Platform은 집계용 클러스터 생성, 클라우드 공급자간 데이터 마이그레이션 또는 자체 관리형 클러스터를 Confluent Cloud와 같은 완전 관리형 Kafka 서비스에 브릿징하는 것과 같은 복제가 필요한 경우, 엔터프라이즈 환경에서 이미 검증이 완료된 Replicator를 사용할 수 있습니다.

멀티 클라우드 환경간의 데이터 마이그레이션

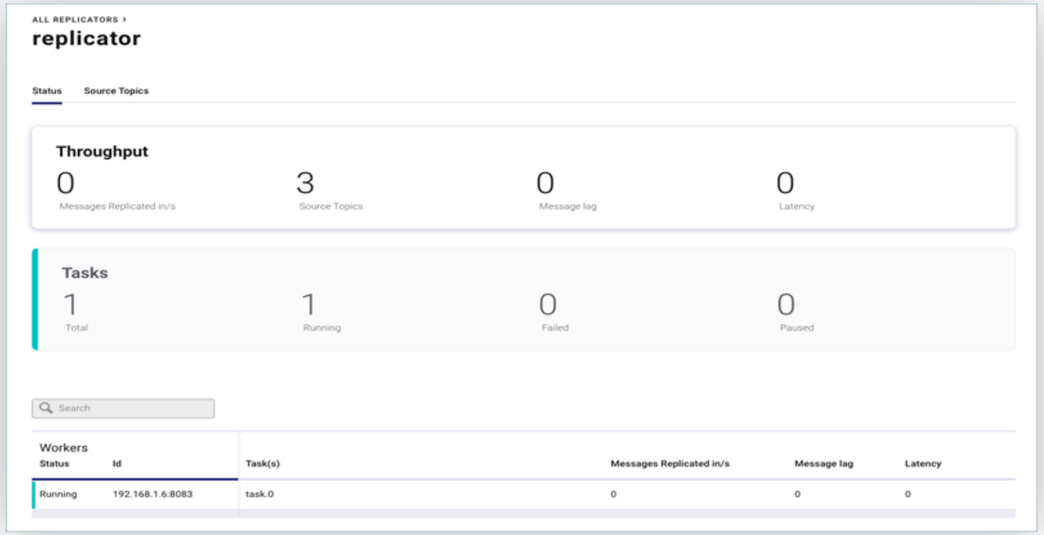
클러스터간 데이터 마이그레이션을 위한 Replicator

- 거리에 관계없이 클러스터 또는 Topic 하위 집합 복제
- 여러 클러스터를 함께 통합하거나 전체 클러스터를 선호하는 환경으로 마이그레이션
- Confluent Cloud로 하이브리드 클라우드 배포 활성화



멀티 사이트 배포된 Replicator를 쉽게 Monitoring

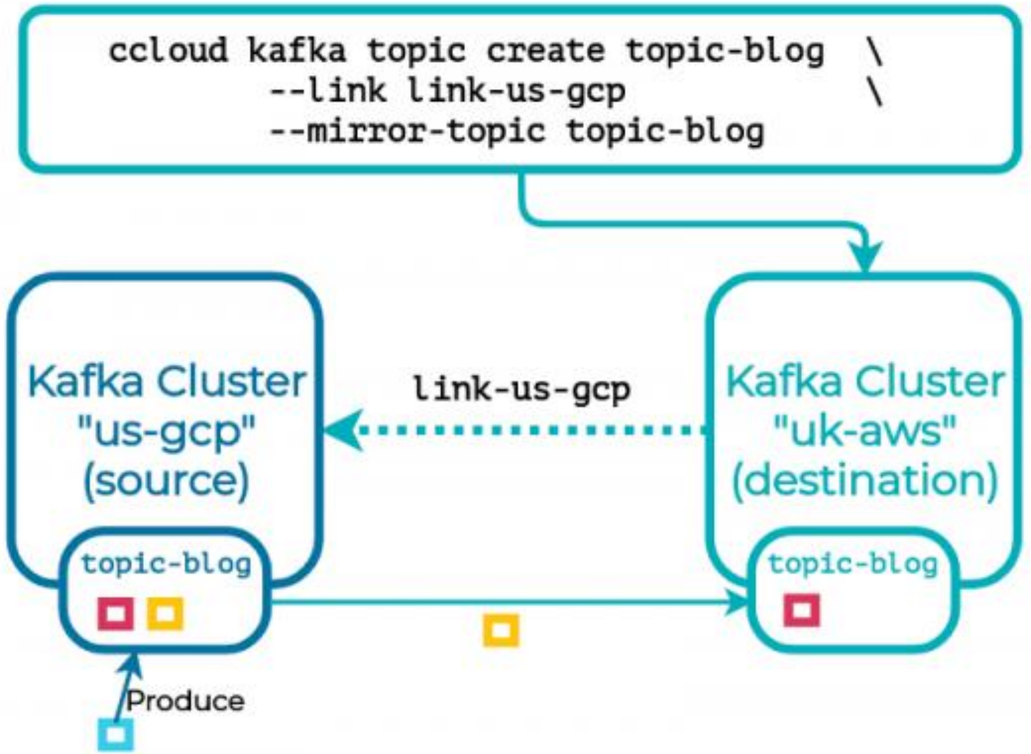
- Control Center에서 Replication 모니터링
- 재해 발생 전에 잠재적인 Replication 문제 식별



Cluster Linking

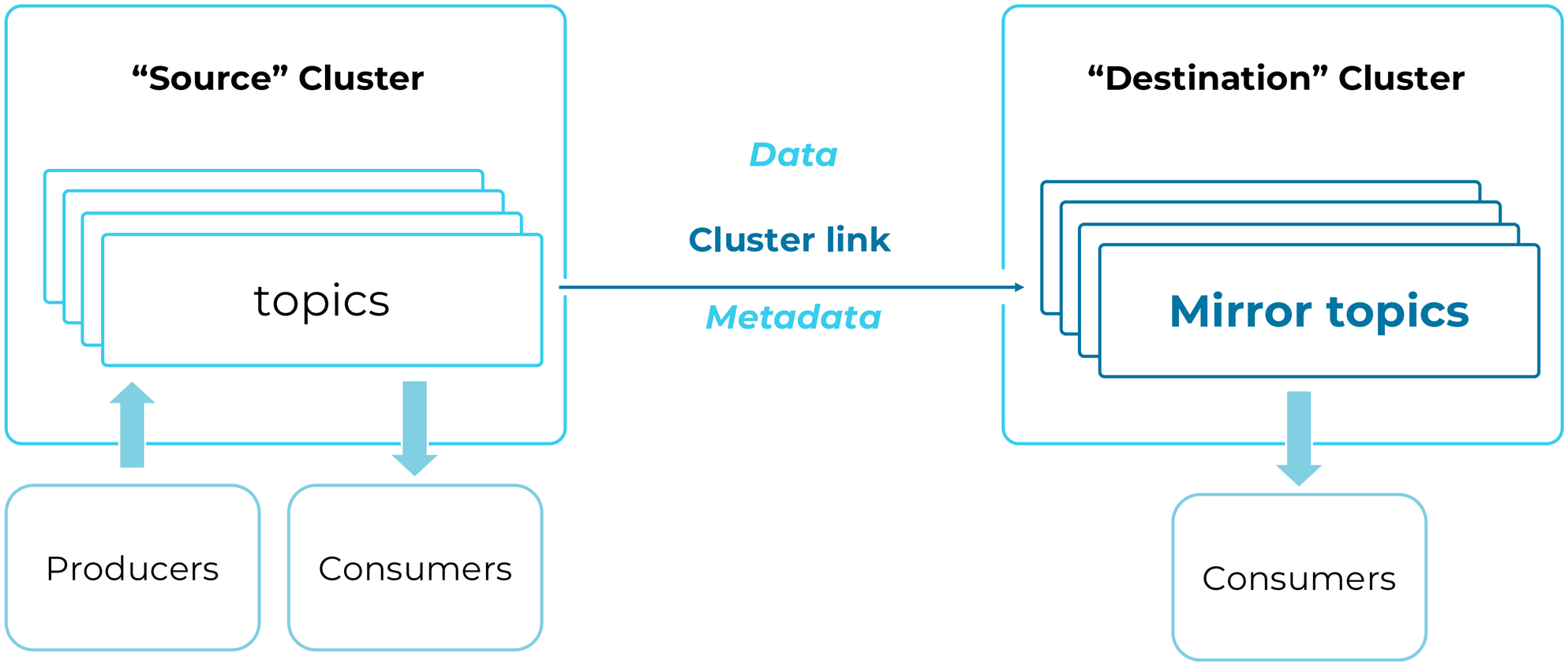
Cluster Linking을 사용하면 추가적인 connector 연결 없이 클러스터를 함께 직접 연결하고 한 클러스터에서 다른 클러스터로 Topic을 미러링할 수 있습니다.

Cluster Linking을 사용하면 다중 데이터 센터, 다중 클러스터 및 하이브리드 클라우드 배포를 훨씬 쉽게 구축할 수 있습니다.





Cluster Linking을 통해 Cluster간 연결

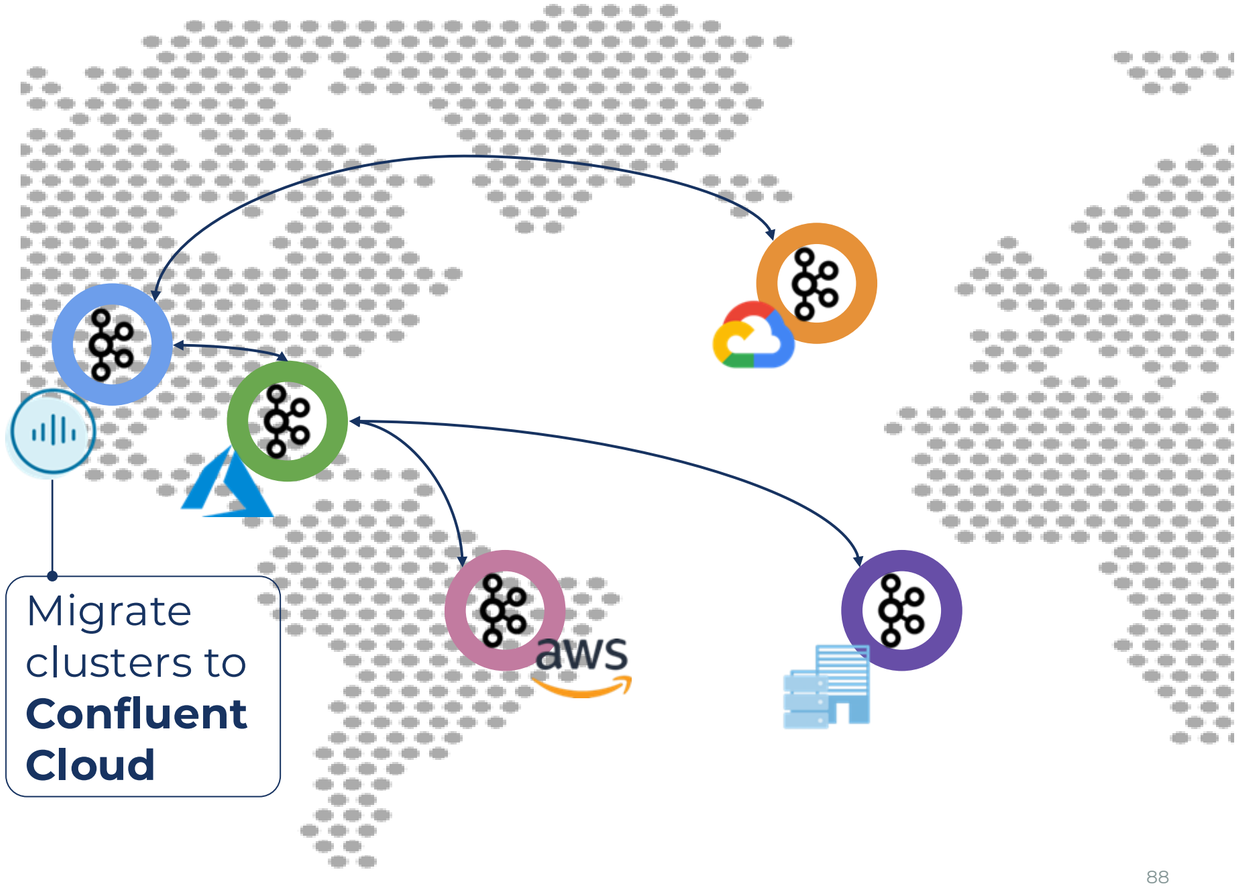


Cluster Level의 단방향 복제(양방향 복제 구성도 가능)

Cluster Linking

Cluster Linking은
Kafka의 **하이브리드**
클라우드 및 다중 클라우드
배포를 단순화합니다

Cluster Linking은 추가적인 인프라가 필요하지 않으며 offsets이 유지됩니다:



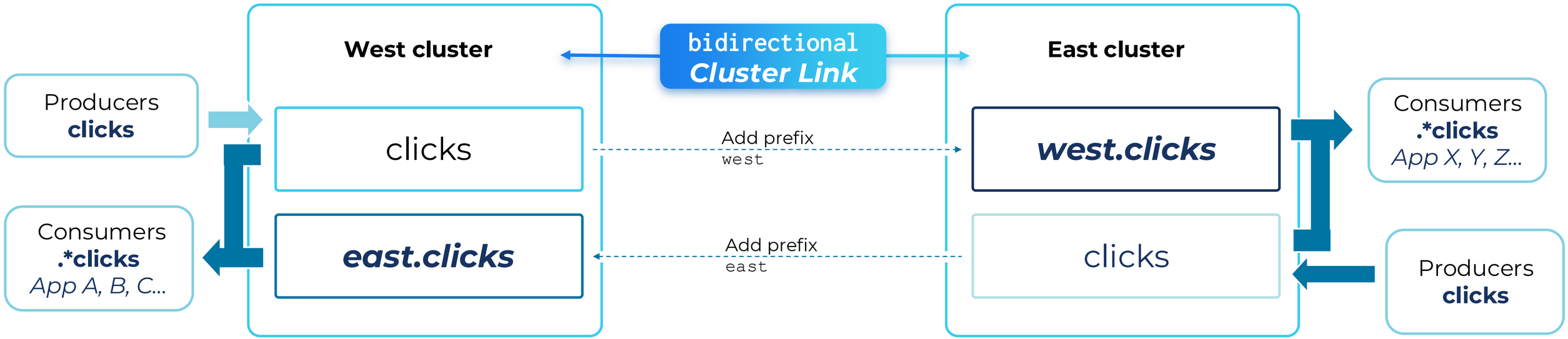
Confluent Cluster Linking Active-Active Pattern

조직에서 요구하는 RTO 및 RPO를 기준을 충족하기 위해 요구사항 분석 및 설계 반영이 반드시 필요하며, Confluent의 전문 컨설팅 서비스를 통해 지원 받으시길 권장 합니다.

HA/DR active-active pattern

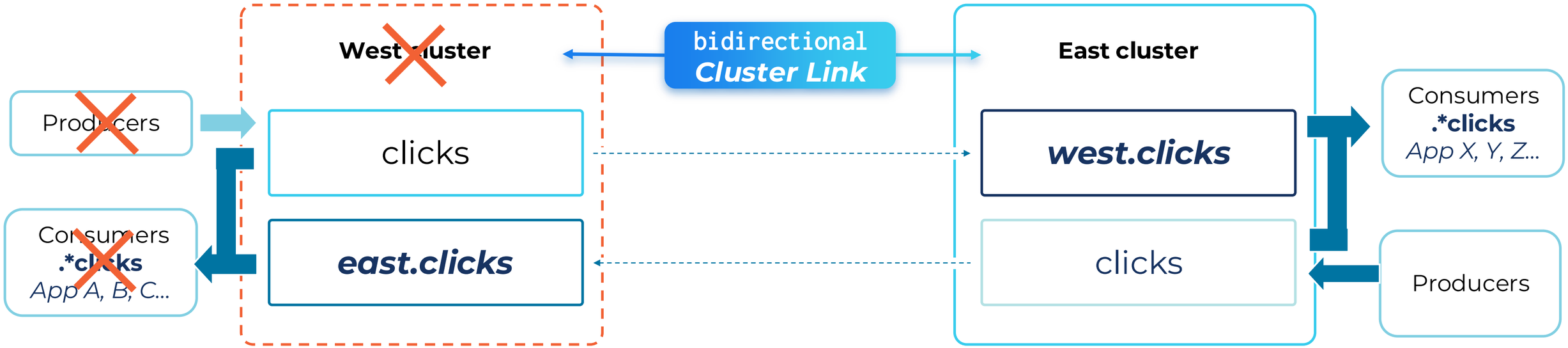
Steady state - “clicks” topic data

Consumer Offsets을 올바르게 처리하려면 Bidirectional Cluster Link가 필요합니다.
Confluent Cloud Dedicated 클러스터와 Confluent Platform 7.5 이상에서 사용할 수 있습니다.



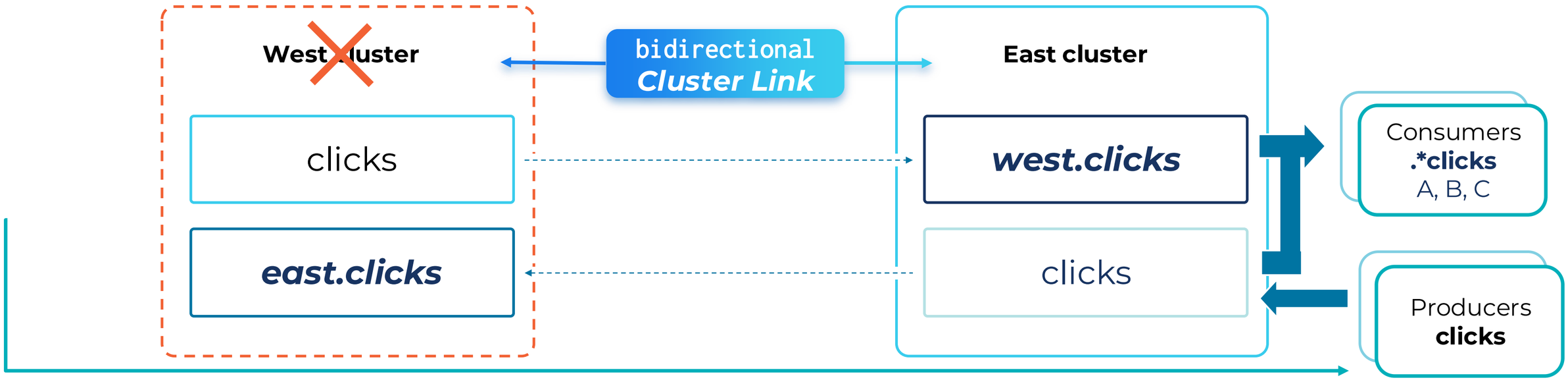
HA/DR active-active pattern

Disaster strikes!



HA/DR active-active pattern

Failover

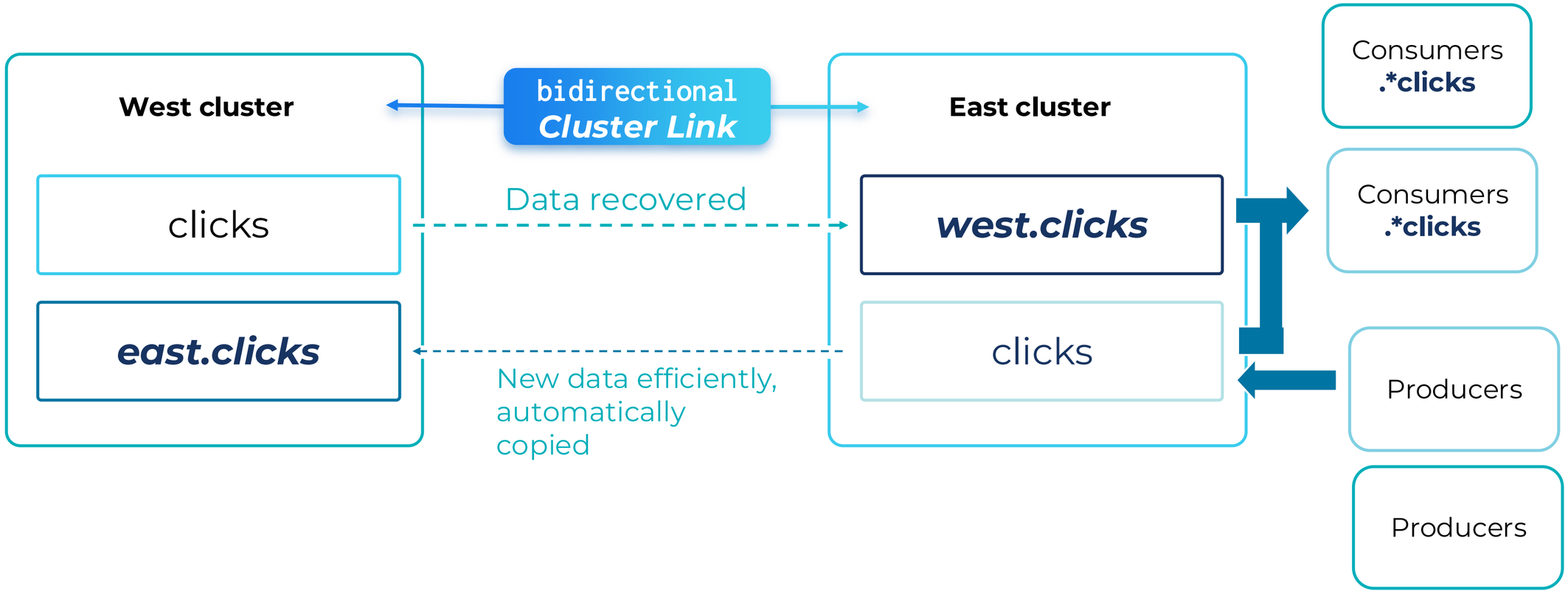


DR 지역의 producer/consumer 재기동

HA/DR active-active pattern

Recovery of pre-failover data is automatic

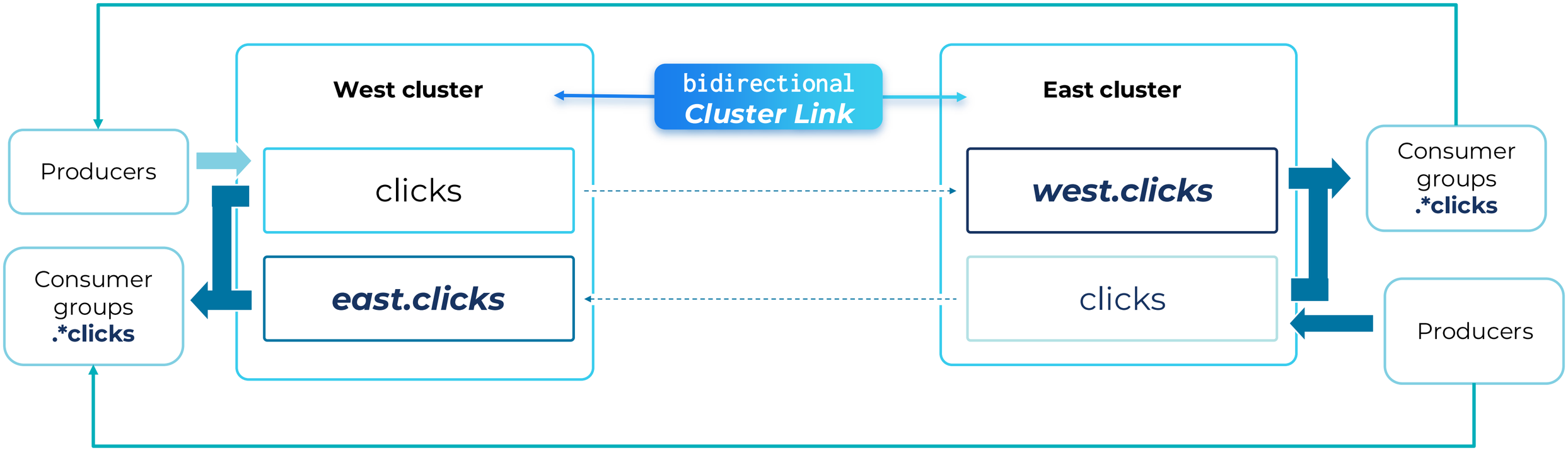
Failover 전 데이터 복구가 자동으로 수행되며, 또한 새로운 데이터가 효율적으로 자동 복제됩니다.



HA/DR active-active pattern

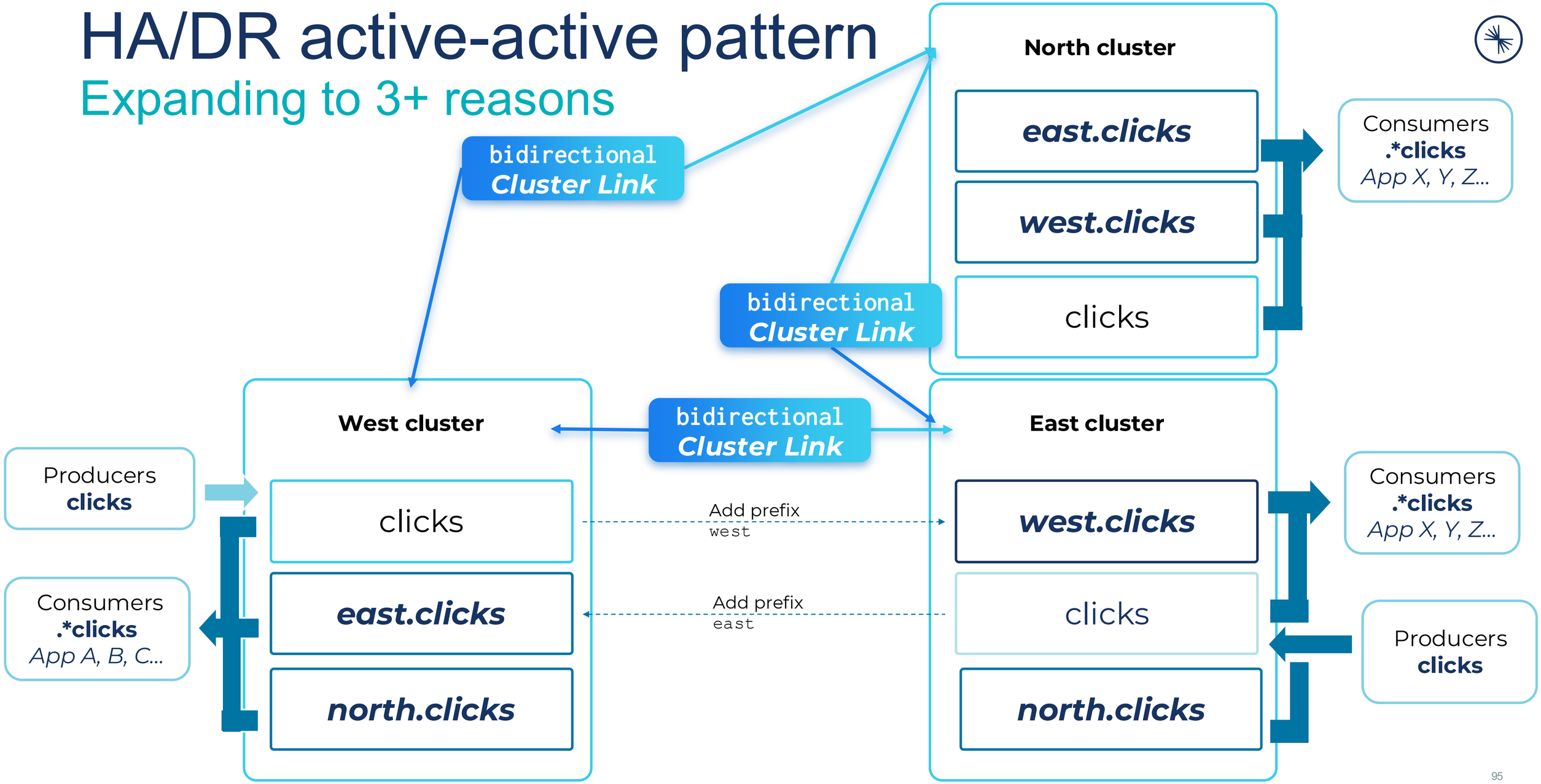
Failback is flexible and easy: *per producer, per consumer group*

Failover는 producer별, consumer group별 쉽고 유연하게 적용할 수 있습니다.
Client의 연결을 Failover 할 cluster로 연결하도록 재기동 합니다.



HA/DR active-active pattern

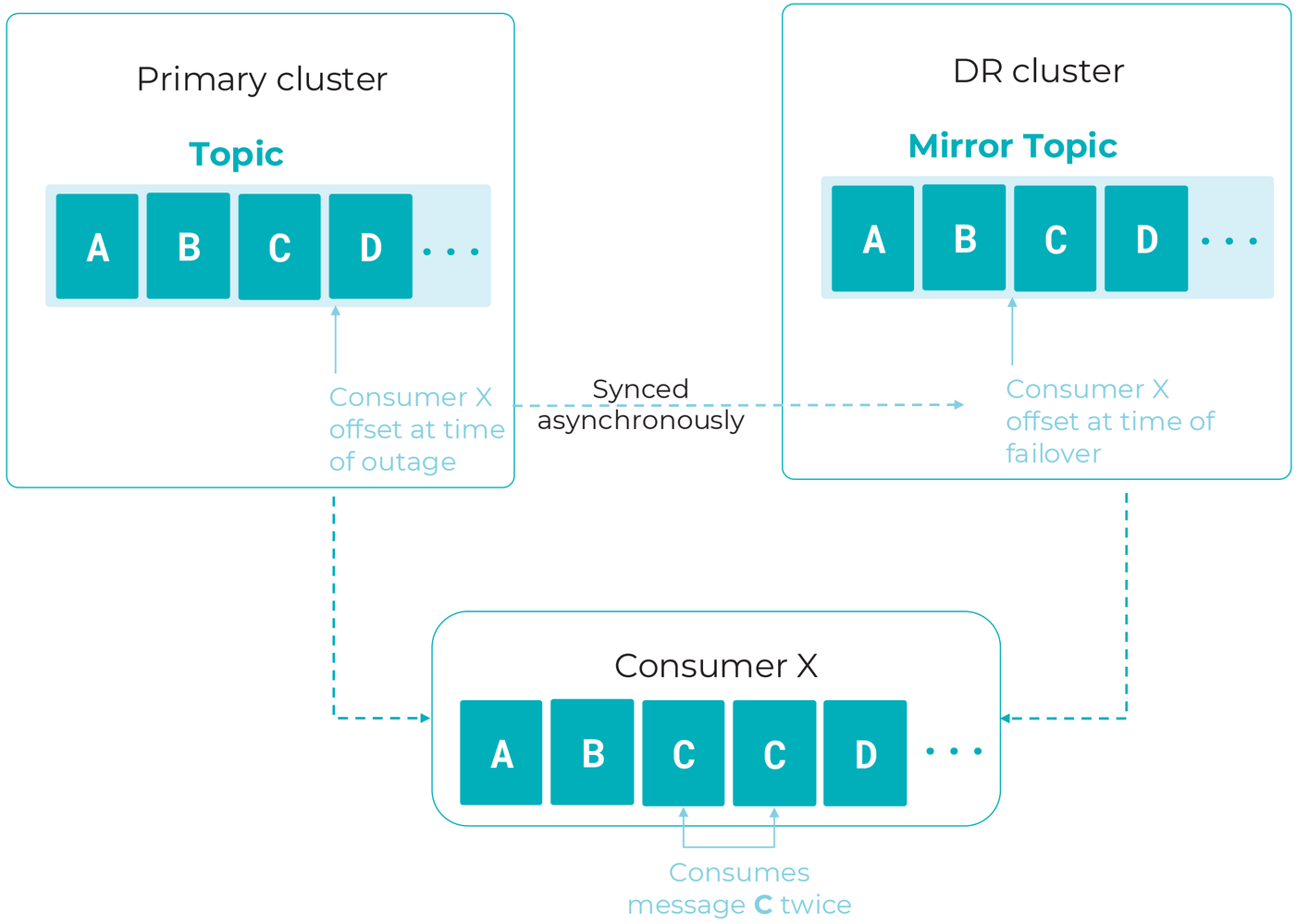
Expanding to 3+ reasons



HA/DR - Consumers must tolerate some duplicates

Consumer는 반드시 중복 메시지(**“idempotency”**)**를 허용해야 합니다.**

Cluster Linking는 비동기식 이기 때문에 Application Layer에서 중복 메시지 처리에 대한 허용 또는 처리 로직이 반드시 필요합니다.



Replicator vs. MRC vs. Cluster Linking

Confluent는 고객의 요구사항과 환경에 맞는 다양한 재해복구 및 멀티 사이트 운영 방안을 제공합니다. Multi Site 간 동기화를 지원하여 고객의 서비스 애플리케이션의 Active-Standby, Active-Active 서비스를 지원 데이터를 안전하게 보호하고 비즈니스 영속성을 지원합니다.

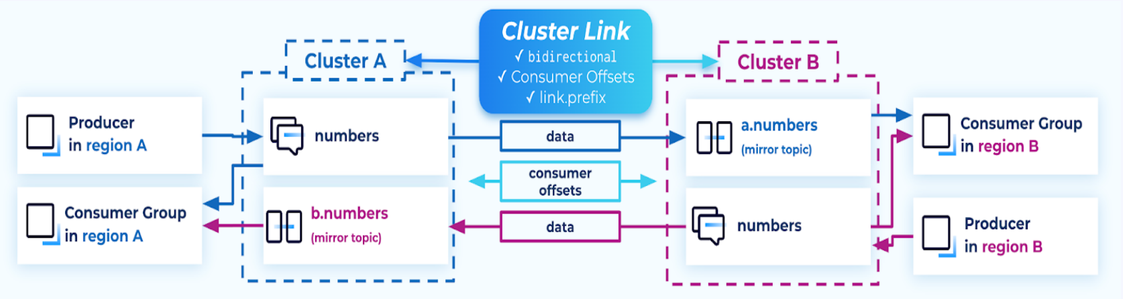
	 Replicator	 Multi-Region Clusters	 Cluster Linking
기능			
사용 사례	**Geo-replication, Data Migration & Aggregation (e.g., Hybrid & Multi cloud)**	**Disaster Recovery & High Availability, Active-Active Service**	**Geo-replication, Data Migration & Aggregation (e.g., Hybrid & Multi cloud)**
비고	• 장거리 간 복제 지원 (e.g., 대륙간) • 네트워크 요구사항 적음 • 여러 클러스터 집계 기능 • 데이터 복제 간 랙 발생	• RTO 가 0에 가깝고, 토픽 레코드의 RPO가 0 • 자동화된 클라이언트 fail over 제공 • fail back 기능 • Latency < 50 ms 네트워킹 요구	• 추가 인프라 없이 비용 효율적이고, 안전하며, 안정적인 클라이언트 연결 • 완벽하게 미러링된 복제를 통한 글로벌 데이터 공유 • 기본 오프셋 보존

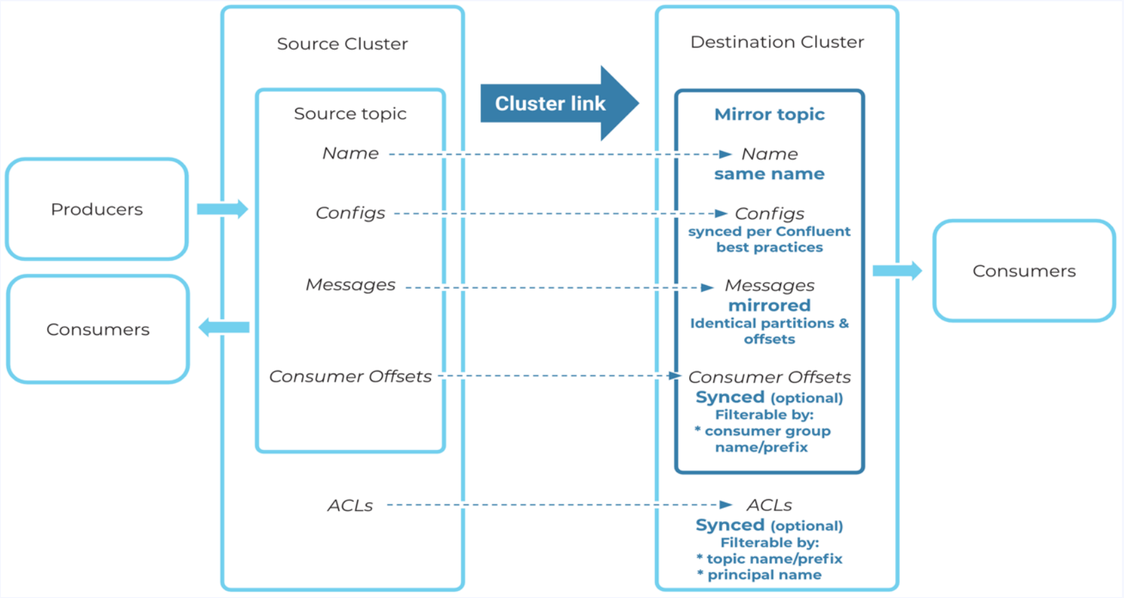
Cluster Linking 구성 방안 (국내 제조/항공사/금융사 도입 사례)

2DC Cluster Linking 모범 사례 구성안(2개 데이터 센터 기준)

- Cluster Liking을 이용하여 2개의 데이터 센터간 Cluster Level의 복제 구성 방안
- Cluster Linking을 사용하면 추가적인 connector 연결 없이 소스 클러스터에서 다른 클러스터로 Topic을 미러링 가능
- 추가 인프라 없이 비용 효율적이고, 안전하며, 안정적인 클라이언트 연결
- 완벽하게 미러링된 topic data, topic configs, consumer offsets, ACLs 복제를 통한 글로벌 데이터 공유
- 기본 오프셋 보존
- Mirror Topic은 Read-Only이기 때문에 Active-Standby DR 구성 가능
- Mirror Topic 생성시 Replication factor 설정이 가능하여, Standby Site의 Cluster에 구성을 고려하여 복제본을 조정할 수 있음
- Data Center간 Latency에 대한 제약 조건에 맞추어 유연한 구성 가능

Cluster Linking을 이용한 데이터 센터간 Cluster level 복제

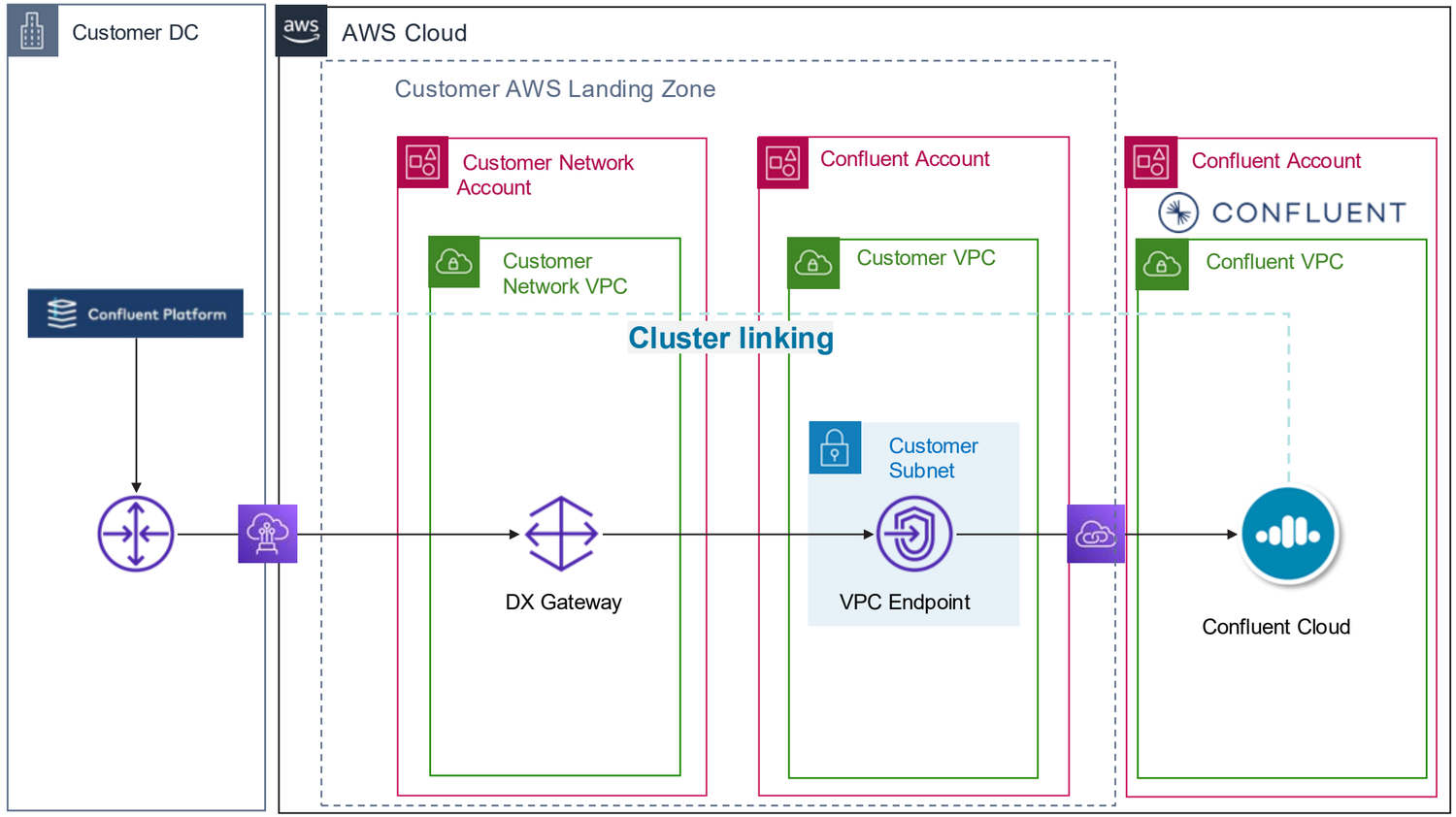




클라우드 연결: 하이브리드 구성

- Confluent Cluster Linking을 사용하면 온프레미스 또는 Confluent Cloud에 있는 클러스터 간에 클러스터를 직접 연결하고 토픽을 미러링할 수 있습니다. 클러스터 연결은 멀티 리전 또는 하이브리드 클라우드 사용 사례와 마이그레이션 시나리오 모두에 사용할 수 있습니다

- AWS 랜딩 존에서 제공하는 프라이빗 연결을 활용하면 공용 인터넷을 거치지 않고 AWS 백본에서 클러스터 연결을 수행할 수 있습니다



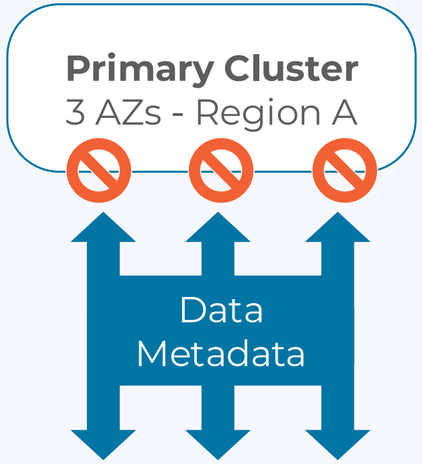
Cluster Linking을 이용한 쉽고 빠른 재해복구(DR) 환경 구성

Cluster Level의 단방향 복제(양방향 복제 구성도 가능)



Cluster Linking

서로 다른 환경의
클러스터를 완벽하게
미러링된 데이터 복제로
연결하여 데이터 손실 및
복구 시간을 최소화를 지원



Data Metadata

지속적이고 뛰어난 성능의
지리적 복제를 통해 **낮은 복구 시점 목표(RPO)**를 달성합니다.

Failover

Cluster Linking의 Failover API를 호출하고 클라이언트가 재해 복구(DR) 클러스터를 가리키도록 설정하면 손쉽게 페일오버를 수행하여 **낮은 복구 시간 목표(RTO)**를 달성할 수 있습니다.

DR Cluster
3 AZs - Region B

데이터는 항상 동일한 "Mirror Topic" 형태로 제공됩니다

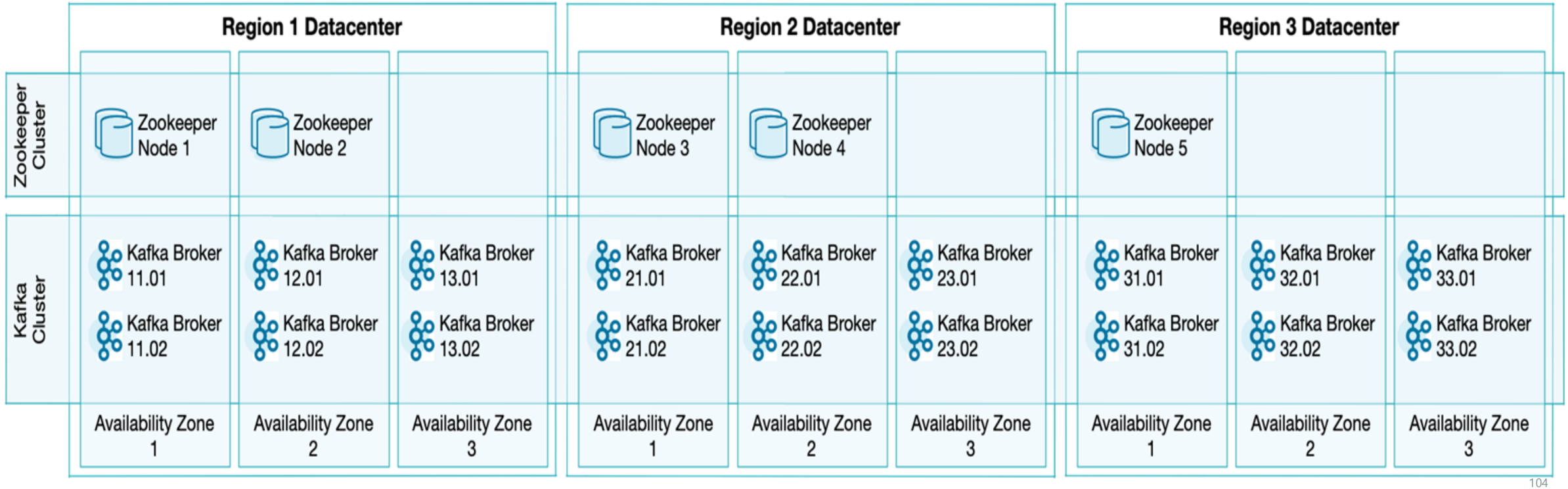
메타데이터는 동기화 상태로 유지됨:
* topics & configs
* consumer offsets
* application ACLs

고객의 다양한 재해복구 요구사항에 맞는 모범 사례 제공

Confluent는 재해복구에 특화된 기술을 제공하여 글로벌 수많은 고객의 다양한 재해복구 요구사항에 맞는 최적화된 모범 사례를 제공하고 지원합니다.
아래는 3개 데이터 센터 기준의 재해복구 구성 방안에 대한 모범 사례 입니다.

3DC 모범 사례 구성안(3개 데이터 센터 기준)

- 3DC 모범 사례 에는 3개 이상의 데이터 센터 간에 3개 이상의 데이터 복사본이 분산되어 있어 데이터를 안전하게 보호하고 Active-Active 서비스 운영이 가능
- HA/DR 전략에 대한 서비스형 토픽(Topics-as-a-Service) 접근 방식을 제공하여 서비스 Application의 부하 분산 및 서비스 가용성, 비즈니스 영속성을 제고
- Confluent Professional Service를 통해 고객의 요구사항 및 환경을 분석하여 최적의 아키텍처와 애플리케이션 설계 방안 제공



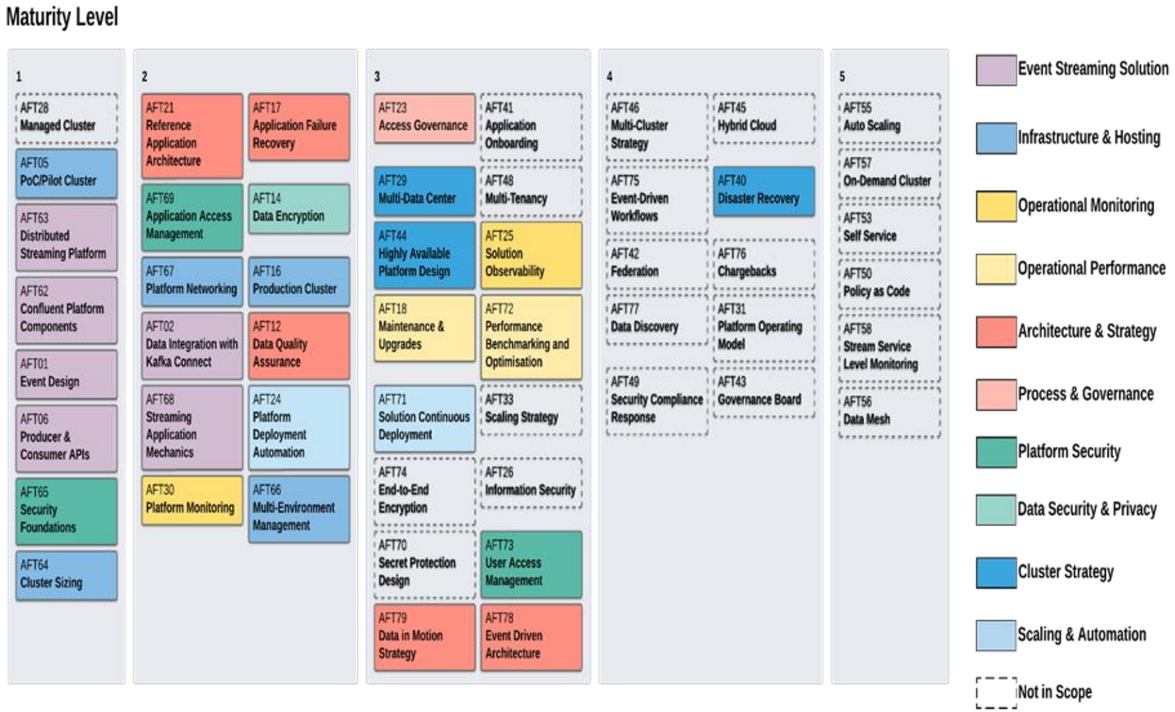
※용량 제약: Replication factor는 3으로 설정하여 Broker의 Disk 기준 1/3의 용량을 사용할 수 있습니다. Replication factor 구성에 따라 용량 제약이 변경 될 수 있습니다.

전문가 그룹의 수준 높은 기술 지원

Kafka 전문 컨설턴트의 다년간의 노하우를 기반으로 한 최고 수준의 서비스 제공

Confluent는 Apache Kafka 기반의 Confluent Platform 구축 프로젝트를 성공적으로 수행한 전문 Professional Service Consultant 인력들이 참여하여 고객사의 요구사항과 환경을 고려하여 최적의 아키텍처 컨설팅 서비스를 제공하고 있습니다. 플랫폼 아키텍처는 물론 애플리케이션 개발 가이드 등 Kafka 도입에 필요한 다양한 컨설팅 영역의 서비스를 지원 합니다.

성숙도 레벨 기반의 Confluent 도입 프레임워크 기반 컨설팅 지원



가이드: https://assets.confluent.io/m/26933af228cc1d15/original/20211129-Guide-Adopting_Confluent.pdf

미션 크리티컬 솔루션을 위한 아키텍처 설계 및 적용 가이드 제공

고 가용성 플랫폼 아키텍처 설계

멀티 데이터 센터 설계를 포함하여 최소 99.9%에서 최대 99.999%의 가용성을 제공하는 DR 설계를 제공합니다.

플랫폼 보안 강화

필수 보안 관리 기능을 통해 주요 보안 위협을 해결합니다

배포 자동화

완전 자동화된 배포 환경을 구축하여 반복 가능한 배포와 향후 확장 및 배포 요구 사항을 신속하게 처리합니다.

플랫폼 모니터링

필수 메트릭 모니터링을 통해 일상적인 플랫폼 운영을 지원

* 하나의 리전 전체에 대한 장애 내 결함성을 제공하기 위해서는 멀티 리전 설계가 필요할 수 있습니다.

Kafka 최고 전문가 집단의 24x7 기술지원

Confluent는 Apache Kafka의 80% 이상의 소스 커밋을 제공하고 있으며, Apache Kafka Project의 90% 이상의 커미터를 보유하고 있습니다. 이러한 전문가 집단이 기업고객 미션크리티컬 서비스의 안정적인 서비스를 유지할 수 있도록 24x7 기술지원 서비스를 제공합니다.

컨플루언트의 기술지원 조직과 엔지니어링 조직은 고객의 이슈를 최소화 할 수 있도록 유지적인 협업을 통해 신속하고 최적의 기술지원을 제공합니다.

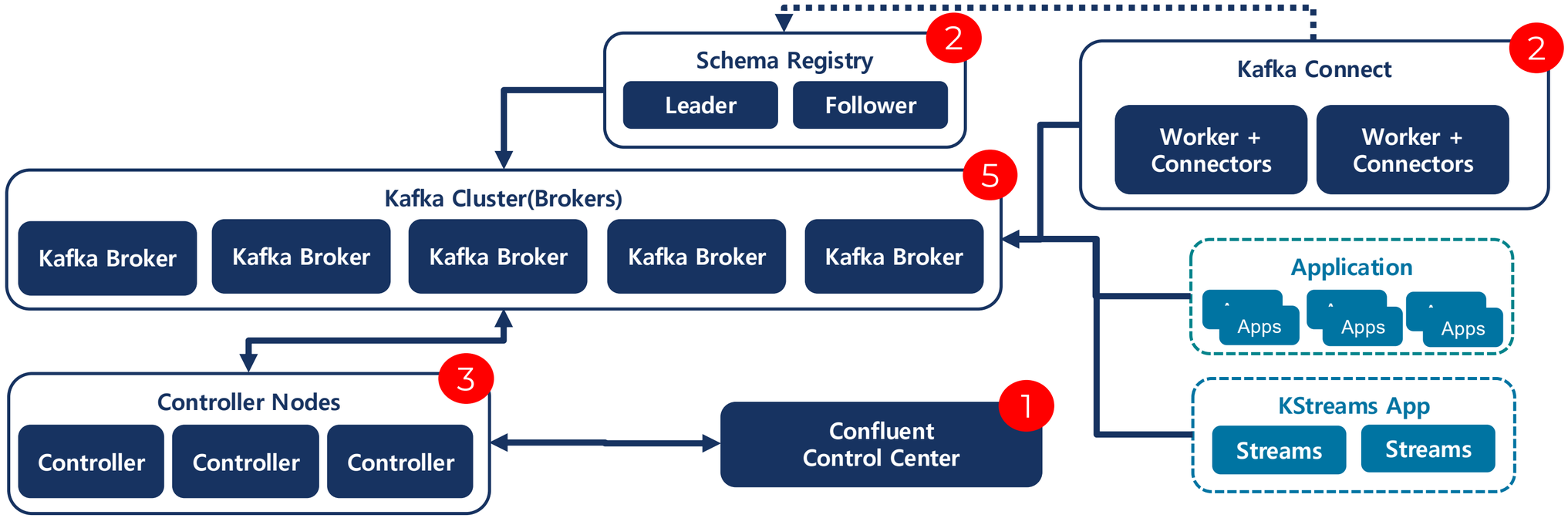
Confluent Platform Subscription 별 지원 정책

	Gold	Platinum
지원 범위	24/7	24/7
지원 횟수	무제한	무제한
최초 대응 시간	P1 - 60분 이내 P2 - 4시간 이내 P3 – 1 영업일 이내 P4 – 2 영업일 이내	P1 - 30분 이내 P2 - 2시간 이내 P4 – 2 영업일 이내
기능/지원 제한	없음	없음
지원 체널	웹 포털, 이메일, 비디오 컨퍼런스	웹 포털, 이메일, 비디오 컨퍼런스, 직통전화
P1용 직통 전화	아니요	예
긴급 패치	예	예
지식기반 기술 자료 액세스	예	예
Customer Success Technical Architect(CSTA)	아니요	예
Extended Support Package*	애드온 추가 가능	예, 포함됨

** 확장된 지원/유지 관리 패키지는 소프트웨어 버전을 업그레이드하는 데 추가 시간이 필요할 수 있는 고객을 위해 소프트웨어의 주요 버전에 대한 수명 종료 날짜를 1년 더 연장 지원합니다.*

Confluent Platform Reference Architecture

Confluent Platform Reference Architecture



Confluent Component	용도	제안 수량
Controller	Kafka Metadata 관리 서비스(카프카 브로커는 정족수 투표를 위해 Controller[이전 Zookeeper]에 의존 합니다.)	**3 노드**
Kafka Broker	Kafka의 핵심 서비스 노드 (Topic 저장 및 Message 송/수신 서비스)	**5 노드**
Confluent Control Center	Web 기반의 관리 툴(Confluent Platform을 위한 중앙 집중식 관리, 모니터링 및 경고 도구)	**1 노드**
Schema Registry	Schema 관리 서비스(토픽 스키마를 위한 리포지터리)	**2 노드**
Kafka Connect (Connect Worker)	Kafka Connector 운영 노드(데이터 소스와 싱크를 쉽게 통합할 수 있습니다. 코드를 작성하는 대신 구성 파일을 사용하여 시스템을 통합할 수 있습니다. 또한 200개가 넘는 커넥터로 구성된 라이브러리도 늘어나고 있습니다. 가치 창출 시간을 단축합니다.)	**2 노드**



Confluent Platform Reference Architecture: https://www.confluent.io/whitepaper/confluent-enterprise-reference-architecture/

Confluent Platform 권장 및 최소 사양

Confluent Component	운영 환경 권장 사양				최소 구성 사양		
	CPU(Core)	Memory	Disk	수량(고가용성 제공)	CPU(Core)	Memory	Disk
KRaft Controller	6	16 GB	512 GB (SSD 권장)	3 node	3	4 GB	100 GB (SSD)
Kafka Broker	24	64 GB	12 TB (24 X 1 TB HDD disk. RAID 10 권장)	4 node (온라인 업그레이드/패치, 장애를 고려하여 4 node를 권장)	4	16 GB	1 TB (HDD, RAID 10 권장)
Confluent Control Center (Web UI Admin/User Portal)	12	32 GB (JVM default 6 GB)	300 GB (SSD 권장)	1 node (관리 컴포넌트로 고가용성 제외)	4	16 GB	250 GB (HDD, RAID 10 권장)
Schema Registry	4	16 GB	100 GB (HDD)	2 node	2	4 GB	50 GB (HDD)
REST Proxy	16	32 GB(최소 2GB + Producer당 64MB + Consumer당 16MB 권장)	100 GB (HDD)	2 node	16	32 GB(최소 2GB + Producer당 64MB + Consumer당 16MB 권장)	50 GB (HDD)
ksqlDB	8	64GB	300 GB (SSD 권장)	2 node	4	20 GB	100 GB (SSD)
Kafka Connect (Connect Worker)	12	64 GB (커넥터에 따라 0.5 - 4 GB heap size 조정 필요)	100 GB (HDD)	2 node	4	16 GB	50 GB (SSD)
사이징 기준 예상 처리량(Throughput)	운영 환경 권장 사양을 적용시 예상 최대 처리량: * Broker 4 node 기준: - 예상 최대 처리량 0.5GBps - 예상 최대 가용 스토리지: 12.8TB(3벌 복제 기준)						
비고	*** 최적화된 사이징을 위해서는 Confluent Professional Service를 이용하여 요구사항, 환경 분석 및 사이징/아키텍처 디자인 등의 컨설팅 서비스를 적극 권장합니다.** * DISK의 용량은 Usable size를 의미 합니다. ** Disk type SSD로 표시된 경우 520MB/s의 Throughput을 권장합니다. ** Kafka Broker의 Single volume Throughput은 125MB/s로 RAID 10 구성으로 600MB/s의 Throughput을 권장합니다. ** 목표 처리량 및 인프라 환경에 따라 구성에 보다 낮은 사양의 DISK로 구성 및 운영도 가능합니다. *** NIC는 10gE 기준 *** VM 구성 시 각 node는 Host별 1대씩 배치를 권장합니다. 특히 Broker Node의 경우 성능 저하가 발생될 수 있기 때문에 동일 호스트 배치되지 않는 구성이 반드시 필요합니다. **** REST Proxy의 메모리는 Producer당 64MB, Consumer당 16MB 추가로 산정하여 사이즈를 적용하는 것을 권장합니다.						

Confluent Platform Node Definition

Confluent Platform 아키텍처 디자인 및 배포의 맥락에서 **노드의 정의**를 이해하는 것이 중요합니다.

노드는 다음 요소 중 어느 것도 초과하지 않는 물리적 또는 가상 컴퓨팅 머신에서 실행되는 Confluent Platform의 구성 요소의 소프트웨어 인스턴스입니다. :

- 56 코어 또는 가상 코어
- 24 하드 드라이브
- 50 TB 총 하드 드라이브 또는 flash/SSD 용량
- 256 GB RAM 메모리

모든 기능이 노드로 계산되는 것은 아닙니다.
(e.g., RBAC, Kafka Streams, Auto Data Balancer, ...)

다음 Confluent Platform 구성 요소는 노드로 간주합니다:

- Schema Registry
- REST Proxy
- ksqlDB
- Control Center
- Replicator
- Operator
- MQTT Proxy
- Kafka Broker*
- Kafka Connect*
- ZooKeeper*
- KRaft Controller*

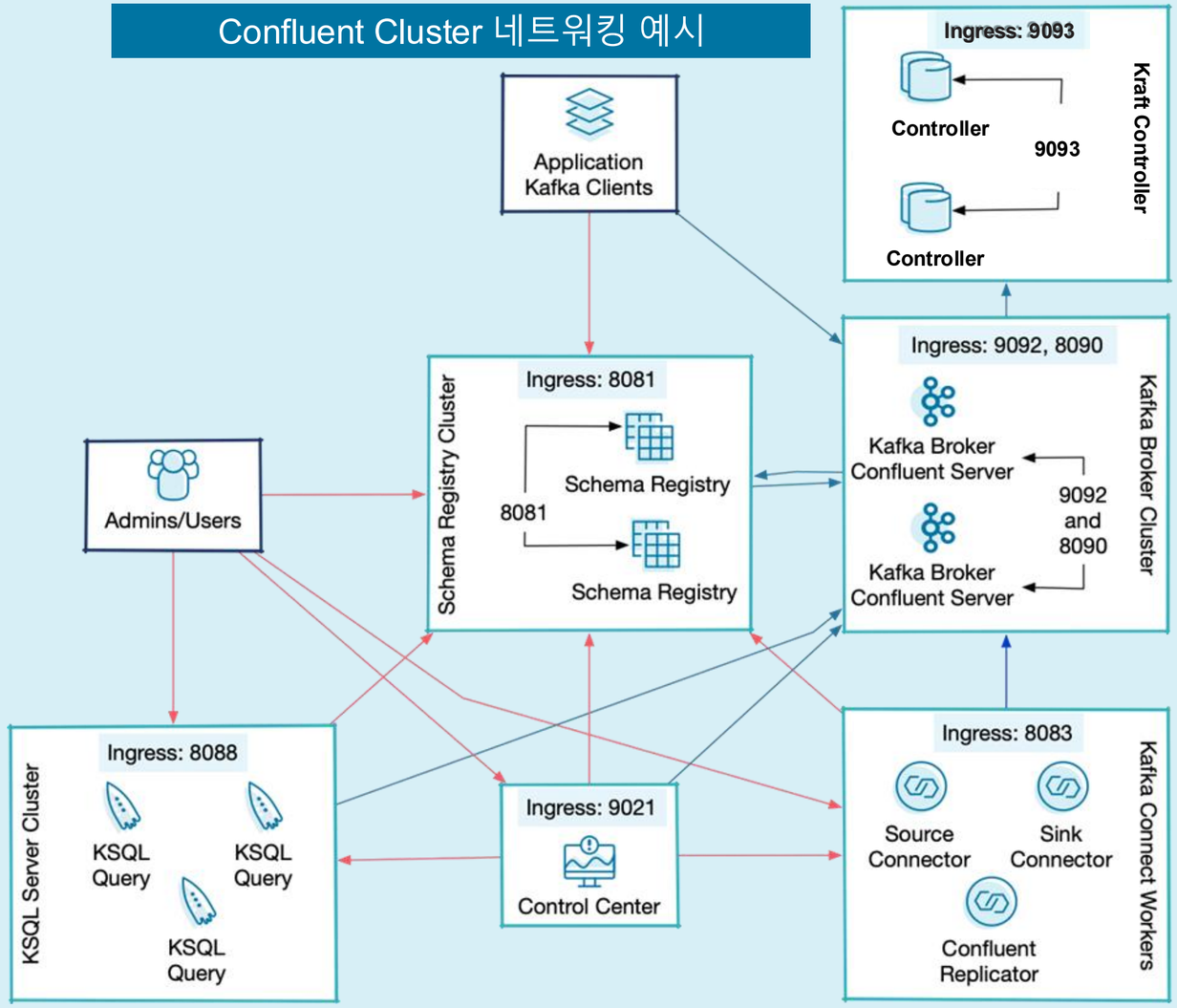
* Open Source Apache Kafka component

Confluent Cluster Network Architecture

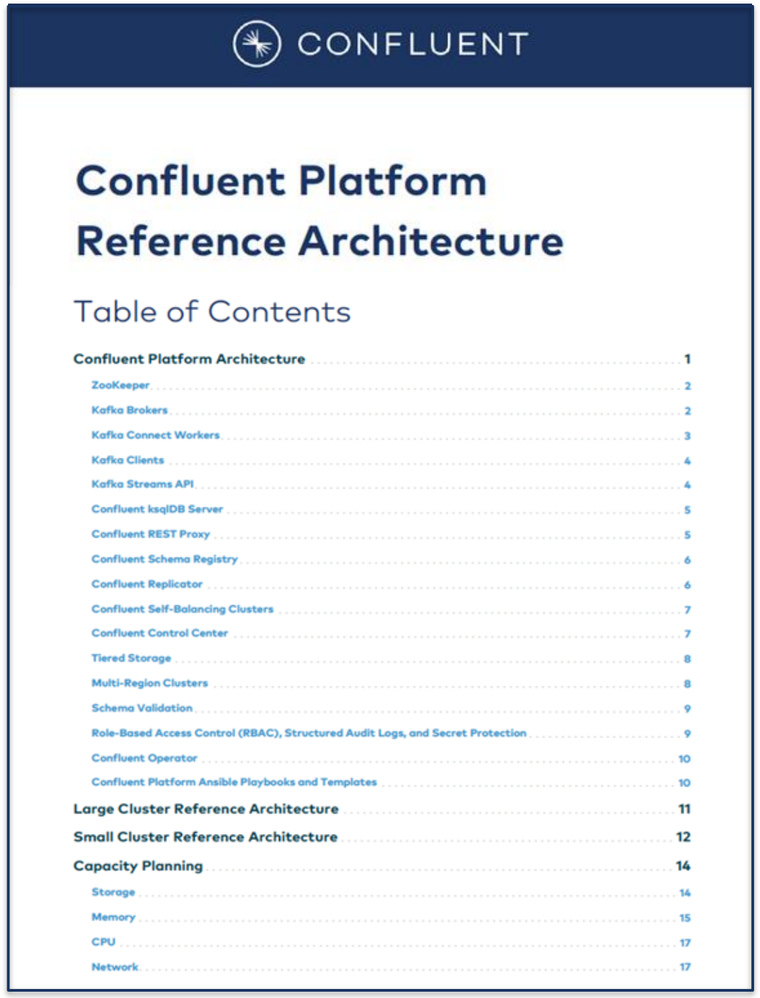
Network Encryption
스택의 각 구성 요소는 클러스터의 내부 구성 요소와 외부 구성 요소 간의 암호화를 위해 TLS SSL을 사용합니다.

Port
기본적으로 Confluent Platform 구성 요소는 아래의 포트를 사용하여 통신하며 방화벽에서 해당 포트가 열려 있어야 합니다.

Component	Port
Kafka brokers (plain text)	9092
Kafka inter-broker communication	9091
Confluent Control Center	9021
Kafka Connect REST API	8083
ksqlDB Server REST API	8088
Metadata Service (MDS) if enabled	8090
REST Proxy	8082
Schema Registry REST API	8081
Kraft Controller	9093



Confluent Reference Architecture White Papers



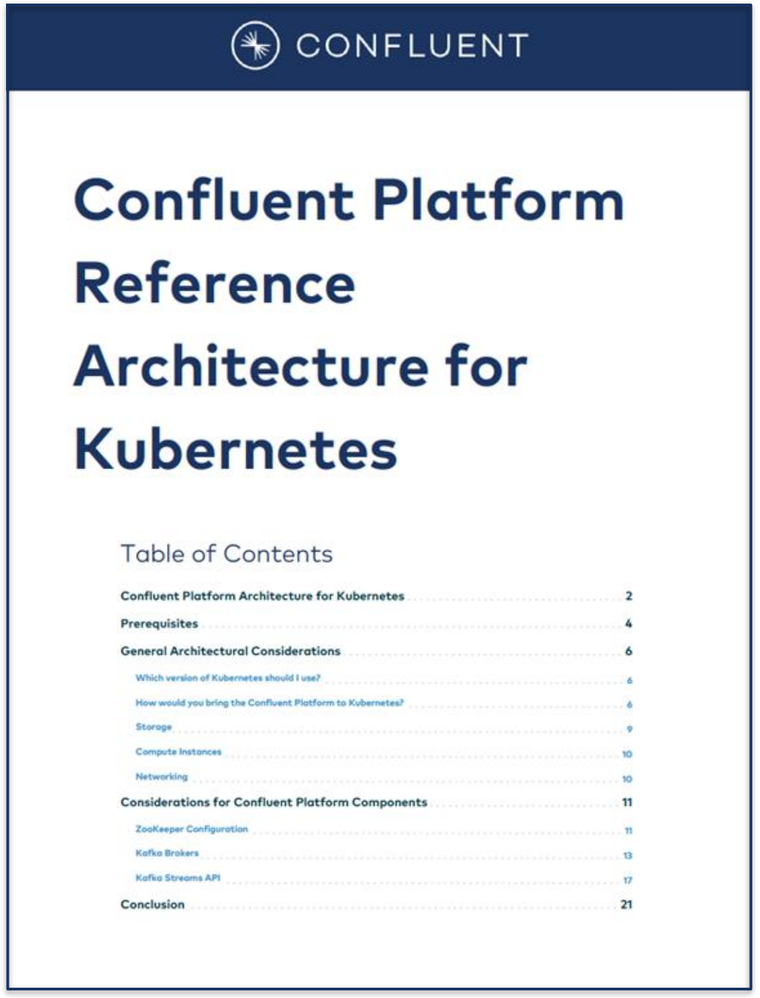
CONFLUENT

Confluent Platform Reference Architecture

Table of Contents

Confluent Platform Architecture 1
ZooKeeper 2
Kafka Brokers 2
Kafka Connect Workers 3
Kafka Clients 4
Kafka Streams API 4
Confluent ksqlDB Server 5
Confluent REST Proxy 5
Confluent Schema Registry 6
Confluent Replicator 6
Confluent Self-Balancing Clusters 7
Confluent Control Center 7
Tiered Storage 8
Multi-Region Clusters 8
Schema Validation 9
Role-Based Access Control (RBAC), Structured Audit Logs, and Secret Protection 9
Confluent Operator 10
Confluent Platform Ansible Playbooks and Templates 10
Large Cluster Reference Architecture 11
Small Cluster Reference Architecture 12
Capacity Planning 14
Storage 14
Memory 15
CPU 17
Network 17

https://www.confluent.io/whitepaper/confluent-enterprise-reference-architecture/



CONFLUENT

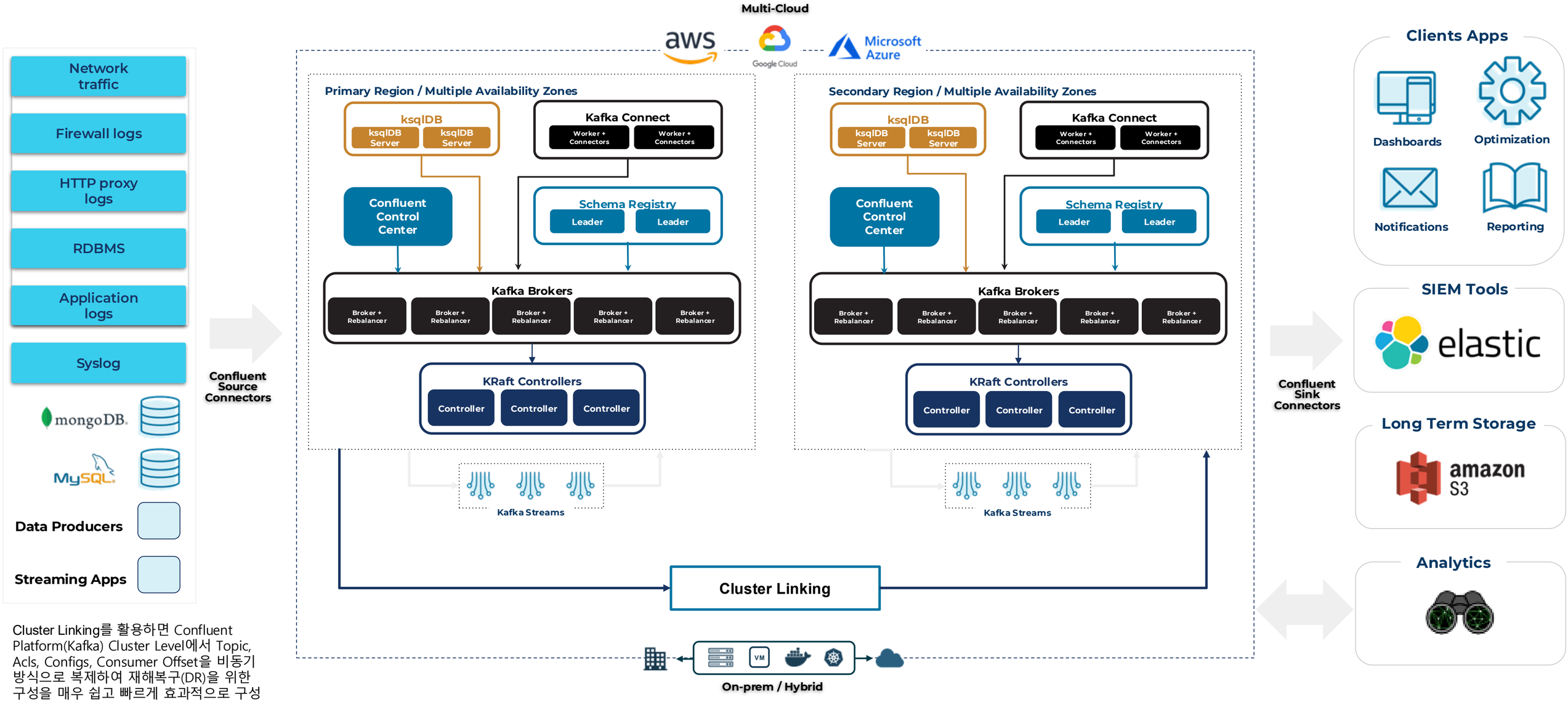
Confluent Platform Reference Architecture for Kubernetes

Table of Contents

Confluent Platform Architecture for Kubernetes 2
Prerequisites 4
General Architectural Considerations 6
Which version of Kubernetes should I use? 6
How would you bring the Confluent Platform to Kubernetes? 6
Storage 9
Compute Instances 10
Networking 10
Considerations for Confluent Platform Components 11
ZooKeeper Configuration 11
Kafka Brokers 13
Kafka Streams API 17
Conclusion 21

https://www.confluent.io/resources/confluent-platform-reference-architecture-kubernetes/

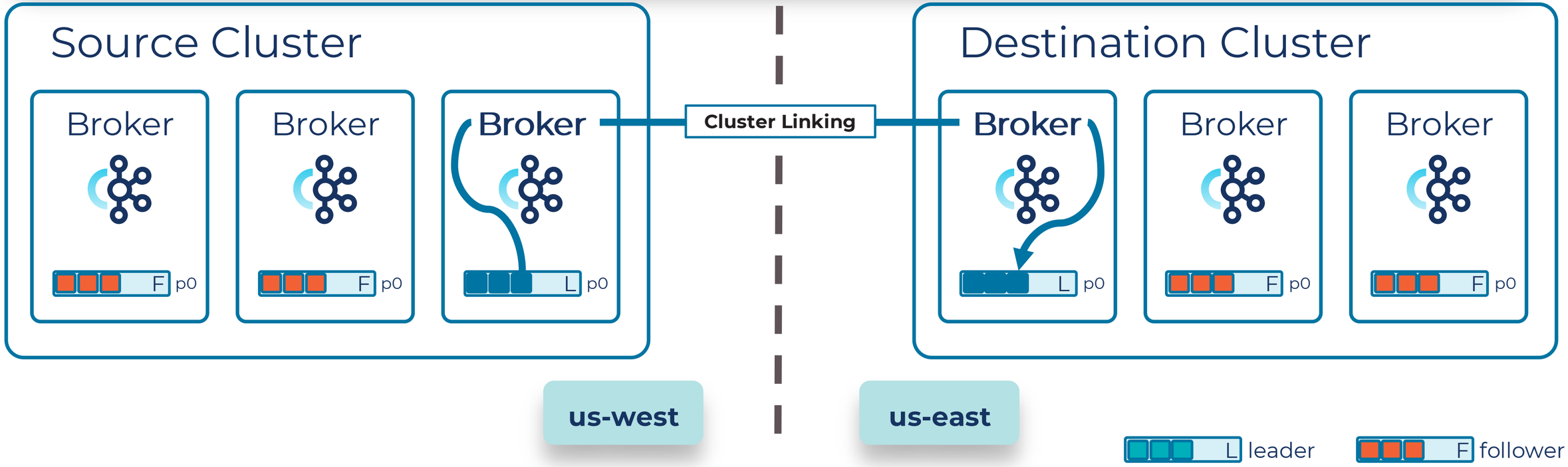
Highly-Available / Fault Tolerant with Disaster Recovery 예시



Cluster Linking를 활용하면 Confluent Platform(Kafka) Cluster Level에서 Topic, Acls, Configs, Consumer Offset을 비동기 방식으로 복제하여 재해복구(DR)을 위한 구성을 매우 쉽고 빠르게 효과적으로 구성할 수 있습니다.

Confluent Cluster Linking

- 별도의 복제 컴포넌트 불필요: Cluster Linking은 Confluent Platform에서 제공하는 Cluster Level의 복제 기술로 Replicator 또는 Mirror Maker2 같은 별도의 복제 노드가 필요 없습니다.
- 오프셋 보존: Cluster Liking은 Consumer Offset을 Source Cluster에 Destination Cluster 동기화하여 DR 선언 시에 기존의 Application이 마지막으로 처리한 Offset 이후로 바로 서비스를 처리할 수 있습니다.
- 매우 효율적: Cluster Linking은 Topic, ACLs, Configs, Consumer Offset을 모두 비동기로 복제하기 때문에 매우 효율적으로 재해복구(DR)을 구성하고 활용할 수 있습니다.



3. 레퍼런스

Confluent는 산업 전반에 걸쳐 입증된 성공 사례를 제공합니다

FINANCIAL SERVICES



DIGITAL NATIVE & TECH



CONSUMER/RETAIL



HEALTHCARE & INSURANCE



COMMUNICATIONS & MEDIA



AUTOMOTIVE/TRANSPORTATION



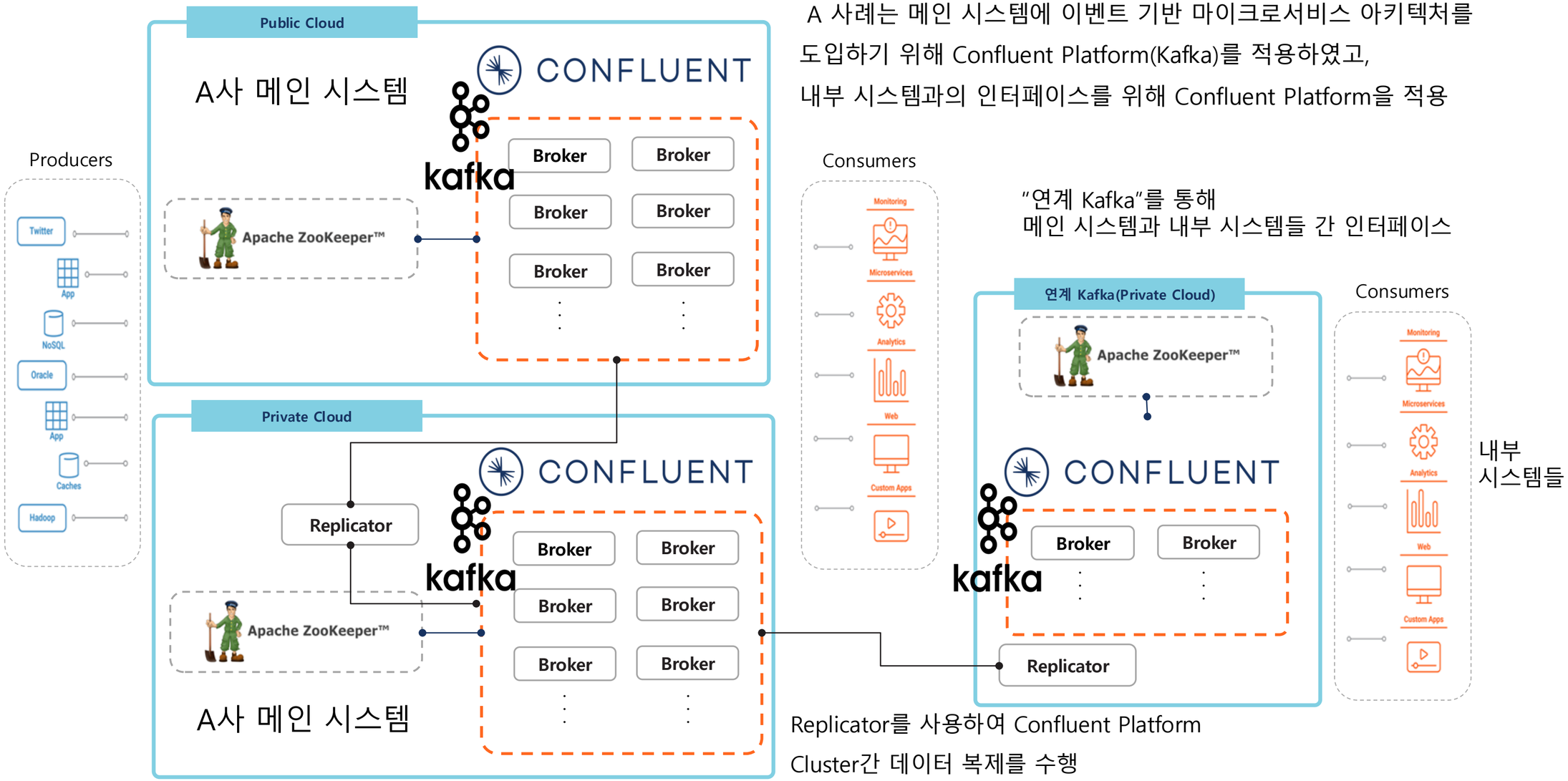
MANUFACTURING



TRAVEL



L 통신사 사례



A 사례는 메인 시스템에 이벤트 기반 마이크로서비스 아키텍처를 도입하기 위해 Confluent Platform(Kafka)를 적용하였고,
내부 시스템과의 인터페이스를 위해 Confluent Platform을 적용

"연계 Kafka"를 통해
메인 시스템과 내부 시스템들 간 인터페이스

Replicator를 사용하여 Confluent Platform Cluster간 데이터 복제를 수행

※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

L 통신사 (OSS/BSS) 차세대 프로젝트 – Kafka 구성 요구사항

L 통신사는 OSS/BSS 차세대 프로젝트의 표준 인터페이스를 Kafka로 선정하여 전사의 데이터 및 메시징을 실시간으로 처리하고 필요한 정보를 실시간 데이터 프로세싱을 통해 효과적으로 사용할 수 있도록 데이터 중심적으로 프로세스와 아키텍처를 개선하였습니다.

OSS/BSS 차세대 프로젝트 – Kafka 구성 요구사항

- 차세대 OSS/BSS, 현행화, 교육계, 과금 클러스터로 구성
- Private Cloud와 Public Cloud(AWS)가 조합된 Hybrid Cloud 형태로 구성하고, 현행화 클러스터는 Private Cloud에만 구성
- 차세대 OSS/BSS 프로젝트 Kafka는 운영계, 통합 시험계(이하 통시계), 개발계, 교육계, 현행화 클러스터는 운영계, 통시계, 개발계로 시스템을 나누어 구성
- 클러스터 간 복제가 필요한 토픽은 replicator를 이용해 양방향 복제가 이루어져야 함
- Active(main) 클러스터 전체 노드에 중대한 장애를 대비하여 항상 백업 Standby(admin) 클러스터에 전체 토픽을 단방향 복제하며, 장애 발생 시 신속한 failover를 수행할 수 있도록 구성
- 총 16개의 Confluent Platform(Kafka) Cluster 구성

OSS/BSS 차세대 프로젝트 – Kafka 운영 요구사항

- 운영시스템의 다운타임 최소화 - Rolling upgrade 등, 긴급 장애지원 등(Confluent Partner Carepack을 통한 On-site 지원 포함)
- 운영 프로세스 최적화 지원 – Confluent Professional Support Service를 통해 Health Check 및 Design Workshop 등 수행
- 성능 최적화 및 아키텍처 디자인 지원 – MRC(Multi-Region Cluster) 디자인 업데이트 성능 튜닝 방안 제공 등

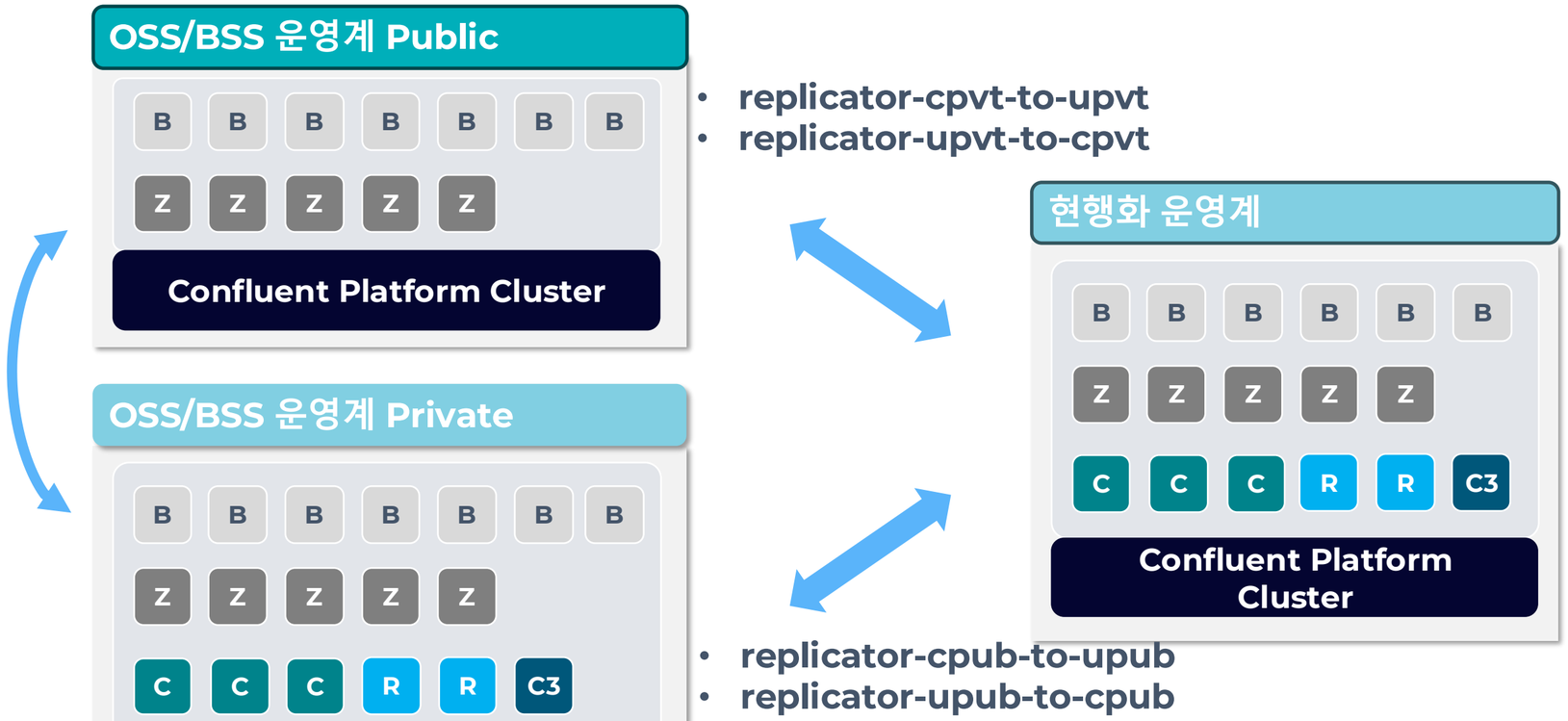
※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

서비스 환경 별 데이터 복제
Replicator를 이용한 안전한 실시간 데이터 복제 프로세스 적용

Confluent Replicator를 이용하여 Cluster간 Topic 데이터를 실시간 비동기 양방향 복제를 통해 데이터 공유 및 이전 자동화 구현

Replicator 사용 구조

- ex. 운영계



- **replicator-pub-to-pvt**
- **replicator-pvt-to-pub**
- **replicator-cpvt-to-upvt**
- **replicator-upvt-to-cpvt**
- **replicator-cpub-to-upub**
- **replicator-upub-to-cpub**

※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

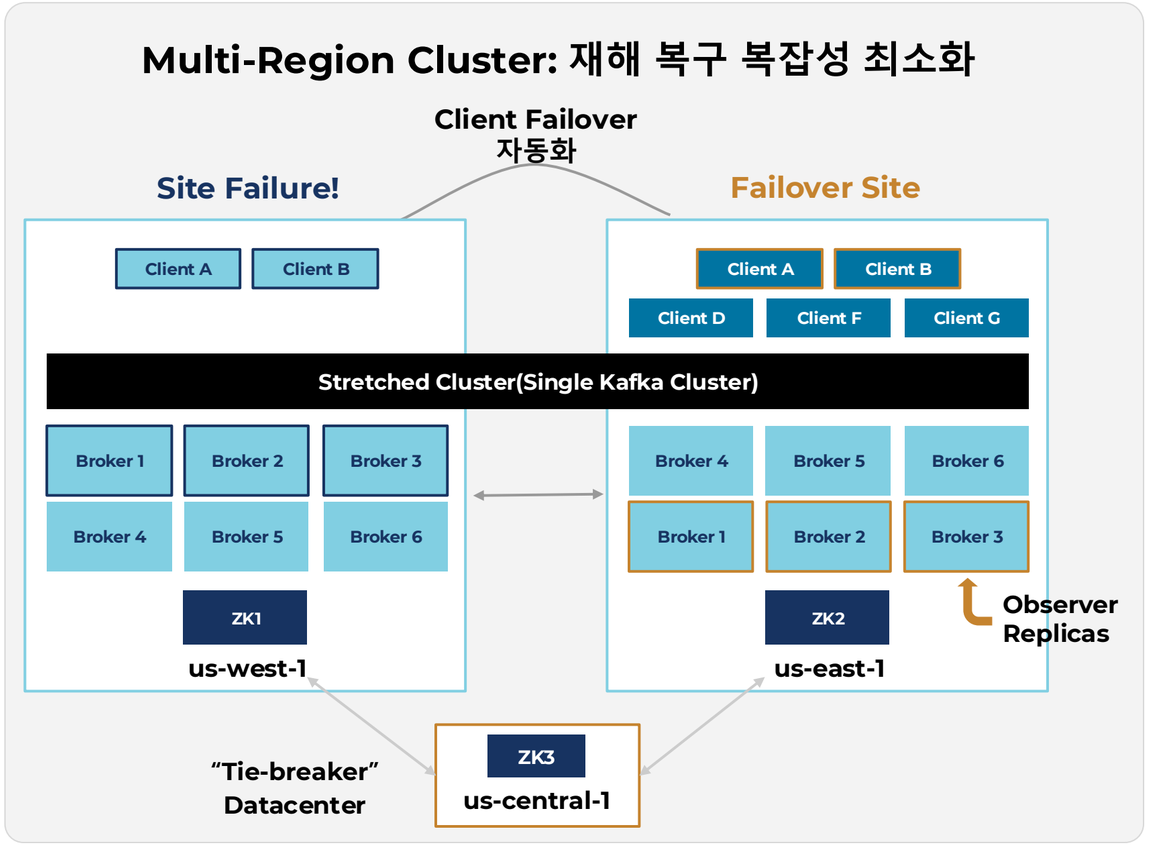
서비스 고가용성을 위한 구성 제공
Multi-Region Cluster를 이용한 고가용성 환경 구축

Confluent Multi-Region Cluster

- 재해 복구와 관련된 모든 작업을 간소화합니다
- 클러스터들이 여러 데이터 센터에 배포되고 Confluent Platform의 내부 복제 엔진을 활용하여 Topic 데이터가 여러 위치에 자동으로 복제되도록 합니다
- 자동화된 클라이언트 장애 조치를 지원하므로 운영자는 여러 개발 팀 간에 조정하거나 DNS 재구성 및 오프셋 변환 없이 백업 데이터 센터로 즉시 장애 조치하도록 할 수 있습니다.
- **L 통신사 OSS/BSS 운영계 및 현행화 Cluster에 적용함**

Multi-Region Cluster의 장점 및 고려 사항

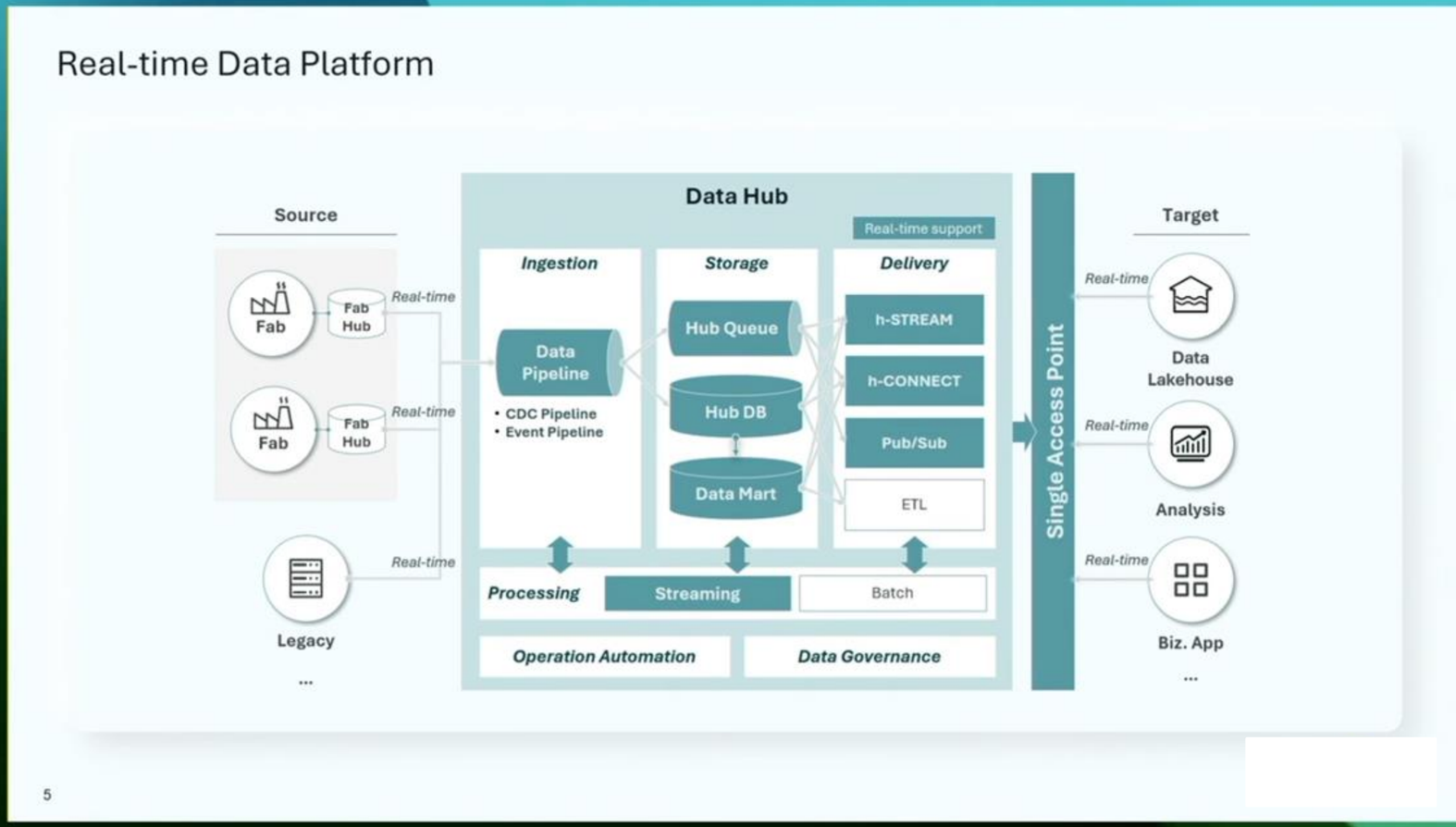
- RTO 가 0에 가깝고, 토픽 레코드의 RPO가 0
- 자동화된 클라이언트 fail over 제공
- DataCenter 복구시 자동 Fail back 가능
- 고려사항: Latency < 50 ms 네트워킹 요구(최대 100 ms 미만)
- 고려사항: 3개의 데이터센터 구성 필요



※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

Kafka Summit London 2024 :

Building a Self-Service Stream Processing Portal: How And Why

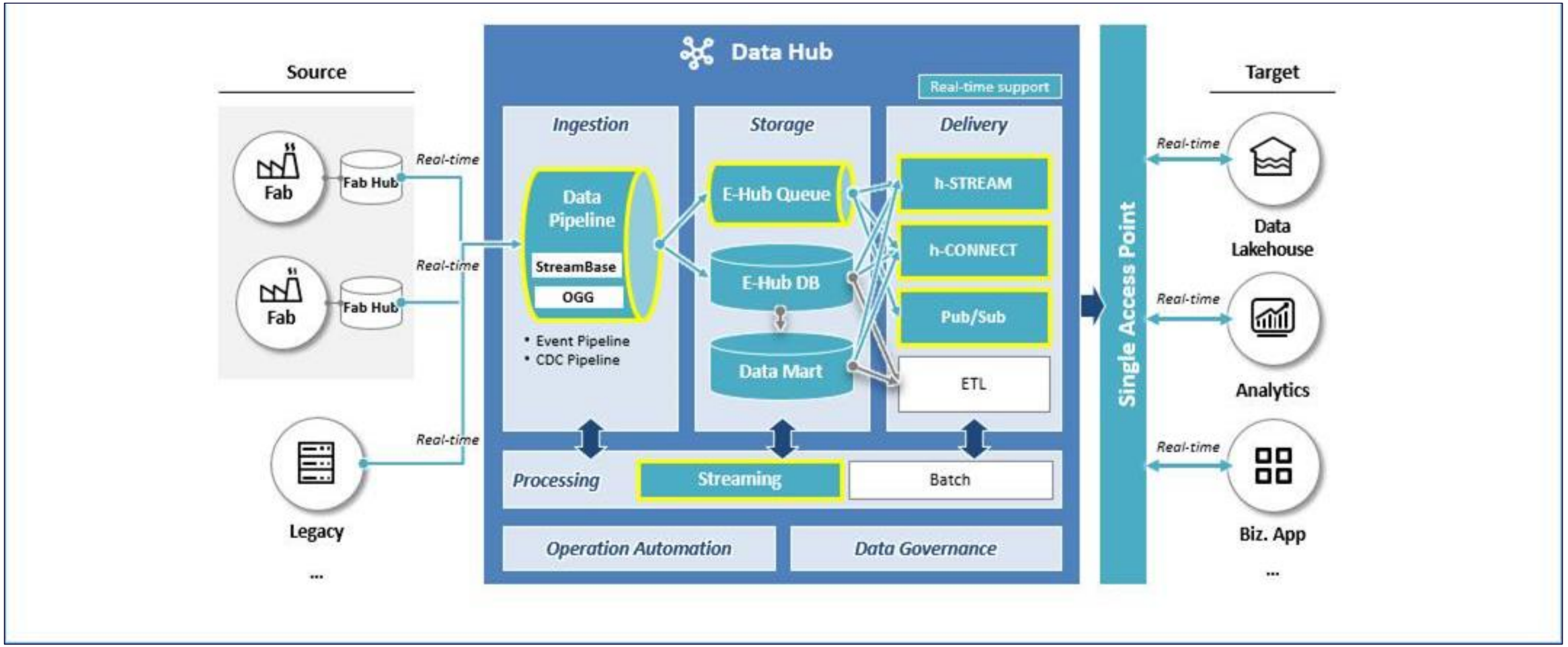


※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

국내 기업 도입 사례 - S사 반도체

데이터허브(Data Hub) 시스템은 반도체 제조공장 데이터를 관리하기 위한 S사의 **실시간 스트리밍 플랫폼(Real-time Streaming Platform)**

- ✓ **데이터 일관성을 보장**하고 사용자 시스템 **서비스 품질을 보장**하기 위해 **단일 액세스 포인트**를 제공
- ✓ **사용자가 적시에 결정**을 내릴 수 있도록 **실시간** 데이터 인터페이스를 제공

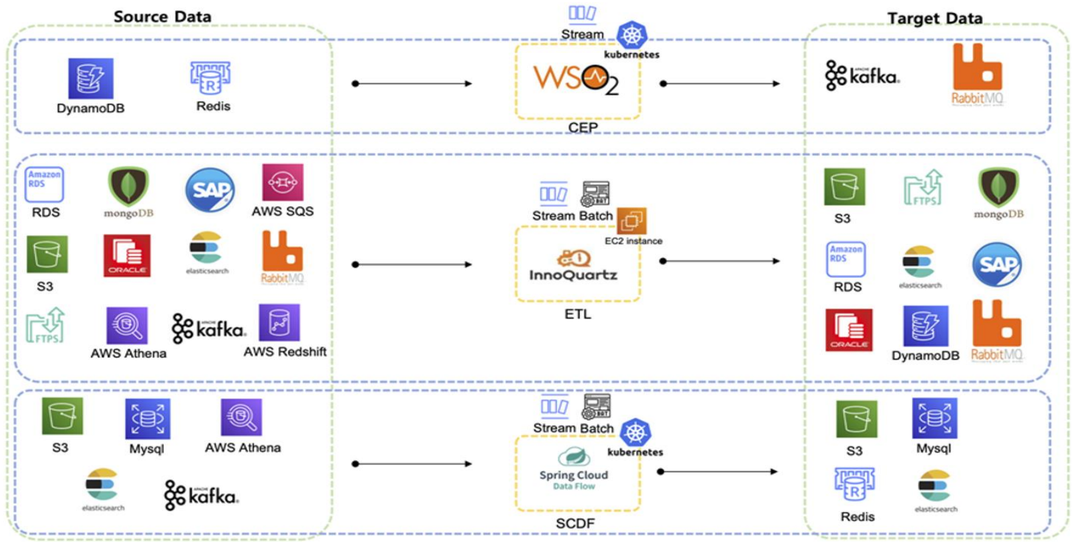


※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

L 쇼핑 데이터 파이프라인 통합/개선 프로젝트

데이터 파이프라인 통합/개선 배경

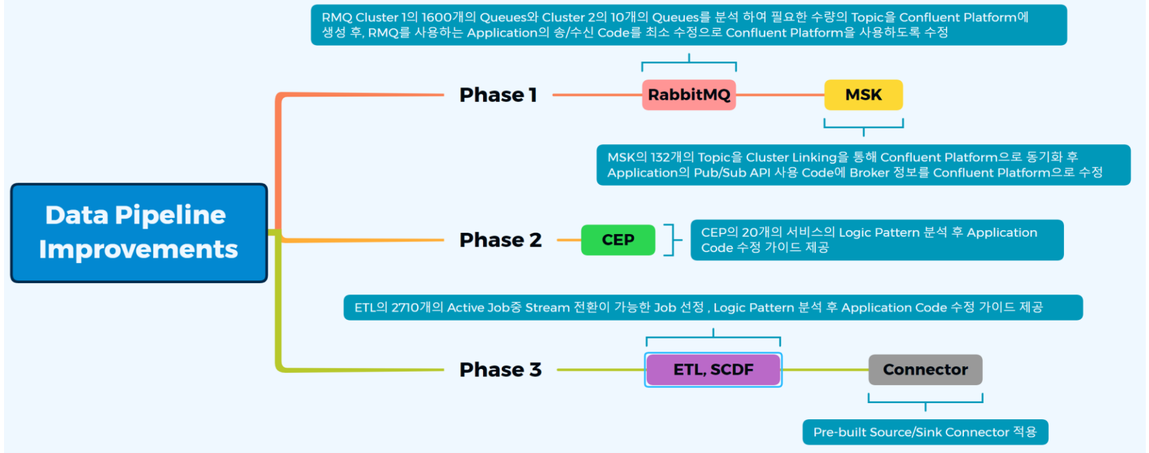
- CEP, ETL 유지보수 및 외주 용역 등에 높은 비용 발생(담당자 변경 및 퇴사등)
- CEP, MQ, Kafka, ETL, SCDF 데이터 파이프라인 일원화 및 데이터 추적관리 거버넌스
- 데이터 관리 문제, 데이터 파이프라인 운영/주체관리 효율성 제고



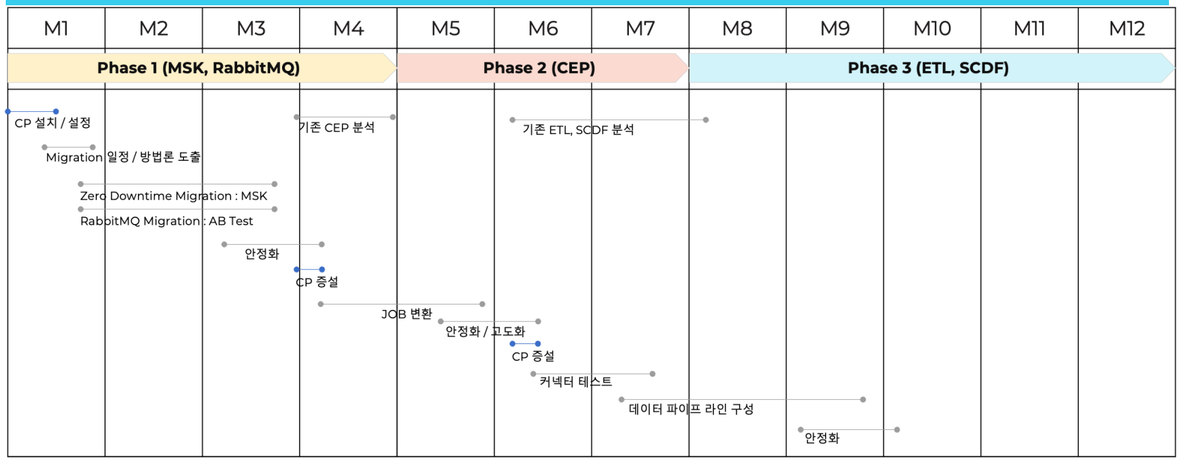
데이터 파이프라인 통합/개선 진행 프로세스

고객 요구사항 및 환경 분석 → PoC로 고객 요구사항 검증 → 프로젝트 제안 → Confluent PS 컨설팅 수행 → 구축 → 단계별 개발 및 마이그레이션

운영중인 서비스의 영향도 최소화를 위한 단계적 마이그레이션 방안 적용



비용 효율적인 프로젝트 일정 및 구성 관리

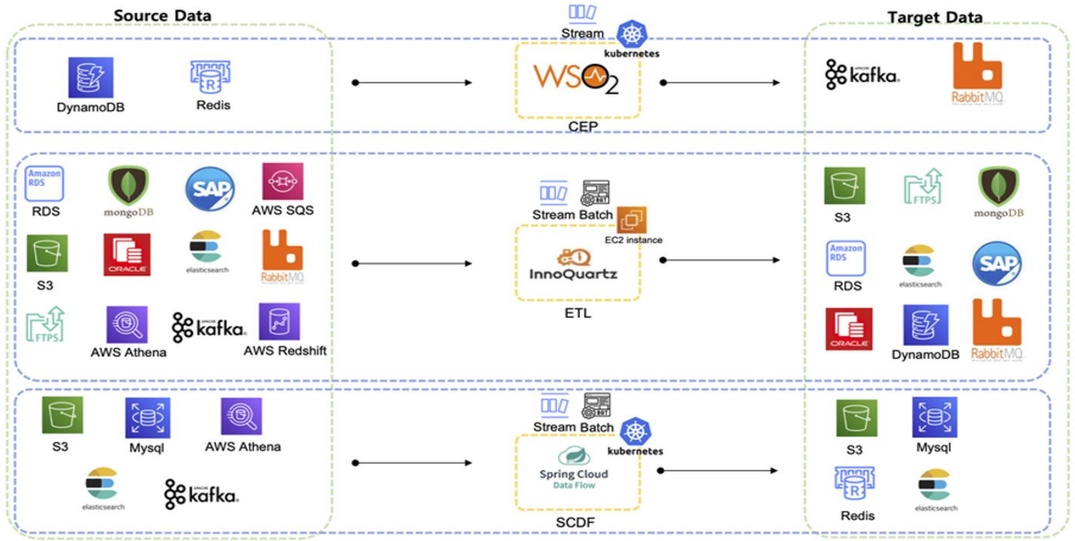


※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

L 쇼핑 데이터 파이프라인 통합/개선 프로젝트

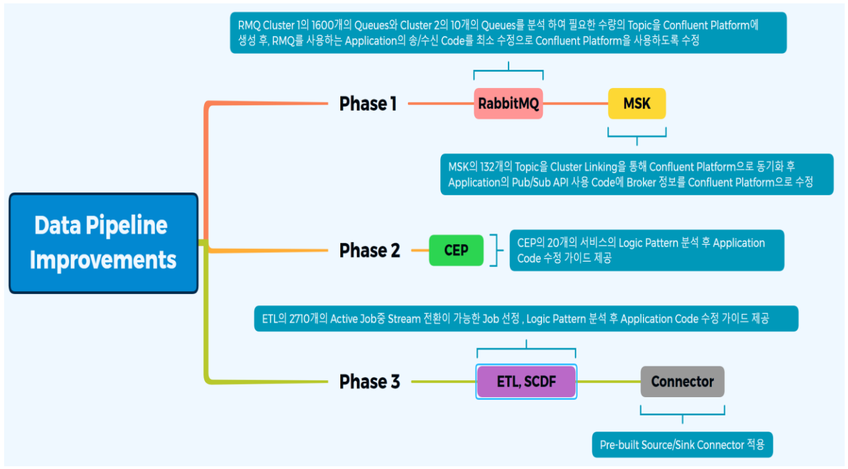
데이터 파이프라인 통합/개선 배경

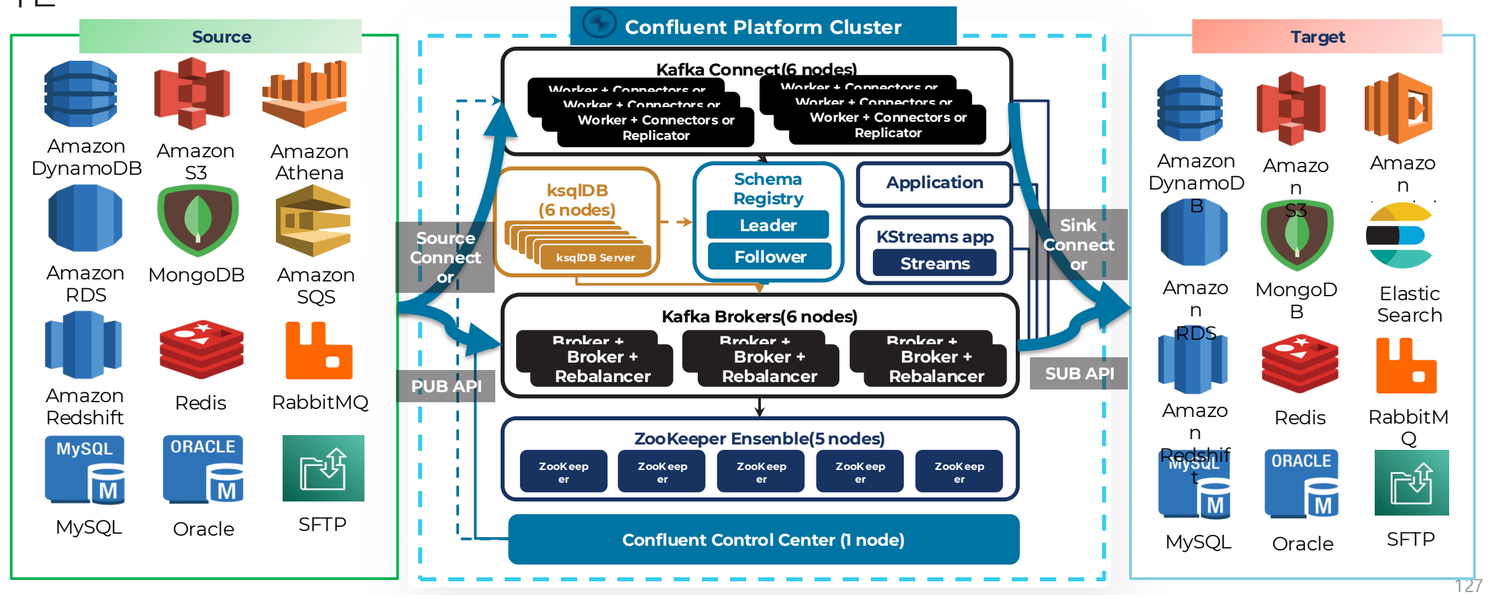
- CEP, ETL 유지보수 및 외주 용역 등에 높은 비용 발생(담당자 변경 및 퇴사등)
- CEP, MQ, Kafka, ETL, SCDF 데이터 파이프라인 일원화 및 데이터 추적관리 거버넌스
- 데이터 관리 문제, 데이터 파이프라인 운영/주체관리 효율성 제고



데이터 파이프라인 통합/개선 진행 프로세스

고객 요구사항 및 환경 분석 → PoC로 고객 요구사항 검증 → 프로젝트 제안 → Confluent PS 컨설팅 수행 → 구축 → 단계별 개발 및 마이그레이션





※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

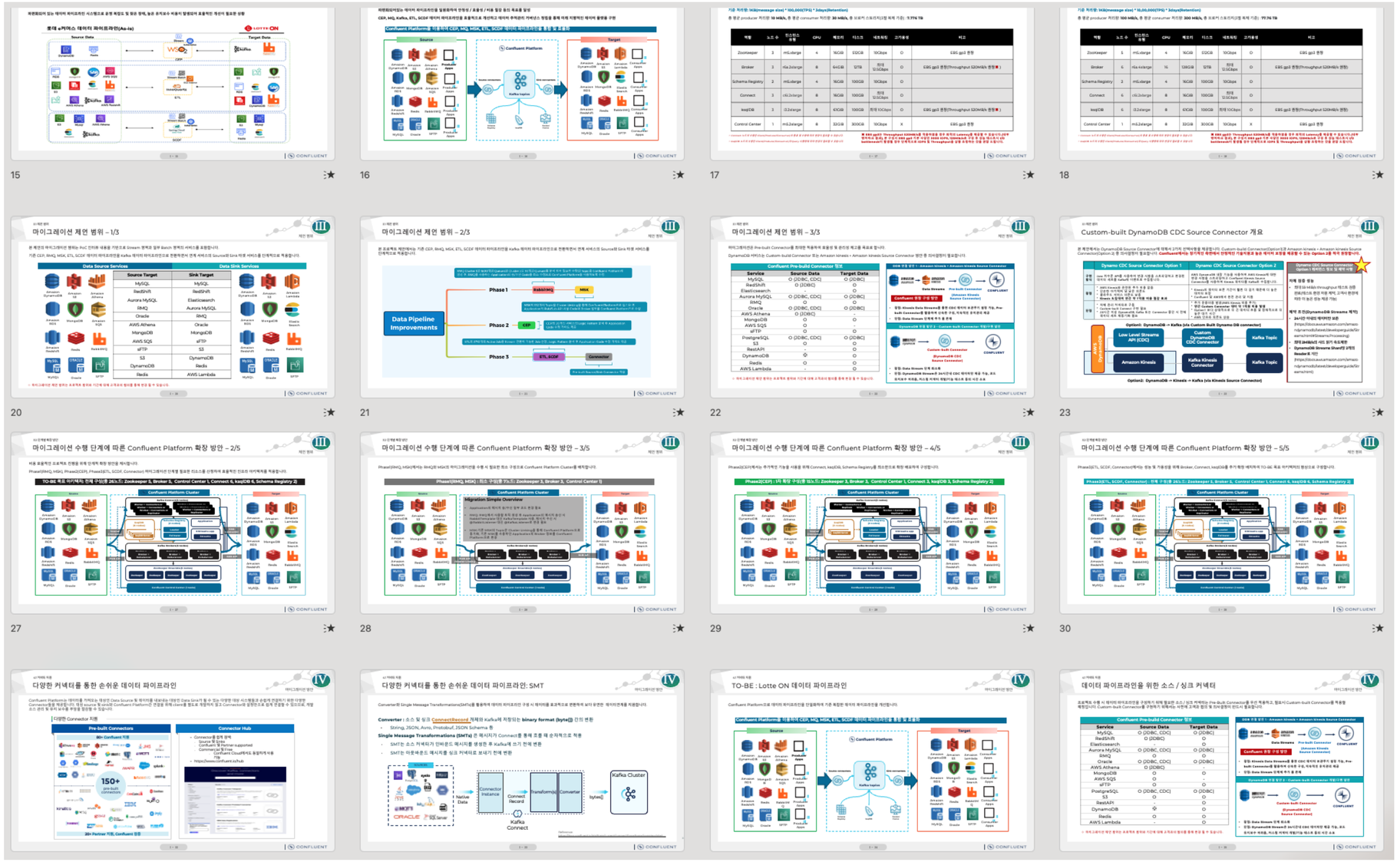
L 쇼핑 데이터 파이프라인 통합/개선 프로젝트

비용 효율적인 프로젝트 진행을 위해 단계적 확장 방안 적용

- Phase1(RMQ, MSK)
- Phase2(CEP)
- Phase3(ETL, SCDF, Connector
- 마이그레이션 단계별 필요한 리소스를 산정하여 효율적인 인프라 아키텍처를 적용

Migration Quick Overview

- Application의 메시지 송/수신 일부 코드 변경 필요
- RMQ: RMQ에서 사용할 토픽 생성 후 Application의 메시지 송신 시 RabbitTemplate 대신 KafkaTemplate 사용, 메시지 수신 시 @RabbitListener 대신 @KafkaListener로 변경 필요
- MSK:기존 MSK의 Topic은 Cluster Linking을 통해 Confluent Platform으로 동기화, 기존 MSK를 사용하던 Application의 Broker 정보를 Confluent Platform으로 변경



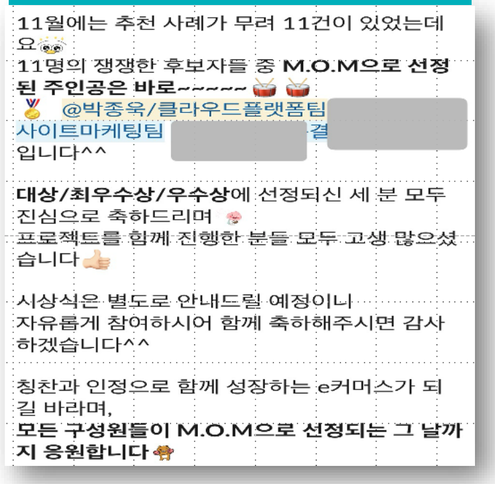
※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

L 쇼핑 데이터 파이프라인 통합/개선 프로젝트

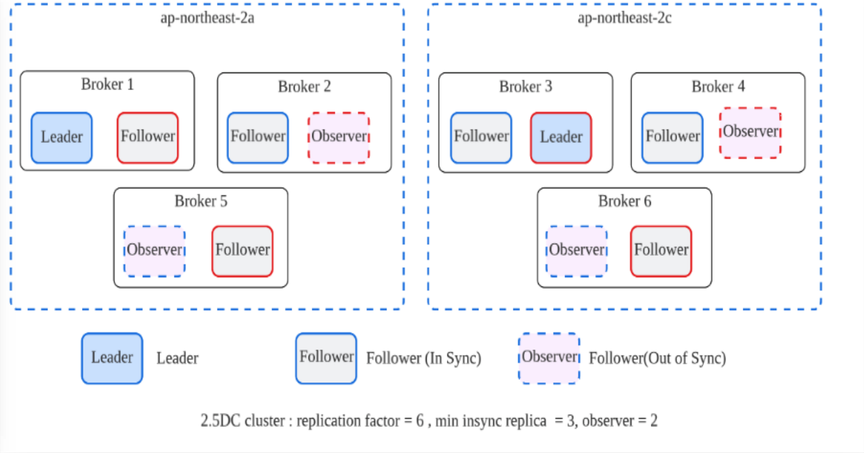
Confluent Platform 도입 효과

- TCO 절감: 운영,인력,개발,보수 용역 및 라이선스 비용 절감
- 데이터 거버넌스 및 통합: 전사 데이터 실시간 통합 및 데이터 제품화
- EDA 최적화: MSA구현 최적화, 이벤트 데이터의 실시간 비동기 처리로 MicroService간 데이터 종속성 제거, 단일 장애 지점 제거 및 개발 생산성 극대화

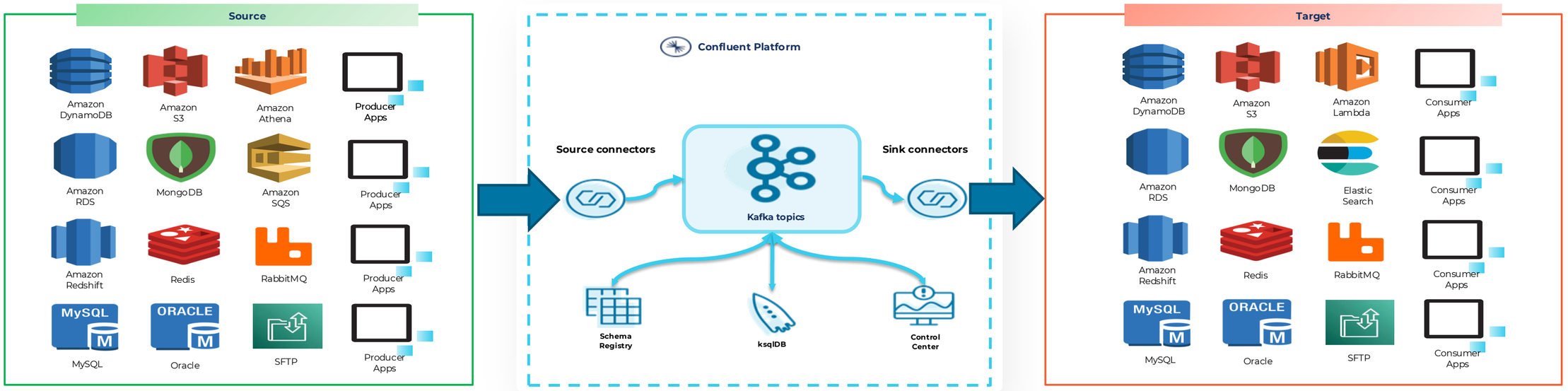
2024년 고객사 프로젝트 대상 수상



Multi Region Cluster Architecture 도입으로 높은 서비스 가용성 확보



Confluent Platform을 이용하여 CEP, MQ, MSK, ETL, SCDF 데이터 파이프라인을 통합 및 효율화

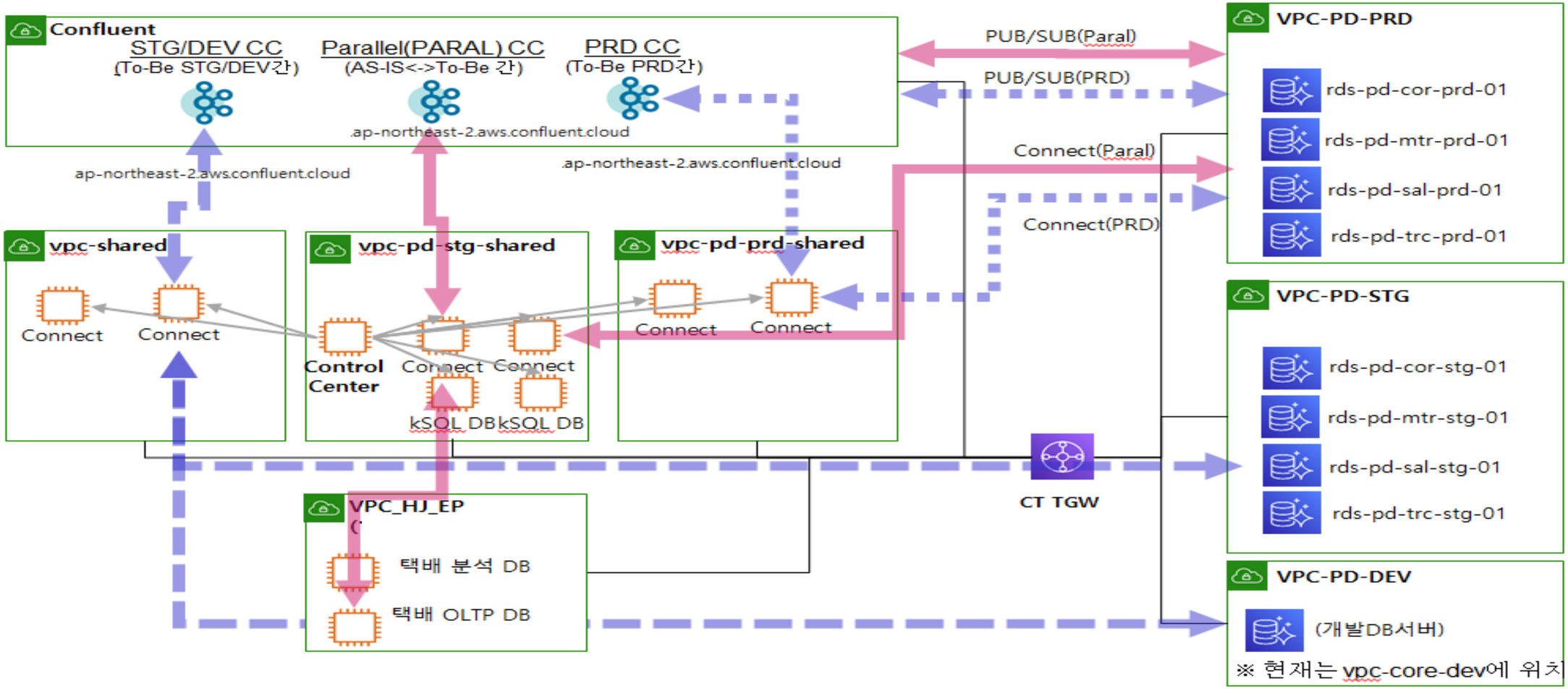


※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

H사 택배 Core 시스템 구축 사업

택배 코어 시스템을 퍼블릭 클라우드 환경으로의 전환 사업 수행

- MSA와 실서비스 무중단 시간 요구사항 수용 가능한 플랫폼 구축
- Confluent 솔루션을 통한 On-Prem 환경의 Public Cloud 로의 데이터 이관



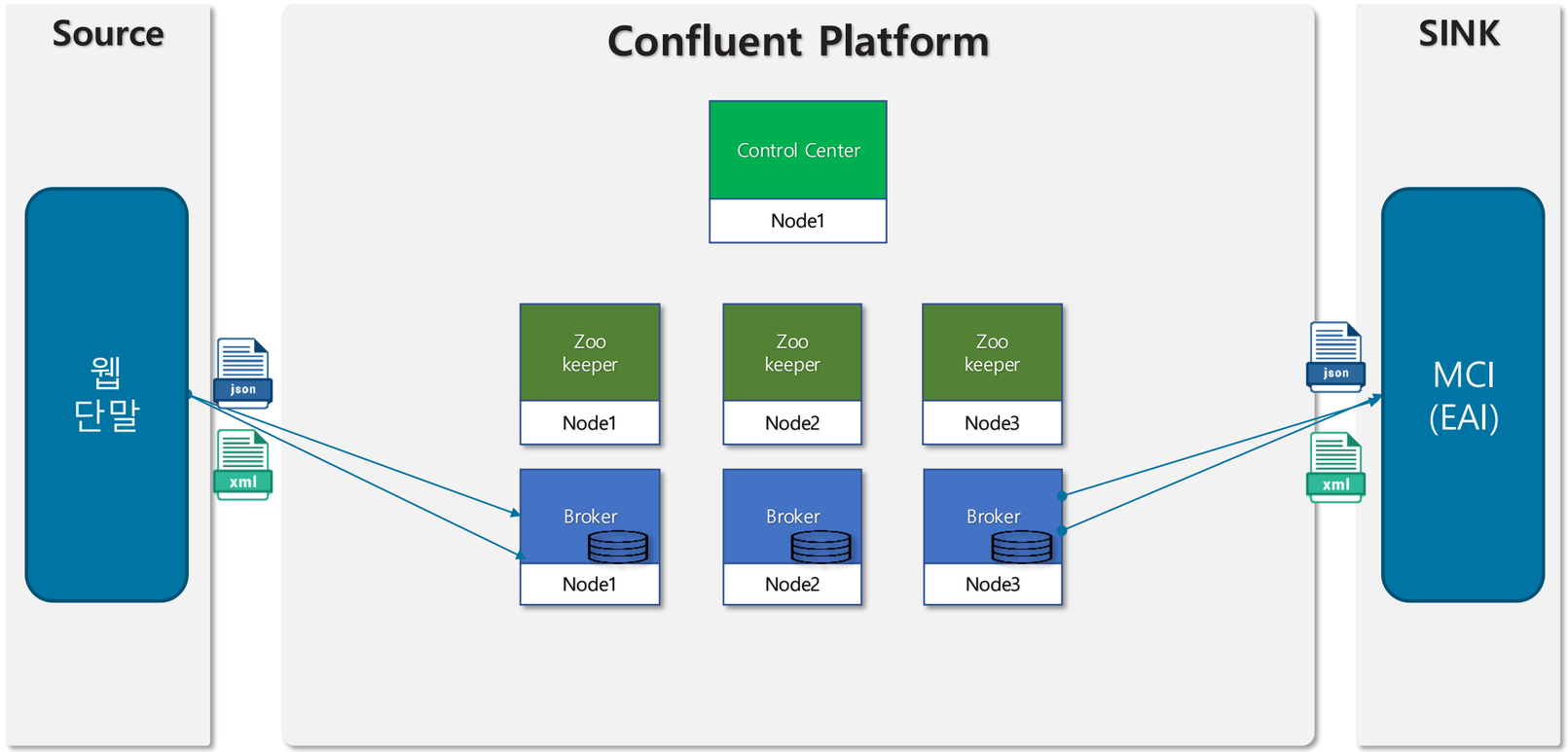
※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

국내 S 금융 사례

웹단말 시스템과 채널간의 유기적인 거래를 위한 Queue 시스템 구축

웹단말에서 발생되는 Event는 다양한 포맷(JSON, XML, ..)으로 Pub/Sub API를 통하여 Confluent Platform으로 수집되며, 유실 없이 보관주기에 따라 안전하게 저장됩니다. 또한, 실시간으로 MCI(EAI)에 의하여 Polling된 데이터는 안전하게 MCI로 전달되게 되어 유기적인 거래가 가능한 Queue 시스템 구축을 가능하게 합니다.

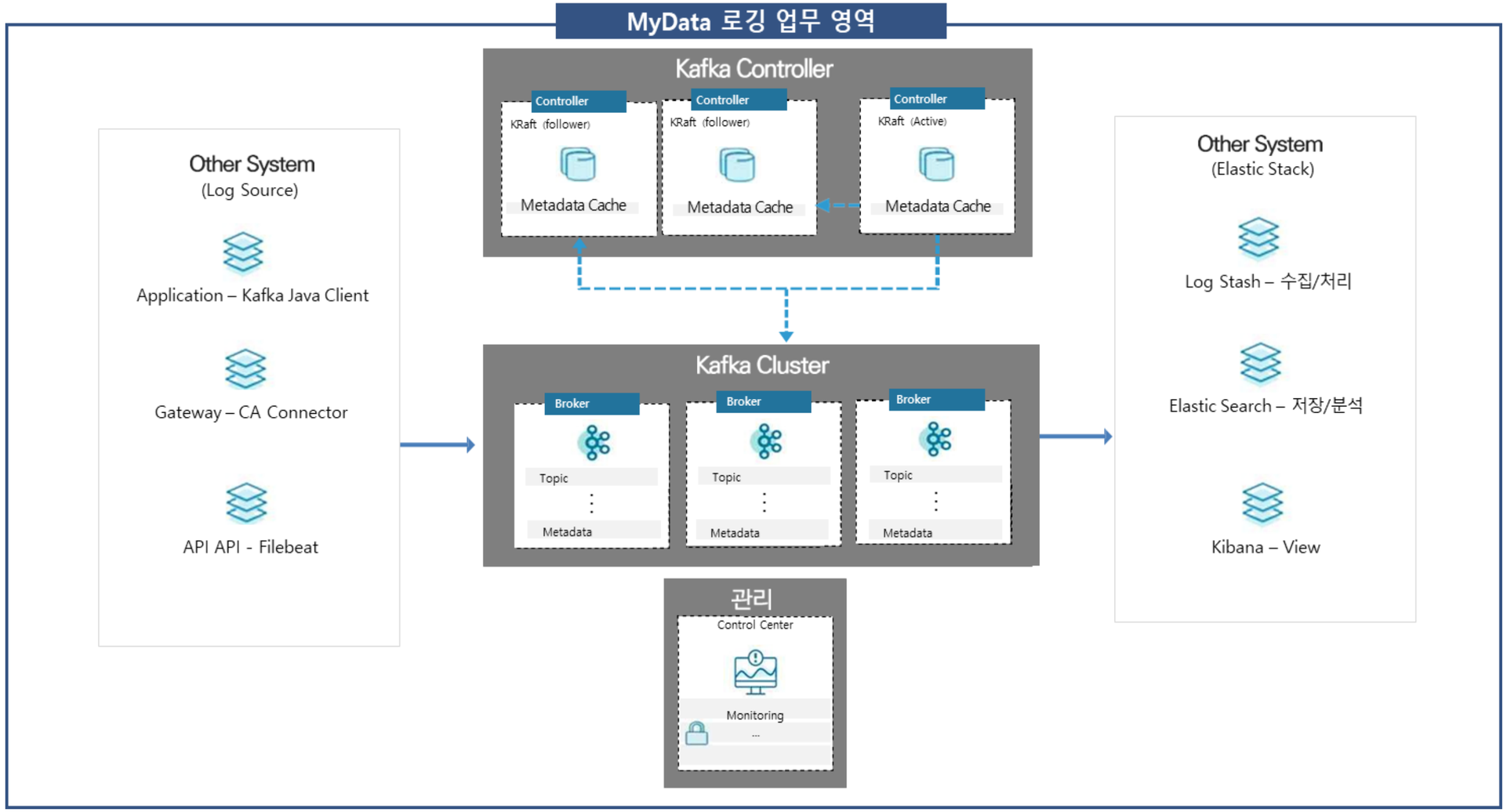
- Confluent Platform 기반 Queue 시스템 구축



※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

국내 S 은행 금융 사례 - MyData 로깅시스템

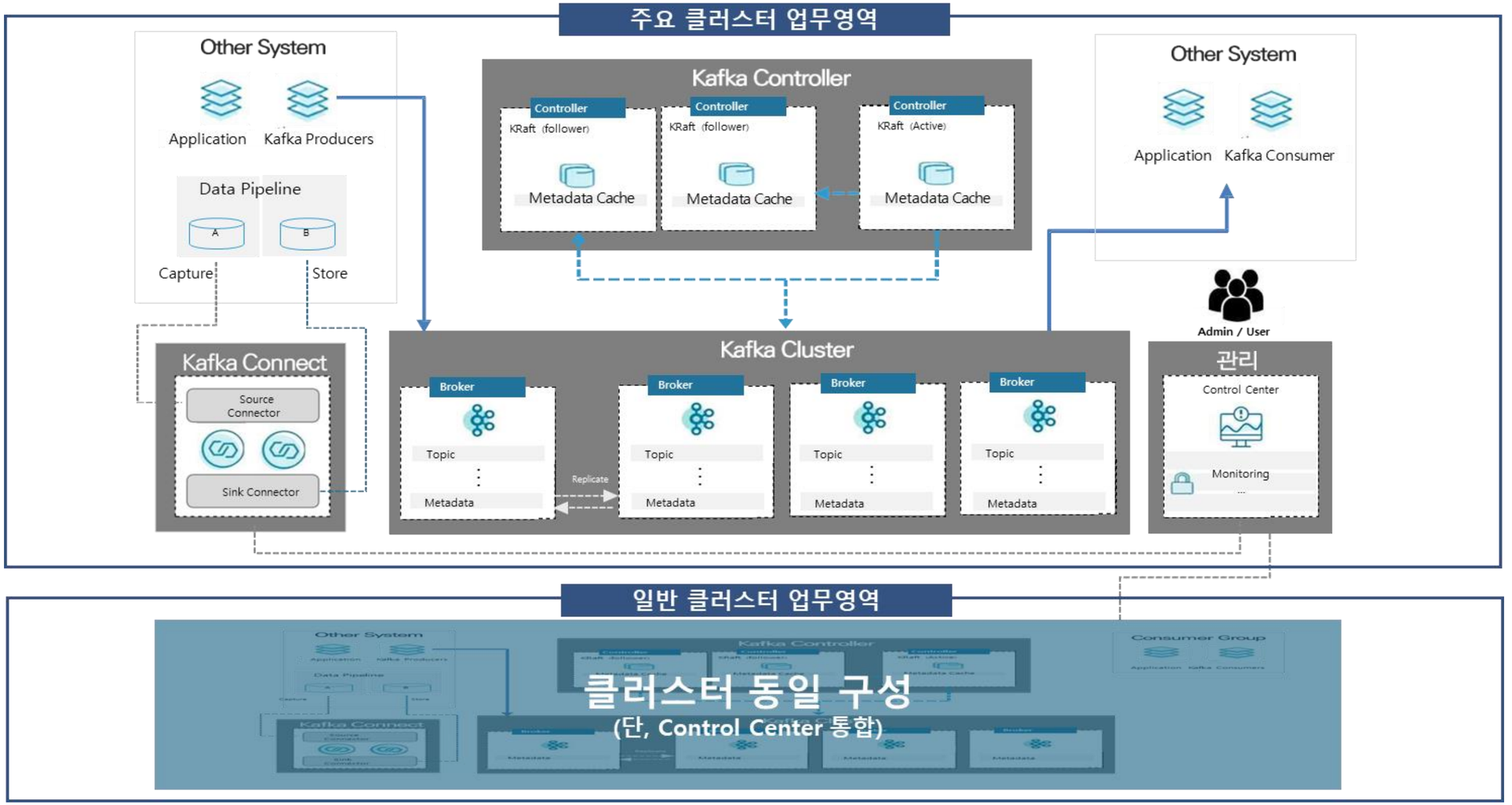
MyData 플랫폼 운영과 마이데이터 산업의 확장에 대하여 빠른 속도와 안정성을 기준으로 데이터 전송 및 수신하기 위하여 Confluent 도입되었습니다.



※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

K 금융결제 기관 사례 – 차세대 통합 로깅시스템

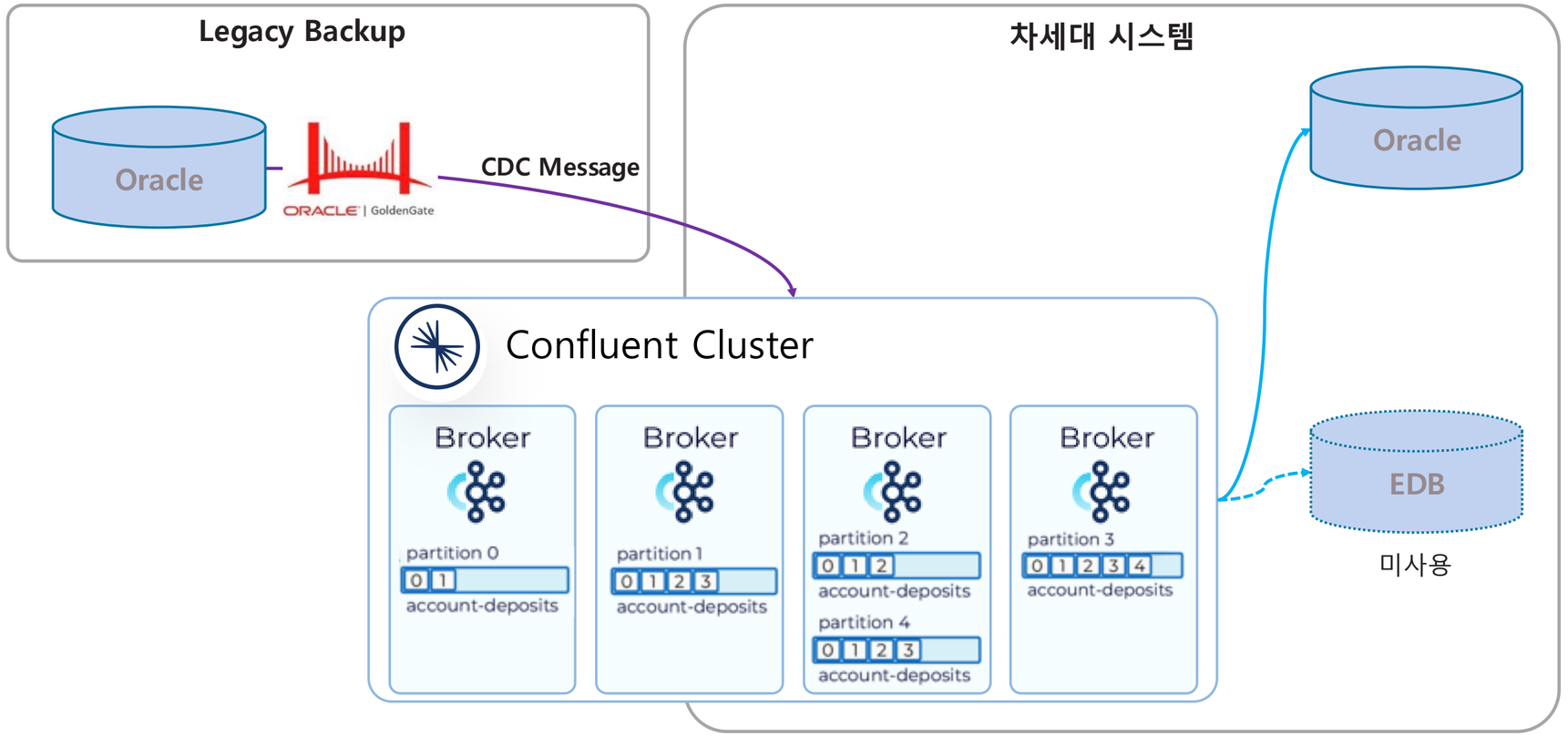
차세대 통합 로깅 시스템의 고도화를 통해 서비스 안정성과 영속성, 재해복구 대응 체계 구축



※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

국내 S 투자증권 금융 사례

차세대 시스템을 개발하면서 Legacy 시스템과 차세대 시스템의 데이터 동기화 작업이 필요하였습니다.
Legacy 시스템의 Oracle DB의 데이터는 Oracle사의 Ogg connector 로 CDC 메시지를 전송하고, Confluent를 통해서 차세대 시스템의 Oracle DB에 데이터 동기화를 수행합니다.
최초 설계시는 Target DB로 Oracle DB가 아닌 EDB(PostgreSQL) 사용을 검토하면서, 이기종 DB에 동기화를 하기 위해 Confluent를 사용하는 것으로 결정되었으나 최종적으로는 동일한 Oracle DB를 사용하고 있습니다.
DB 동기화를 위한 용도 외에도 MSA 아키텍처에서 Message Hub로서의 마이크로 서비스간의 비동기 메시지 처리를 수행하고 있습니다.

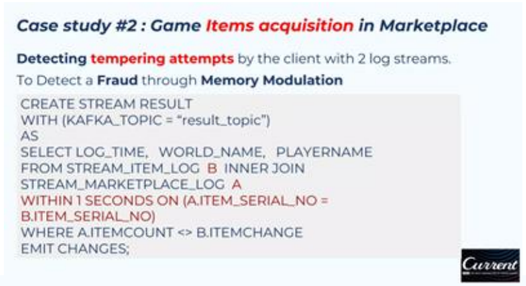


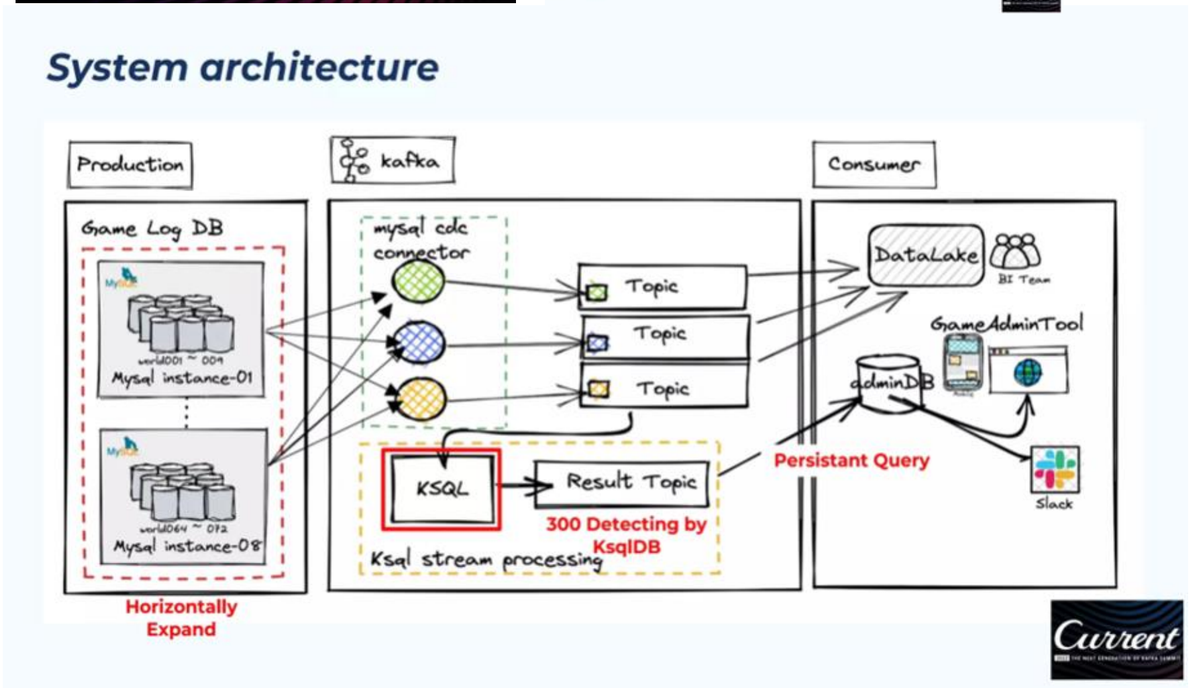
※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

Kakaogames – Fraud Detection in Real Time MMORPG

Kakaogames는 더 나은 분석을 위해 다양한 게임 스튜디오 전체의 실시간 게임 로그 데이터를 표준화하는 파이프라인을 구축하여 게임 내 어뷰징을 추적하고 출시 시간을 단축하며 궁극적으로 풍부한 게임 경험을 제공하였습니다. 그 결과 실시간 데이터 추적 및 분석, 시장 출시 시간 단축, 향상된 게임 경험을 통해 활성 사용자 및 수익 증가의 효과를 얻으셨습니다.







2022년 Current(Confluent 연중 고객행사)에서 고객 성공 사례 발표
ksqlDB를 이용한 실시간 사기 탐지 적용 사례

Challenges

- 사용자 행위에 대한 분석을 현재 일 배치 위주로 분석 후 서비스에 반영함으로 인한 서비스 품질 저하
- 다양한 외부/적정 솔루션을 운영하기 위한 인력 및 노하우 부족

Solutions

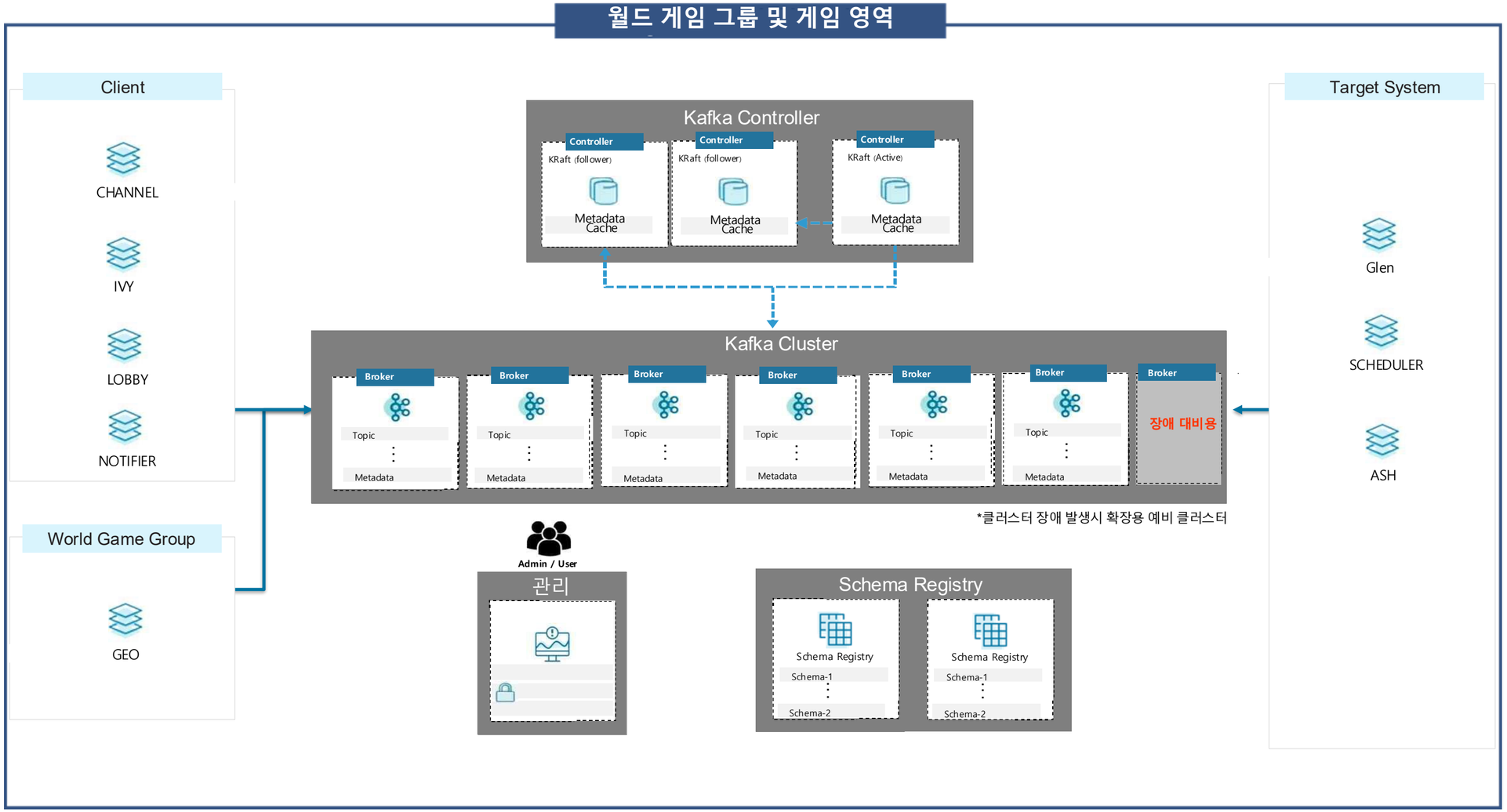
- Confluent Platform(Kafka), ksqlDB

Results

- 빠른 서비스 출시
- 수 많은 서버의 사용자 행위를 실 시간으로 수집하여 다양하게 실 시간으로 분석 – 이상 징후 탐지 (Fraud Detection)
- 400 개 이상의 Persistent Query 수행
- 적은 인력으로 안정적 서비스 수행

출처: https://www.confluent.io/ko-kr/events/current-2022/successful-fraud-detection-in-real-time-mmorpg-games/

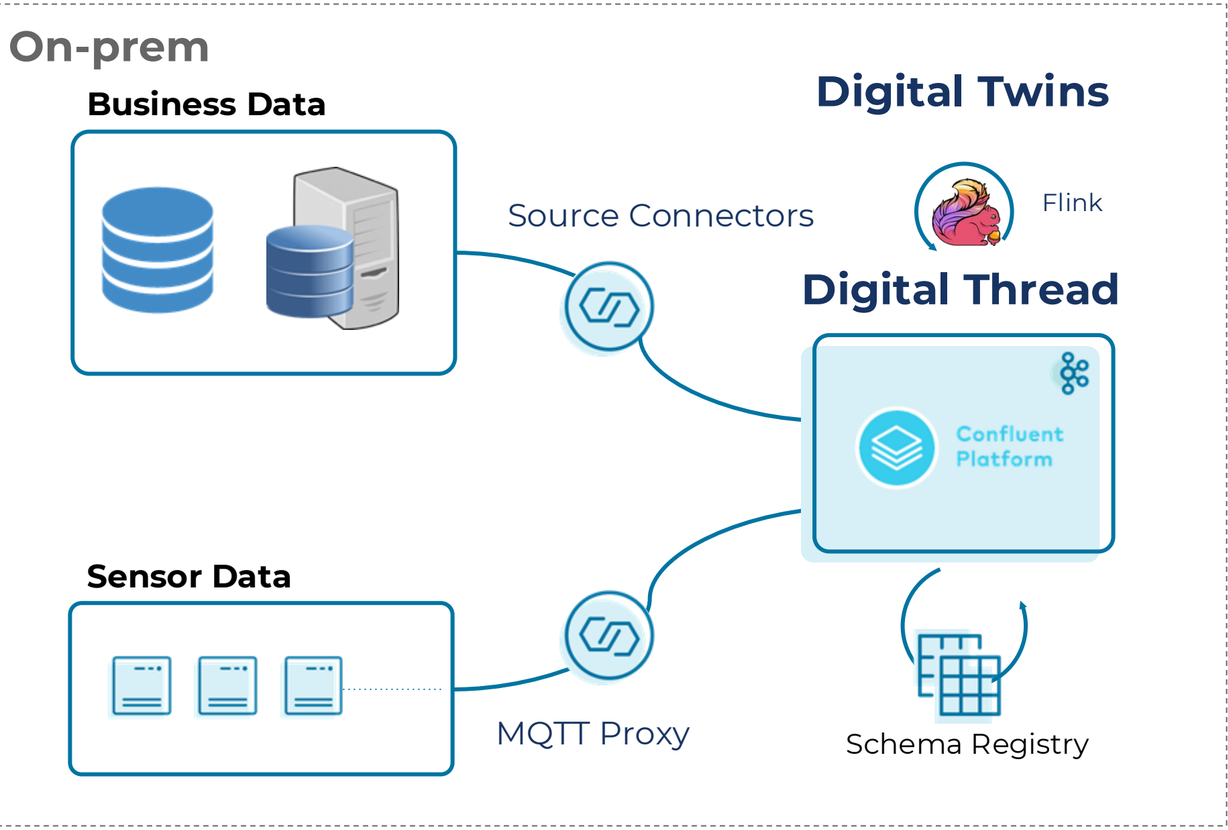
N 게임사 – MMORG 게임 이벤트 플랫폼



※고객 정보제공 동의 문제로 상세 정보 제공은 어렵습니다. 해당 고객 정보에 대한 외부 공유 및 유출을 금하여 주십시오.

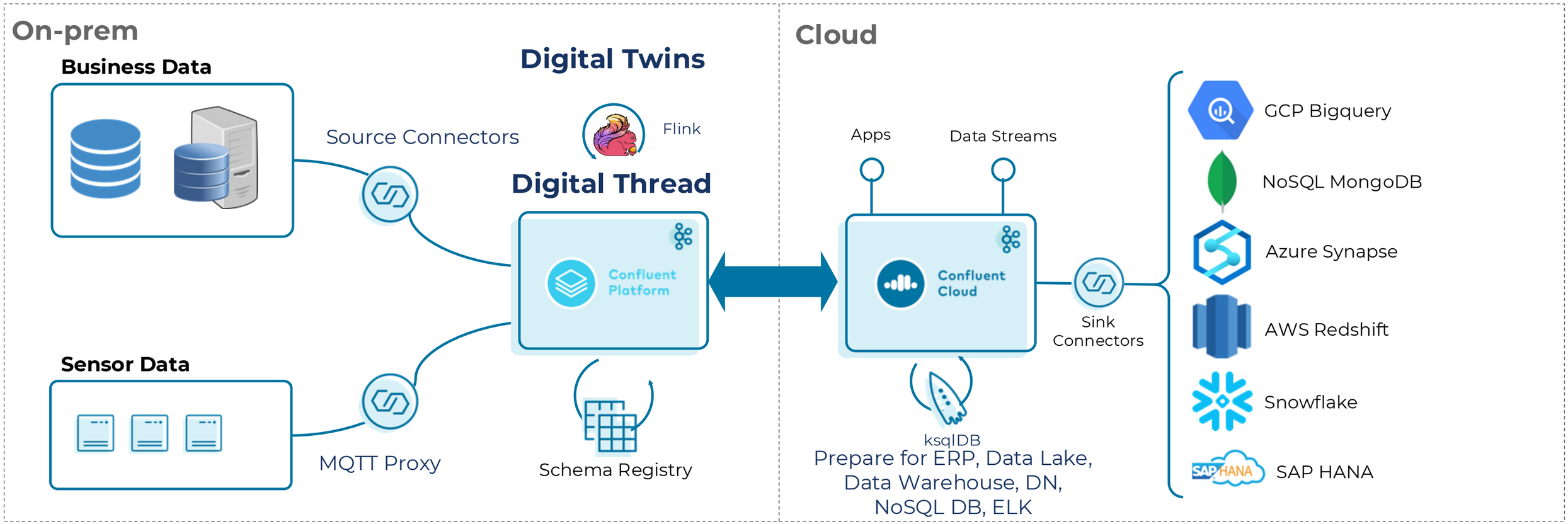
디지털 스레드 - Digital Twins

디지털 스레드는 제품이나 설비의 **전체 생애주기(기획–설계–제조–운영–서비스–폐기)**를 따라 흐르는 **데이터의 논리적 연결(연속된 데이터 "스레드")**을 의미합니다.



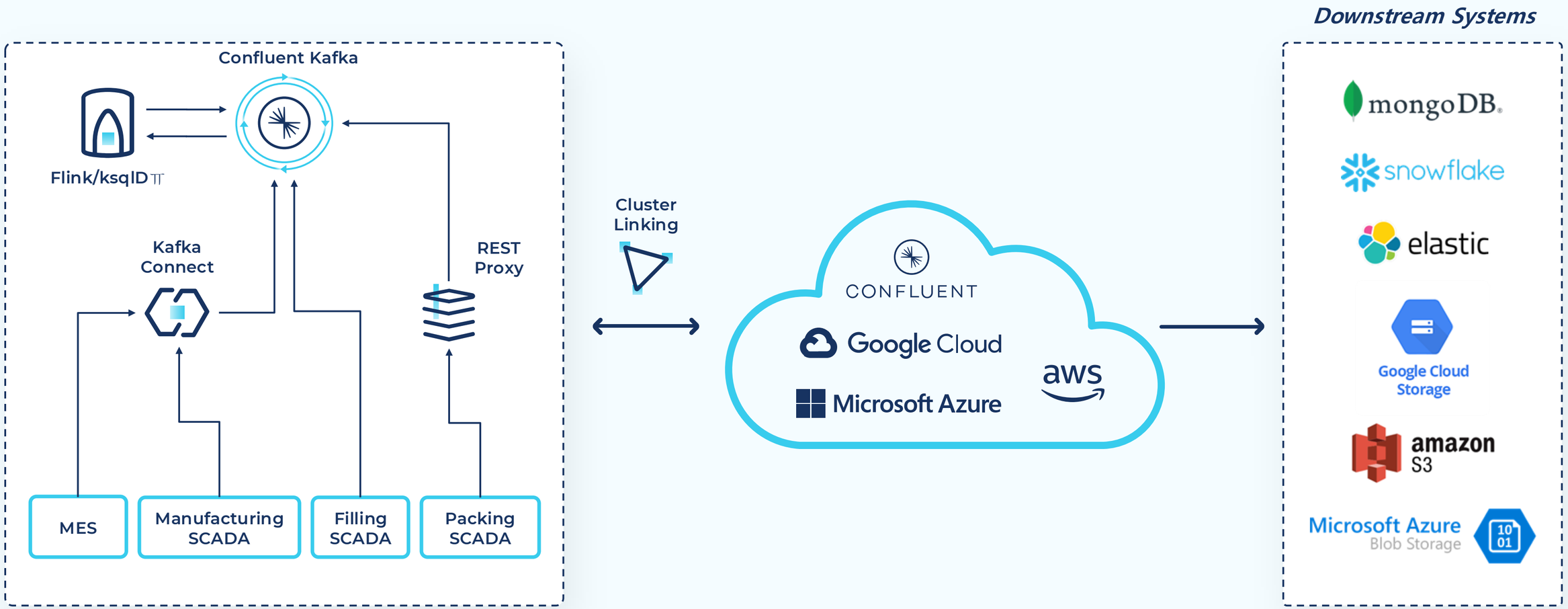
- 커넥터 및 MQTT 프록시를 통해 비즈니스 및 센서 데이터 수집
- 디지털 스레드를 활용하기 위한 핵심 데이터 소스 구축
- 디지털 트윈을 생성하고 가치 창출:
  - 증강 현실
  - 예측 분석
  - 예측 유지보수
  - .....

그라운드에서 클라우드로 손쉬운 기술 혁신을 위한 연결



- 클라우드 연결 및 네트워크 비용 절감
- 필요한 모든 솔루션을 신속하게 연결

제조 환경을 위한 하이브리드 클라우드 아키텍처



✓ 데이터 파이프라인 현대화

✓ 분석 및 AI 활용 사례

✓ 강한 결합을 제거한 유연한 아키텍처

BMW - Cars become Software





BMW GROUP

직면 과제: BMW 그룹은 30개 이상의 생산 시설과 전 세계 판매 네트워크에서 생성된 모든 데이터를 글로벌 비즈니스의 모든 사람이 실시간으로 사용할 수 있도록 해야 했습니다.

솔루션: BMW Group은 Confluent를 사용하여 Apache Kafka 배포를 엔터프라이즈급으로 만들고 글로벌 조직 전체에서 실시간 데이터 스트리밍 및 데이터 흐름을 위한 중앙 허브를 만들었습니다.

결과:

- 혁신을 위한 전사 표준 데이터 플랫폼 구축
- Confluent의 완전 관리형 서비스로 탄력적인 확장을 통해 데이터 인프라 관리 부하 및 사용자 오류 제거
- 실시간 데이터에 대한 유비쿼터스 액세스
- 생산 공장의 에지 IoT 데이터를 클라우드 애플리케이션 및 서비스에 연결하는 안정적이고 가동 중지 시간이 없는 데이터 스트리밍 플랫폼
- 정보에 입각한 의사 결정을 주도하는 실시간 비즈니스 통찰력
- SAP 솔루션, OPC UA와 같은 산업용 IoT 프로토콜, 클라우드 애플리케이션 및 서비스와의 완벽한 비즈니스 통합

BMW Group: Global Data Replication and Disaster Recovery

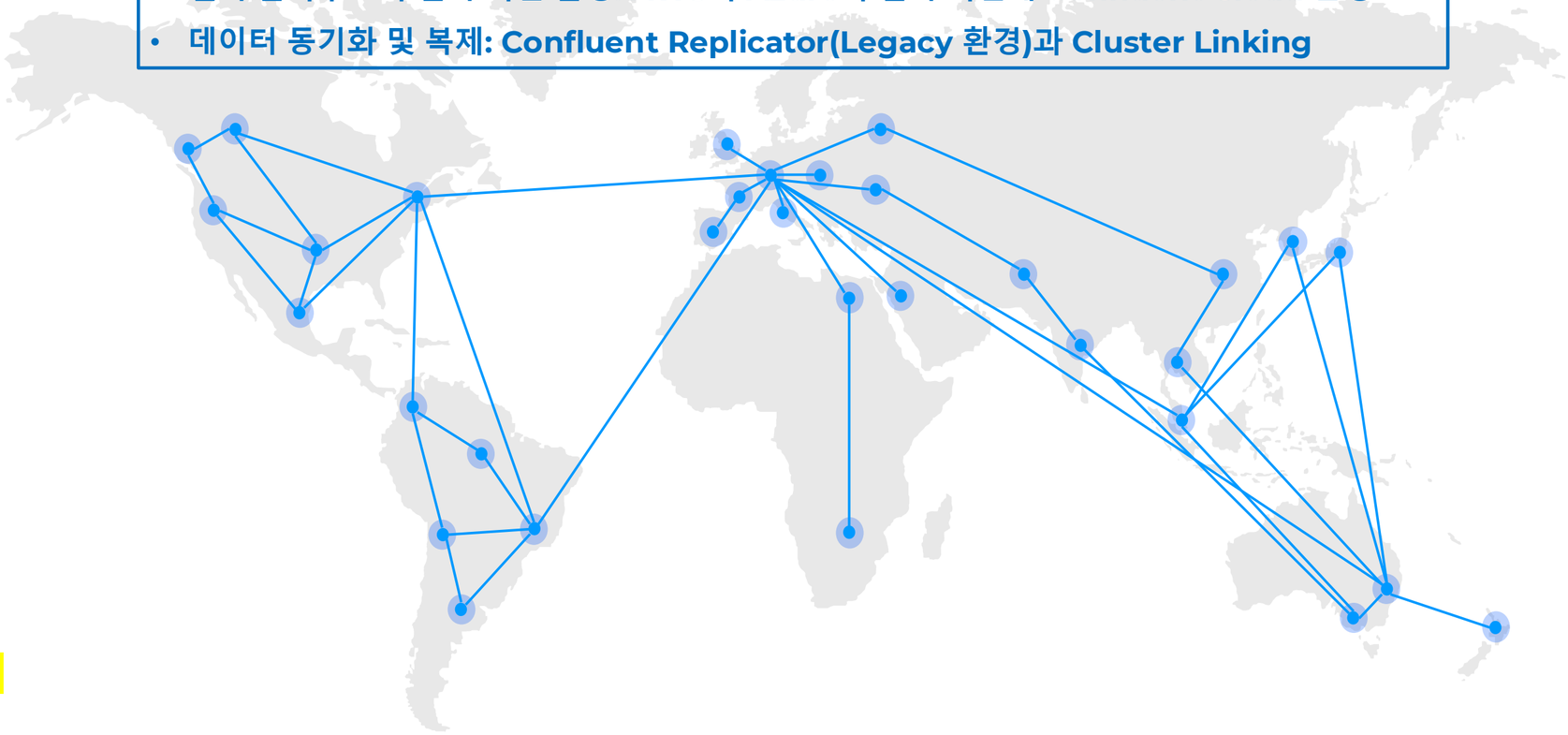
BMW 그룹에서는 다양한 비즈니스 유닛(BU) 간 데이터 공유, 멀티 클라우드 아키텍처, DR(Disaster Recovery) 전략 고도화, 그리고 글로벌 제조 현장 지원을 위해 Cluster Linking을 적극 도입하고 있습니다.

초기 On-prem 환경에 Confluent Platform을 기반으로 실시간 데이터 공유 및 DR 환경을 운영하며, 2021년 이후 AWS와 Azure 환경에 Confluent Cloud 도입으로 Kafka 운영 효율과 실시간 데이터 플랫폼을 고도화하고 있습니다.

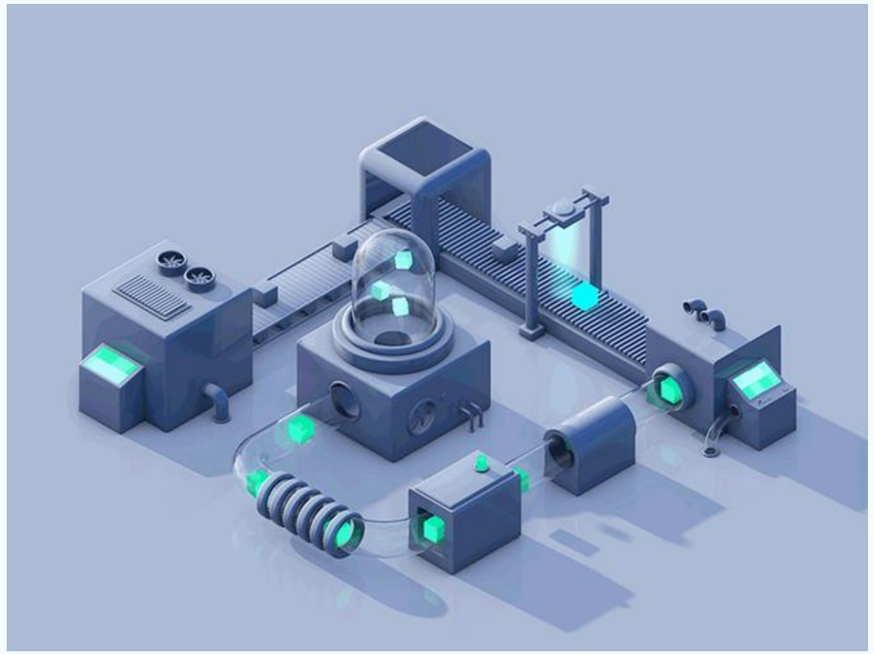
현재 30개 이상의 데이터 센터와 CSP Region에 Confluent Platform과 Confluent환경간 Replicator 기반의 데이터 복제를 Cluster Linking으로 지속적으로 전환 중에 있습니다.

Global Hybrid Cloud간 데이터 복제 및 재해 복구 환경 구성

- **글로벌 멀티 데이터 센터: On-Prem 환경을 위해 Confluent Platform 구축**
- **멀티 클라우드의 멀티 리전 환경: AWS와 Azure의 멀티 리전에 Confluent Cloud 운영**
- **데이터 동기화 및 복제: Confluent Replicator(Legacy 환경)과 Cluster Linking**



예측 유지보수(Predictive Maintenance) 사용 사례



- Temperature
- Vibration
- Pressure
- Sound Pressure
- Motor Current
- Oil Quality
- Magnetic Field



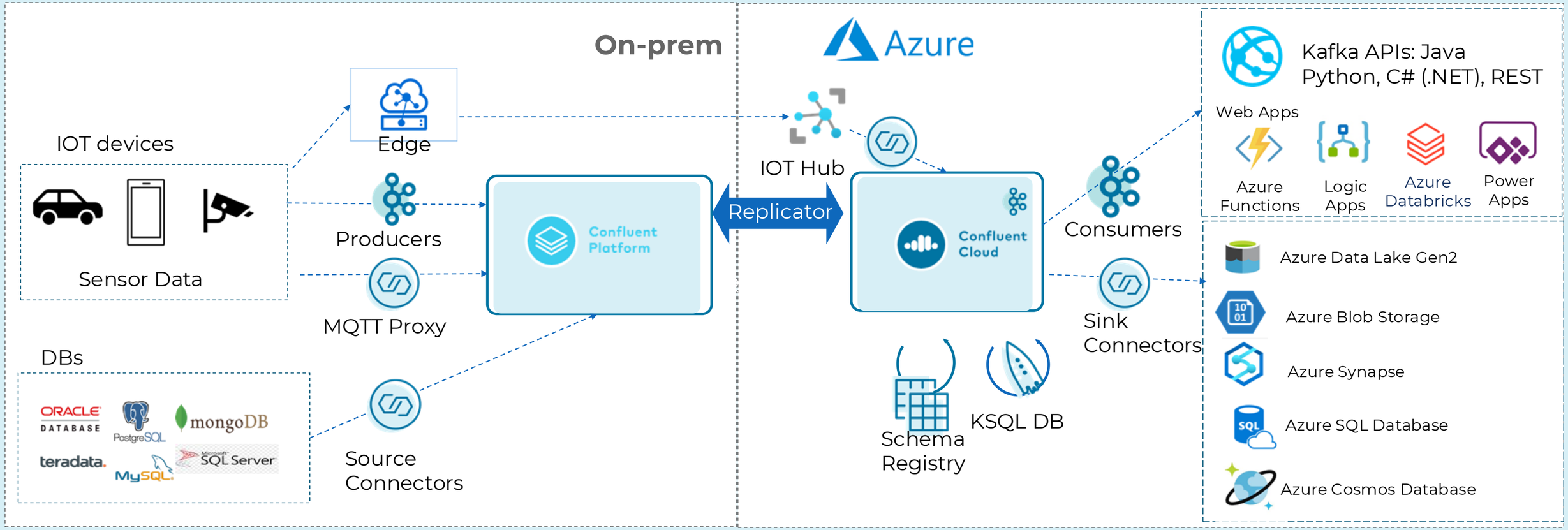


예측 유지보수를 통해 다운타임과 불량·정비 비용을 줄이고 OEE·수율·안전성을 동시에 높였으며, 현재는 AI를 적극적 도입하여 전 과정의 자동화를 개선 목표로 하고 있습니다.

글로벌 서비스를 위한 IOT 사용 사례

Hybrid Cloud 기반 IOT + Telematics 사용 사례

차량 텔레매틱스 데이터를 AI로 분석하여 예측 정비, 운전자 안전·코칭, 사용량 기반 보험, 경로·연비 최적화, 플릿 운영 개선, 맞춤형 인포테인먼트, 교통·인프라 분석 등 다양한 부가가치 서비스를 제공할 수 있는 기반을 만들 수 있게 되었습니다.



Intel – SIEM Optimization with MLOPS and AI

intel

직면 과제: 전사적으로 사이버 보안 위협을 신속하게 탐지하고 대응합니다.

솔루션: Confluent Platform 및 Kafka를 사용하여 높은 처리량, 낮은 대기 시간 이벤트 스트리밍으로 구동되는 확장 가능한 사이버 인텔리전스 플랫폼을 구축하였습니다.

사용한 Confluent 기능:

- Confluent Platform
- Confluent Control Center
- ksqlDB
- Multi-region clusters
- Role-based access control

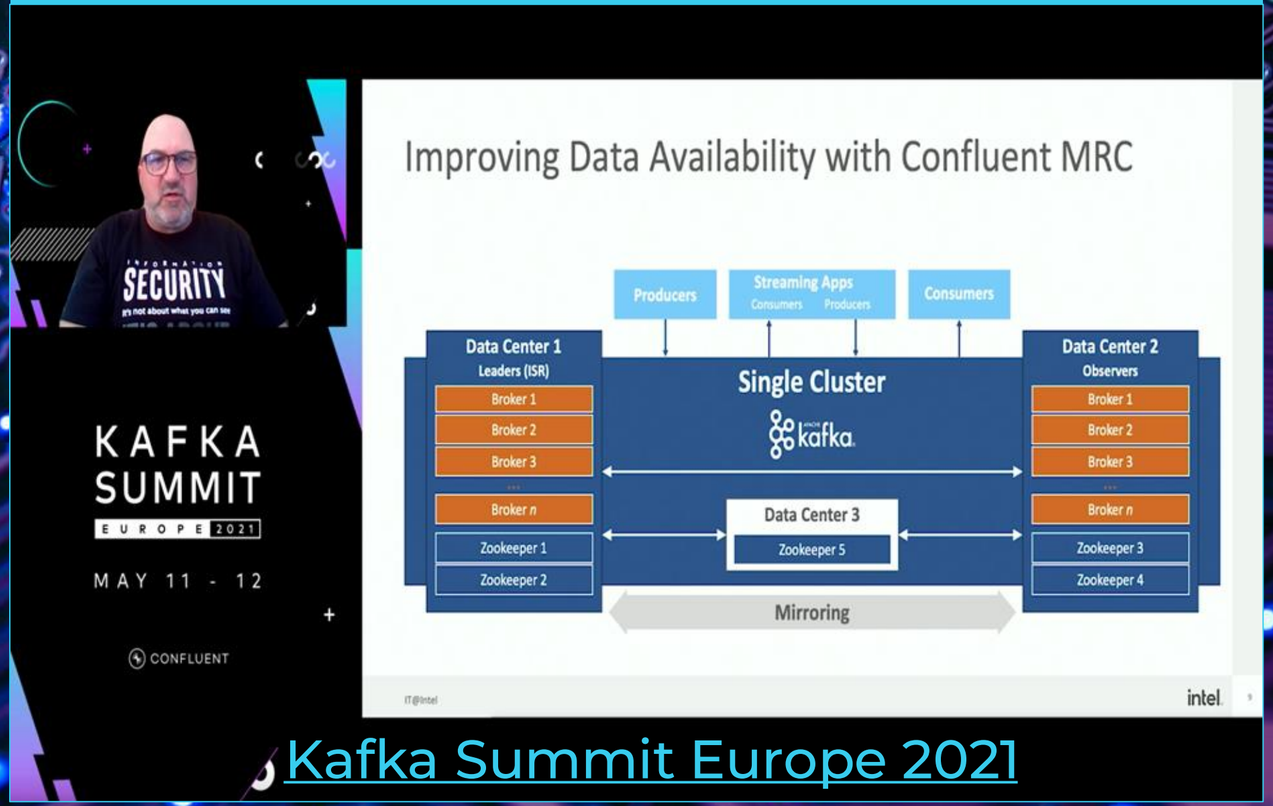
Results:

- 처리량이 많고 대기 시간이 짧은 이벤트 스트림으로 보안 위협을 탐지하고 대응하는 평균 시간 단축
- 맞춤형 지점 간 통합을 제거하여 기술 부채 감소
- 인스트림 데이터를 필터링하여 Splunk 및 다운스트림 시스템의 컴퓨팅 및 수집 비용 절감
- 더 빠르고 정확한 의사 결정을 가능하게 하기 위해 거의 실시간으로 여러 개의 서로 다른 보안 데이터 소스를 통합합니다

"Confluent의 엔터프라이즈 기능 및 고성능 인텔 아키텍처와 결합된 Kafka 기술은 인텔이 안전하게 빠르게 성장할 수 있도록 하는 우리의 사명을 지원합니다."

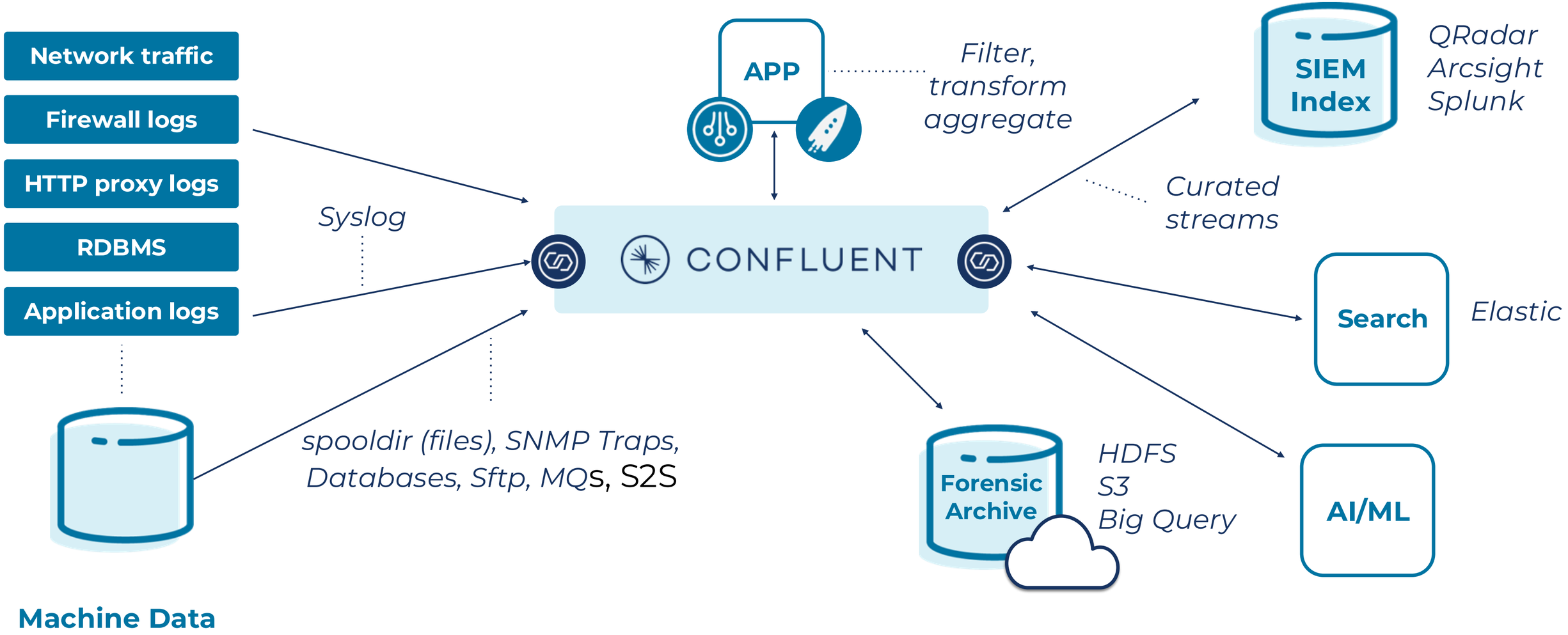
— Brent Conran, Vice President and CISO

Apache Kafka를 사용하여 현대적이고 확장 가능한 사이버 인텔리전스 플랫폼 구축

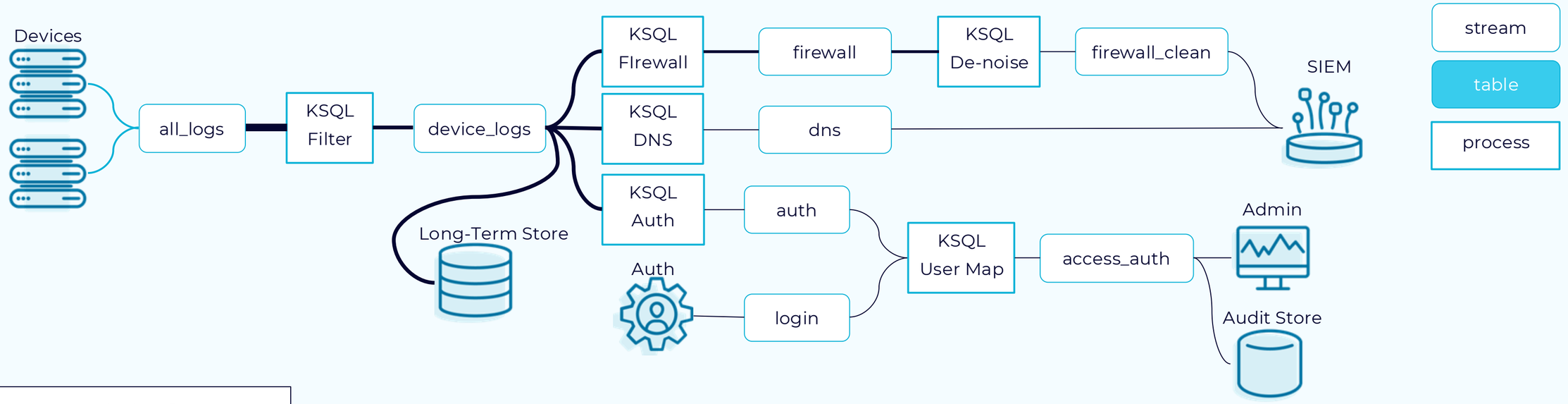


Kafka Summit Europe 2021

Intel의 SIEM 아키텍처 이해



SIEM에 대한 로그 유량 제어




```
CREATE STREAM device_logs AS
  SELECT
    event_id,
    event_type,
    EXPLODE (event)
    timestamp,
    ....
  FROM all_logs
  WHERE event_type = 'firewall'
    OR event_type = 'dns'
    OR event_type = 'auth'
EMIT CHANGES;
```

Event 타입별 구조화

```
CREATE STREAM auth AS
CREATE STREAM dns AS
CREATE STREAM firewall AS
  SELECT
    event_type,
    port,
    message,
    timestamp,
    ....
  FROM device_logs
  WHERE event_type = 'firewall'
EMIT CHANGES;
```

Event 타입별 스트림 라우팅

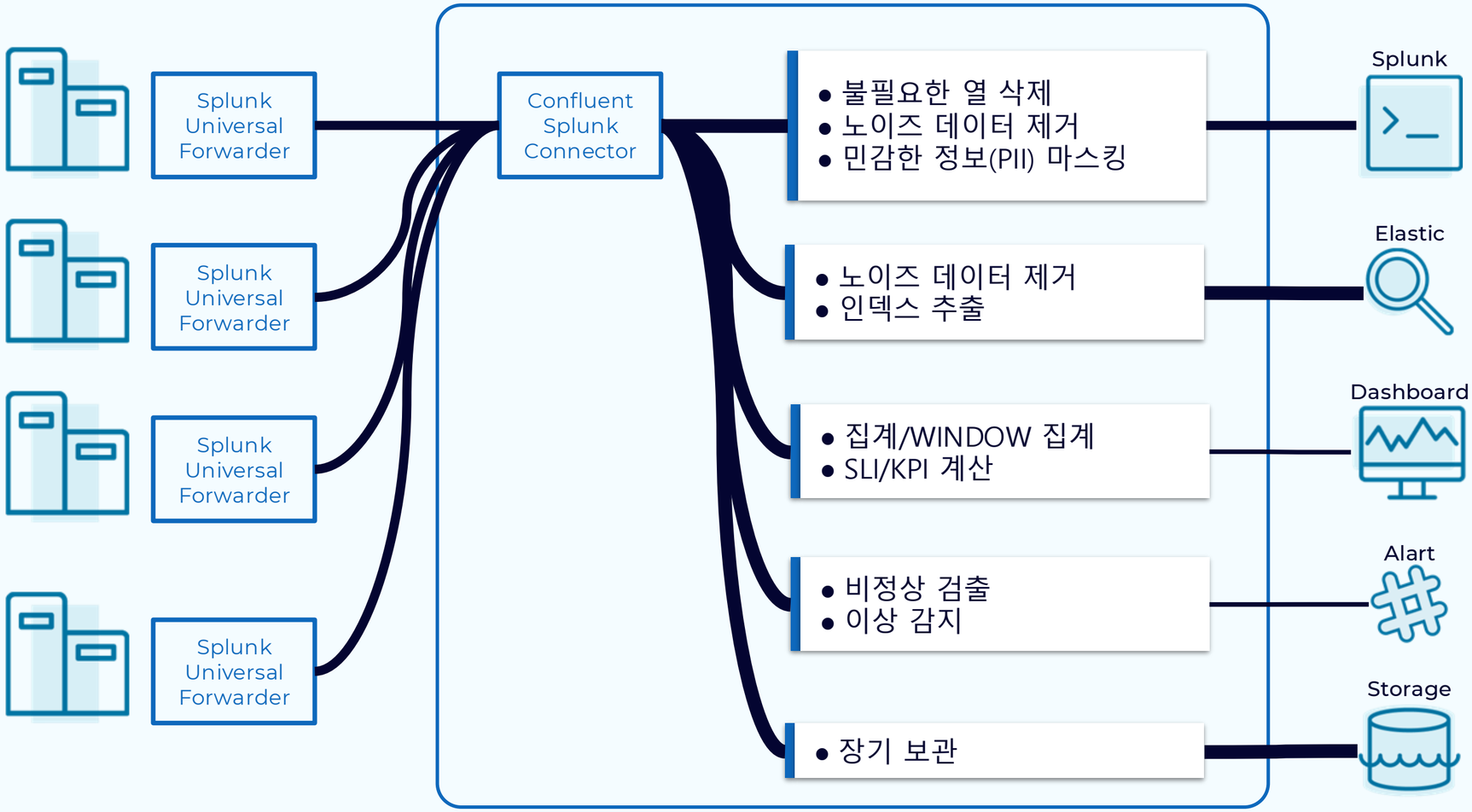
```
CREATE STREAM access_auth AS
  SELECT
    auth.auth_id,
    auth.auth_type,
    auth.timestamp,
    login.authorized_by
    login.timestamp,
    ....
  FROM auth
  OUTER_JOIN login
    ON auth.login_id = login.login_id
EMIT CHANGES;
```

로그인 아이디와 인증 아이디 매핑

SIEM 비용 절감

- 데이터 생성 서비스와 분석 서비스간 점대점 연결을 Kafka를 통해 1:N, N:M으로 변경
- 필터링, 매핑, 컬럼 드롭, 라벨링 등을 ksqlDB와 kafka Streams로 수행
- 데이터(로그)의 보관은 저렴한 오브젝트 스토리지로 대체(Tiered Storage). 분석 서비스의 데이터 감소로 효과적인 비용 절감

네트워킹 디바이스 데이터 (로그)의 유연하고 효율적인 사용



Splunk의 보안 기능은
활용하면서 필수가 아닌
데이터는 필터링

범용적인 검색 요구에는
인덱스를 추출해 Elastic로 검색
가능하게 변경

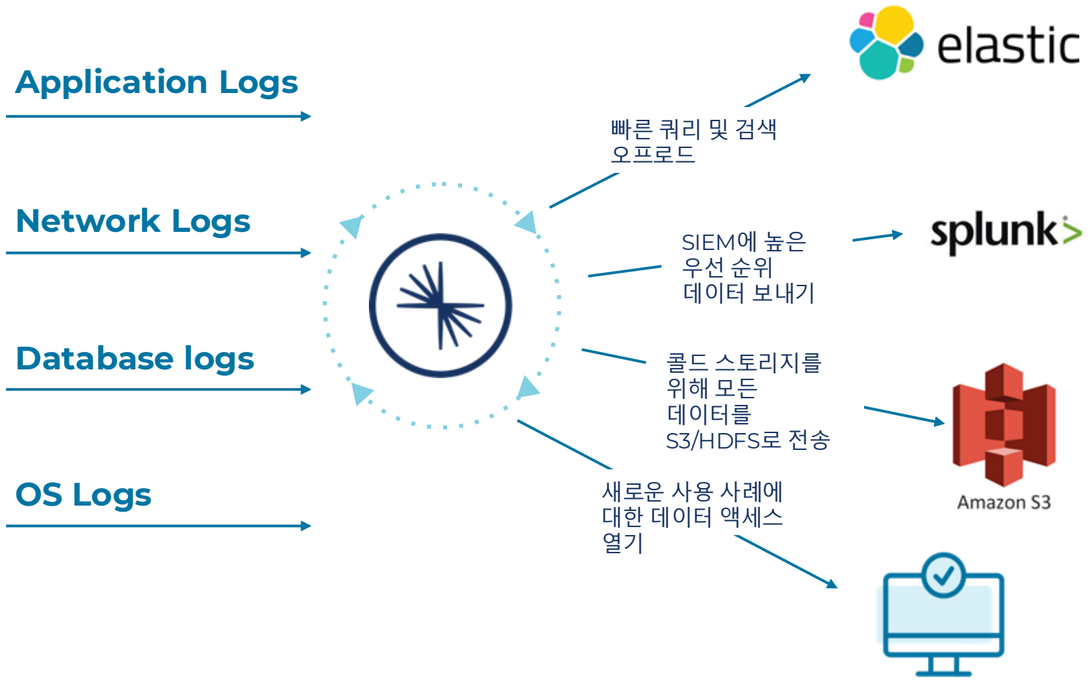
시간대별 집계나 특정 평가
지표의 추출 등을 대시보드화

위험성이 높은 에러나 이상
상태를 검출해 즉시 알람 통보

감사 목적 등의 장기 보존
스토리지에 불필요한 비용 절감.
필요에 따라 주문형으로 추출.

용어: SLI(서비스 수준 지표, Service Level Indicator), KPI(핵심 성과 지표Key Performance Indicator)

데이터 인덱싱, 분석 및 저장 비용 절감



모든 데이터 소스를 Confluent로 수집

이벤트 스트림을 필터링하고 우선 순위 이벤트만 SIEM으로 보냅니다.

SIEM 보유 기간 단축, 장기 보존을 위해 더 저렴한 저장소에 보관

수집 및 인덱싱 비용 절감

라이센스 지출을 줄이기 위해 이벤트를 집계하고 필터링하여 수집 및 색인 볼륨을 조절합니다.

전체 운영 비용 절감

Splunk S2S 커넥터가 있는 Splunk Heavy Forwarder와 같은 독점 포워더의 필요성 감소

저비용 스토리지에 데이터 보존

Pre-built Connector를 사용하여 장기 보존을 위해 데이터를 클라우드 스토리지 인프라로 계층화하고 라우팅합니다.

Confluent, AWS 및 Verizon을 활용한 스마트 팩토리(VZ Connect)



Verizon은 VZ Connect 서비스를 통해 제조/커넥티트 카 영역의 고객 서비스 제공

- 각 공장에서 발생된 센서/공정 데이터를 VZ Conenct를 통해 Cloud 환경으로 저지역 무선 연결을 통해 수집하고 Cluster Linking을 통해 분석 영역으로 데이터를 통합
- 각 공장별 별도의 On-prem 시스템 구축 및 운영을 최소화 하여 새로운 시스템 개발 및 반영에 신속성을 확보
- **CSP에서 제공하는 최신의 분석툴 활용을 최적화 빠른 인사이트와 공정 개선을 효율화 함**

Confluent는 기업 고객을 위한 최적화된 Kafka 솔루션을 제공합니다.

Confluent는 기업 조직에서 요구하는 높은 보안성과 안정성, 가용성 및 사용 편의성을 충족하는 Kafka 제품과 서비스를 모두 제공할 수 있는 유일한 기업입니다. 또한 Confluent는 다수의 글로벌 기업의 성공 사례를 기반으로 조직의 Kafka 도입에 최적화된 아키텍처와 기술지원을 제공해 드릴 수 있습니다.

Complete: Confluent는 기업 고객에서 요구하는 높은 수준의 안정성과 가용성을 충족하는 엔터프라이즈 Kafka를 제공합니다.

DEVELOPER	OPERATOR	ARCHITECT	EXECUTIVE
개발 생산성 극대화	**규모에 따른 효율적인 운영**	**운영 환경에 필수적인 기능**	**비즈니스 성공을 위한 파트너십**
풍부한 Pre-built 생태계 Connectors | Non-Java Clients | REST Proxy | MQTT Proxy	**GUI 기반 관리 및 모니터링** Cloud UI | Metrics API | Control Center | Health+	**엔터프라이즈 등급 보안** RBAC | Audit Logs | Encryption | BYOK | Private Networking	**완전한 Engagement Model** Data in Motion Blueprint
스트림 프로세싱 ksqlDB	**유연한 DevOps 자동화** Admin REST APIs | Terraform APIs | Confluent for K8s | Ansible Playbooks	**스트림 거버넌스** Schema Registry | Schema Validation | Stream Lineage | Stream Catalog	**비즈니스 케이스 근거** TCO | ROI | Risk

Cloud Native: *클라우드의 장점을 최대한 활용하도록 완벽하게 관리되고 재설계된 Apache Kafka©*

Elastic Scalability	Infinite Storage	High Availability
Expand | Shrink | Self-Balancing Clusters	Infinite Storage | Tiered Storage	99.99% SLA | Multi-AZ Clusters | Multi-Region Clusters

Everywhere: *온프레미스에서 클라우드 및 하이브리드 클라우드 전반에 걸쳐 제공되는 플랫폼을 통해 실시간으로 데이터 연결*

Fully managed cloud service	Self-managed software	Hybrid and Multi-cloud
AWS | Azure | GCP	Kubernetes | VMs | Bare Metal	Cluster Linking | Replicator

엔터프라이즈 지원 | 프로페셔널 서비스 | **커미터 중심의 전문성** | 교육 | 풍부한 파트너

Thank You!



CONFLUENT